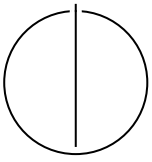
SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Mechanization of Synchronizability Theory for Communicating Automata with Mailbox Semantics

Nicole Kettler



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

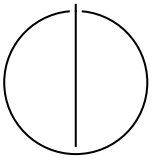
TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Mechanization of Synchronizability Theory for Communicating Automata with Mailbox Semantics

Mechanisierung von Synchronisierbarkeitstheorie für kommunizierende Automaten mit Mailbox-Semantik

Author:	Nicole Kettler
Examiner:	Prof. Dr. Dr. h. c. Javier Esparza
Supervisors:	Prof. Dr. Dr. h. c. Javier Esparza, Prof. Dr. Cinzia Di Giusto, Prof. Dr. Kirstin Peters
Submission Date:	14.10.2025



I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, 14.10.2025 Nicole Kettler

Acknowledgments

I'd first and foremost like to thank my supervisors for allowing me this opportunity and guiding me from beginning to end. I'd especially like to thank Cinzia Di Giusto and Kirstin Peters, who have tirelessly guided me through the whole process, provided feedback for my drafts and presentations, and are continuing to inspire me. A big thank you also to Javier Esparza, who has supervised me already for my Bachelor's thesis, and now for my Master's thesis as well.

Before starting my mobility in Nice, I was unsure if I would be able to coordinate a Master's thesis here. Thanks to both the Technical University of Munich and the Université Côte d'Azur, the universities' coordinators, all the professors who believed in me in my first semester in Nice, and all three of my supervisors working together across countries to make this possible, I was able to complete this thesis. I am forever glad to have had this opportunity, and am excited for what the future has to bring.

Next, I would like to thank my friends, both old and the new ones I've made in Nice. Thank you to Edoardo, the only other Erasmus student in the M2 Informatique Fondamentale, for sharing the same struggles and sticking with me throughout. Also, thank you to Nikan and all the other friends I have met this year; it was a pleasure, and a much-needed balance to this work.

I'd also like to thank my friends in Germany, who still kept in frequent contact with me despite the distance, and who motivated me both directly and indirectly. To highlight a few, thank you to Christoph-Alexander, who has often posed as the rubber duck in my debugging process, to Christian, who first introduced me to Isabelle and infected me with his passion for it, to ShuYu, who shared his knowledge of writing formally correct definitions among other math-related know-how with me, and to Felix, who aided with prettifying some of the LaTeX in this thesis.

I also want to extend my thanks to René Descartes, my favorite philosopher, whose famous quote "je pense, donc je suis" ("I think, therefore I am") still continues to inspire and motivate me every time it comes to mind.

Lastly, I want to thank my family, who lent me their support throughout my entire stay abroad and encouraged me, despite missing me plenty. I couldn't have a more supportive family if I asked for it, and I will be forever grateful, thank you.

Abstract

We study distributed systems, commonly modeled using communicating automata, where participants in a communication protocol are represented as finite state machines exchanging messages. These exchanges can be synchronous (instantaneous) or asynchronous (allowing for delays between sends and receptions, e.g. via buffers).

In previous work [DLP24], synchronizability was explored, a property ensuring that asynchronous communication does not introduce behaviors beyond those captured by synchronous communication. This paper introduced the Generalized Synchronizability Problem, using automata with final states, and showed that it is undecidable. However, decidability was found to be restored for specific communication topologies such as trees by proving that buffer languages in these settings are regular and computable.

In this thesis, we aim to mechanize the main proofs concerning this result from [DLP24] using the proof assistant Isabelle[1]. Throughout this verification process, we identify and correct several unnoticed faults in the original formalization. We propose several counterexamples that contradict the previous main theorem [DLP24, Theorem 4.5] and revise the theorem iteratively. Our contribution is a new formalization of the theorem claiming synchronizability is decidable for trees, and we prove that it does hold. We also provide a mostly complete Isabelle mechanization for this, where only some lemmas that intuitively hold are not fully machine-verified.

[1] `https://isabelle.in.tum.de`

Contents

1. Introduction

Distributed systems consist of multiple processes or devices that communicate with one another. Communication can occur either synchronously or asynchronously. In the former case, all messages are sent and received instantly without delay. In asynchronous systems, however, there can be. Commonly[1] in asynchronous systems, sent messages are stored in buffers and can be received from these at any point afterward, if at all. A plethora of different semantics and models have been proposed for distributed systems. We will use communicating automata as proposed by [BZ83] to model them, which are finite automata with specialized alphabets.

The messages of a synchronous system can be characterized only by what it can send, as every send must be paired with an immediate reception. In fact, the possible send sequences of synchronous systems are regular languages using our model of choice [BZ83]. Regular languages allow for efficient verification of specific properties by applying standard model-checking methods. In other words, checking whether a particular sequence of messages can occur in a synchronous system is straightforward.

Unfortunately, the same is not generally true for asynchronous systems, as they are Turing-equivalent [BZ83]. In fact, asynchronous systems can encode Turing machines with as few as two peers and two First In First Out (FIFO) buffers [BZ83]. Consequently, verification of asynchronous systems is generally undecidable [BZ83; BB16].

Real-world systems are more often than not asynchronous, so there is an objective to find some way to verify these despite the complexity involved. A common approach is to restrict asynchronous systems in some manner to make them more straightforward to analyze. Examples include restricting the size of the buffers involved or considering only synchronizable systems [GKM06; BB16]. The latter is a notion that has evolved over time and is defined slightly differently depending on the paper.

Generally speaking, a synchronizable system is an asynchronous system whose behavior can be related to that of a synchronous one. How exactly this relation is defined varies throughout the literature. Relevant for us is the notion that can also be referred to as "send-synchronizability" as is used in [DLP24] and [DMS25], where only the send sequences are considered. We receive these sequences, by gathering all message sequences (i.e. concatenations of words from any combination of participants in the network) the network can produce, and extracting only the sends from such

[1] Some asynchronous systems do not use buffers, but we do not explore these in this work.

a sequence [DLP24]. The idea of synchronizability is to find a synchronous system that produces the exact same send-sequences as the asynchronous one that we want to analyze. With a synchronizable system, the synchronous system can then be analyzed efficiently instead of the original asynchronous one, as their behaviors match.

Unfortunately, determining whether a system is synchronizable is generally undecidable for several buffer semantics [DLP24; BB16; DMS25; FL17]. Thus, further restrictions on systems are being researched, for example, regarding their topologies. The topology of a system refers to a directed graph modeling the flow of messages, i.e., a send message corresponds to an arrow in the graph. One such topology is that of trees, as explored in [DLP24].

While there are other topologies, such as rings, that are also explored in research [FL17], we will focus on the second result of [DLP24], which is a theorem that states synchronizability is decidable for mailbox systems that are trees and provides a concrete algorithm to decide it. To gather this result, they model asynchronous systems with communicating automata and consider all states in these automata to be final [DLP24]. A mailbox system is an asynchronous system with mailbox buffers, which means that each participant has their own unique buffer. All messages that get sent to a certain participant get stored in their respective mailbox, regardless of the sender's identity. This stands in contrast to e.g., Peer-to-Peer systems, where each participant (also called a peer) has one buffer per other peer.

Our goal is to mechanize the main theorem [DLP24, Theorem 4.5] that provides the algorithm to decide for a given mailbox system that is a tree, whether it is synchronizable. Their procedure performs some set operations using the set of reachable words in that system for each parent-child pair in the tree. The main idea is to check whether the child is able to receive every send the parent might do and whether some send-actions block receptions. The former is necessary to see if there is any message that can be sent but never received, whereas the latter checks that if something is sent, it can also be immediately received. If a send is blocking a reception, then a peer needs to send another message first before it can receive its parent's message, which cannot occur in any synchronous system. Depending on the results of these checks, the system's synchronizability (or lack thereof) can then be inferred directly. [DLP24]

Research on the different notions of synchronizability is particularly delicate. In the past, results on synchronizability have been published that later turned out to be flawed or even entirely incorrect, identified in follow-up work by other authors, e.g. [BB16; FL17]. Such incidents highlight how easy it is to make subtle mistakes in reasoning about synchronizability or to overlook some edge cases when working purely traditionally. Because of this, the authors of [DLP24] have expressed the aim of using a theorem prover to check their work, which provides the objective for this work.

As mathematical proofs grow in complexity, ensuring their correctness and ensuring

all cases are considered becomes a significant challenge. Theorem provers, or proof assistants, address this by machine-checking proofs that are formalized in a syntax matching the specific theorem prover used. Every logical inference is then verified through the machine, ensuring that a proof derives correctly from its axioms and definitions. This provides a very high level of assurance, as the entire chain of reasoning is rigorously validated. Each proof step needs to be sound in order to be accepted by the proof assistant. This is akin to compiling a program and checking for compilation errors, but instead operating on mathematical proofs. Machine-checking proofs requires all definitions and other required knowledge to be formalized in machine-readable syntax. We call the process of converting handwritten proofs, etc., into such a syntax the mechanization process.

Nowadays, there is a plethora of capable theorem provers to choose from, e.g. Rocq[2], Isabelle[3] or Lean[4]. Our three highlighted[5] instances have all been developed over the last decade(s) and thus have a good foundation of features to offer their users. We have chosen to utilize Isabelle in this work, as the authors of [DLP24] had already begun formalizing their results in Isabelle, and the TUM allows access to both experts and learning materials on Isabelle.

We can now begin detailing our chosen proof assistant, Isabelle. Isabelle's entire logic reduces to a small, verified kernel written in Standard ML [Mil+97]. All proof steps are ultimately reduced to primitive inferences checked by this kernel. This means that as long as the kernel is correct, any theorem the system accepts is logically sound [Pau88; Pau94]. This makes it suitable for our task of checking [DLP24, Theorem 4.5] for correctness. Similar to other theorem provers, Isabelle offers a functional programming-like environment for defining data types and functions [Pau94]. Properties of defined functions can be formally stated and proven within the system. Isabelle has some other inherent benefits, which will be elaborated on in a later section containing more technical details.

To summarize, we will formalize all needed definitions concerning communicating automata and synchronizability in Isabelle [Pau94] and then check whether a mailbox system with tree topology is synchronizable if and only if it satisfies the properties as proposed in [DLP24, Theorem 4.5]. In doing so, we aim to eliminate ambiguity and ensure that no reasoning gaps or edge cases remain in the source paper [DLP24].

[2] `https://rocq-prover.org`

[3] `https://isabelle.in.tum.de`

[4] `https://lean-lang.org`

[5] These choices are in no way meant to diminish other theorem provers. We chose these three, since Rocq (formerly Coq), Lean, and Isabelle should be among the most well-known. Isabelle also has deep ties with the Technical University of Munich (TUM) development-wise.

2. Related Work

This paper extends the work of [DLP24]; therefore, the related work remains largely the same, with the addition of some recent research. The paper [DLP24] forms the basis of this work and will be referred to as the source paper throughout this work. The goal of this work is to mechanize the claims of the source paper with Isabelle. Mechanization is the main topic; thus, the context of synchronizability and other closely related topics is relatively short. This is not to diminish the importance of the literature we will highlight, but this thesis focuses on our contribution, which does not need much more context other than the source paper. This is also partly due to the papers differing strongly in the definitions and approaches they use to reason about synchronizability.

As with the notion of synchronizability, distributed systems and their communication are also modeled in a plethora of ways. One of the more widely established methods is to model both synchronous and asynchronous communication protocols using communicating automata [BZ83], which are networks of finite state automata (the participants) where transitions can be interpreted as sending or receiving actions.

In synchronous communication, the sending and receiving of a message coincide, whereas in the asynchronous case that we consider[1], messages get sent to and received from buffers. In contrast to synchronous communication, the reception from buffers can happen at any time[2], and it is also possible that messages are sent but never received.

Several semantics have been proposed in the literature, e.g. [CMT96; CHQ16; Di +23], or most relevant for us, in [DLP24]. Different buffer semantics are also explored, i.e. how many buffers there are (per participant) and which messages get stored in which buffer. One of the most common system types analyzed in literature are Peer-to-Peer (P2P) systems [BB16; FL17]. We study mailbox systems, which are also a type of asynchronous systems, where each peer has their own unique FIFO buffer. In mailbox systems, messages to a certain peer get stored in its respective buffer (regardless of the sender's identity in the network), and it receives all its messages from this buffer [DLL20; DLP24; FL17]. This is in contrast to communication styles, such as Peer-to-Peer, where each peer has one buffer per other peer, resulting in a much greater number of buffers overall. In this work, we will exclusively focus on mailbox systems, since that is

[1] As mentioned in the introduction, we consider only asynchronous systems with buffers.

[2] The only requirement is that the message is in the correct place (meaning it is the next message to be read) of the buffer upon reception.

the system type that the theorem [DLP24, Theorem 4.5] we want to verify discusses, although most other research tends to reason about both.

If no restrictions are imposed upon an asynchronous system, its buffers are generally considered to be unbounded. This means that there is no limit on the number of messages that can be stored in each buffer, respectively. In conjunction with the buffers being FIFO, each peer can now receive its messages at any point after they have been sent, leading to an extreme amount of possibilities that have to be considered when analyzing such systems. Verification of a distributed system can thus quickly become infeasible. In fact, asynchronous systems can encode Turing machines with as few as two peers and two FIFO buffers [BZ83]. Consequently, verification of asynchronous systems is generally undecidable. [BZ83; BB16]

Synchronous systems, on the other hand, can be easily verified when modeled with communicating automata [BZ83]. Synchronous communication forces each sent message to be immediately received, allowing for all asynchronous cases (where the message is received any number of steps later, if at all) to be disregarded. Formally, the communication of a synchronous system can be captured by a regular language[3], which are straightforward to analyze using standard model checking techniques [BZ83; BB16; DLP24]. Since many real-world systems operate asynchronously, however, research is ongoing to find subclasses of such systems that can be verified, despite this being generally not possible for systems communicating asynchronously. To this end, asynchronous systems are generally constrained in some fashion. Two of the most prominent techniques are bounding the size of buffers and considering only systems that can be related in behavior to synchronous systems [DLP24; BB16; GKM06].

In asynchronous systems, messages are sent to buffers to be received at any point afterward. Without constraint, buffers are generally considered to be unbounded, i.e., there is no limit on the number of messages that can be stored in a buffer. When a limit is introduced, such as the one from [GKM06], we refer to such systems as B-Bounded, where B is some number defining the upper limit of messages that can be stored in a buffer. We want to highlight two types of such systems: the universally and existentially bounded systems with some specified bound B [GKM06]. The former kind are systems where each execution only requires buffers of size B, i.e., all possible executions of the system can be performed with buffers of size B [GKM06]. The existentially bounded systems are ones where the system is causally equivalent to one that only requires buffers of size B. Within this categorization of systems, some problems in the realm of model checking are decidable. Thus, if the bound B is known for a system, it can be verified. If it is unknown, however, deciding whether a bound exists for a given system

[3]This holds for the modelization with communicating automata. For other models, synchronous systems may be hard to analyze.

is undecidable for P2P [KM21] and mailbox systems [Bol+21]. [GKM06]

Moving away from bounded buffers, we now begin exploring some different notions of synchronizability. In [Bou+18], k-synchronizable systems are defined, where each execution is causally equivalent to one that consists of chunks with k messages each. Within such a slice, all sends must occur first, and then the receptions. Unfortunately, a certain buffer bound does not guarantee that there is a k s.t. the system is k-synchronous; and k-synchronizability does not imply any buffer bounds either [Bou+18]. Deciding whether a system is k-synchronizable is decidable for both P2P [Bou+18] and mailbox [DLL20] systems. Even if the k is not known, then for mailbox systems it is decidable whether a k exists such that it is k-synchronizable [DLL23]. For k-synchronizable systems, decision problems, such as whether a configuration is reachable, are decidable [Bou+18].

On k-synchronizable systems, the causal equivalence relation [DLL20; Bou+18] stems from the *happened before* relation [Lam78]. This ensures that actions are performed in the same order, from the point of view of each participant, and consequently allows for comparing and grouping executions into classes.

The last group of systems we want to highlight, as pioneered by the authors of [BB16], are the synchronizable systems. Such systems are asynchronous systems whose behavior can be directly related to that of a synchronous one. Unlike k-synchronizability, this does not rely on causal equivalence; instead, it analyzes only the send traces, which are executions projected onto only send actions, meaning receptions are disregarded [BB16; DLP24; DMS25]. In particular, the actions may be performed in a different order than that of a k-synchronizable system (which would render the execution not causally equivalent) [DLP24]. Due to this distinction, systems belonging to either of these two groups of (k-)synchronizability are incomparable with one another.

Continuing with synchronizability, the authors of [BB16] propose a procedure to determine whether a (mailbox or P2P) system is synchronizable. For their procedure, they compare the set of send sequences (traces) of the synchronous system to the set of 1-bounded traces of the asynchronous system [BB16]. Intuitively, the given asynchronous system (with any buffer size) gets constrained to buffers of size 1 and is then analyzed in terms of what the system can still send. If the 1-bounded sends of the asynchronous system are the same as the synchronous ones, the procedure from [BB16] considers the asynchronous system synchronizable.

Unfortunately, this result was disproven in [FL17], for both P2P and mailbox, by counterexamples where the 1-bounded traces imply synchronizability, but the 2-bounded ones contradict it. In other words, the 2-bounded traces contained instances that were not contained in the set of 1-bounded traces, but which could not be recreated synchronously for the provided network [FL17]. Additionally, they show that checking synchronizability of a P2P system is undecidable in [FL17, Theorem 3], leaving the

problem open for mailbox systems.

In [DLP24], this open problem is studied by relaxing one of the hypotheses in [FL17] and instead considering communicating automata with final accepting states, which is a more general case in this field of research. Communicating automata are generally considered to have no final states (or equivalently, all states are considered final). Using networks with sets of accepting states that are neither empty nor universal, the authors in [DLP24] prove that synchronizability is undecidable.

Since then, the synchronizability of `mbox` systems has been investigated further, and recently it was shown that synchronizability is undecidable for `mbox` systems, even in the general case where final states of automata are not constrained [DMS25]. This is unfortunate, as synchronizable systems are one of the subsets of asynchronous systems that can be verified. Hence, research is ongoing to restrict asynchronous systems like the `mbox` systems to groups where synchronizable ones can be identified.

One way to restrict systems is to consider only those with certain topologies, as is done in e.g. [DLP24] and [FL17]. In [DLP24], the authors investigate how tree-related topologies may affect the decidability of the synchronizability problem. Such a topology is based on the flow of messages in the network, i.e., the topology shows for each peer, to which peers they send and from which they receive messages. [DLP24] The aim is to find topologies where synchronizability becomes decidable again. In [FL17, Theorem 11], synchronizability was shown to be decidable for oriented ring topologies. In [DLP24], the encoding from [FL17] is adapted to mailbox systems, and a check to determine synchronizability of trees is proposed using this adaptation. They reach this decision procedure for trees by proving that the buffer language is regular and computable [DLP24]. Using this, they analyze the content of buffers, rather than the more indirect approach of comparing sets of send traces as in [BB16]. They conclude their findings with [DLP24, Theorem 4.5], which shows that synchronizability is decidable if the topology of a system is a tree.

It is now clear that researching the decidability of (k-)synchronizability is extremely complex, sometimes leading to human errors in the process. In fact, two of the works we mentioned in this section had flaws that other authors later corrected. The first instance is [Bou+18], whose procedure for deciding reachability in k-synchronizable systems was corrected in [DLL20]. The other instance is the wrong claim of [BB16] concerning the decidability of synchronizability, which was disproven in [FL17].

One way to prevent such mistakes from making it to published work is to use theorem provers such as Isabelle [Pau94] to automatically verify proposed theorems and their proofs. Such a tool may help identify edge cases and otherwise make sure all proof steps are formally correct and no steps are missed. Unfortunately, few notions related to synchronizability or communicating automata in general are publicly available

in Isabelle's Archive of Proofs[4]. This archive contains formalizations and proofs of various disciplines and topics, made publicly available by the respective authors in this manner. Concerning distributed systems, this archive contains many proofs unrelated to synchronizability, for example [FT20], which concerns a very specific, but unrelated property that they show to be verifiable for a certain type of distributed systems. Another work formalizes communicating automata, but ones that use stimuli and consequently completely different syntax than the ones we use to reason about synchronizability [BJ19]. This exemplifies the unfortunate gap between the need for verification related to synchronizability topics, especially, and the lack of formalization frameworks available for accomplishing it. This means that to verify any claim, one would first need to formalize the framework, including communicating automata, to begin with the truly relevant parts of the verification. This work will make a first step to bridge this gap, and attempt to formalize the claims concerning the synchronizability of trees from [DLP24] with the theorem prover Isabelle [Pau94].

[4] `https://isa-afp.org`

3. Preliminaries

This section will detail all the necessary definitions and results from the source paper that are needed for the remainder of this work. This work exclusively relies on synchronous systems, as well as on mailbox systems with FIFO buffers and tree topologies, and thus only those will be introduced. However, it should be noted that both in the source paper and in most other related literature, additional topologies and communication protocols are routinely explored. To summarize, our preliminaries here consist of the relevant parts in the "Preliminaries" section of the source paper, as well as some of their fourth section "Synchronisability of Mailbox Communication for Tree-like Topologies". The latter of which contains the claims and results we will attempt to verify throughout this work. Some sections will be direct citations from the source paper, as we necessarily operate on the same definitions. [DLP24]

3.1. Communicating Automata and Formal Languages

For a finite set Σ, a word $w = a_1 a_2 \dots a_n \in \Sigma^*$ is a finite sequence of symbols, such that $a_i \in \Sigma$, for all $1 \leq i \leq n$. We denote the concatenation of two words w_1 and w_2 with $w_1 \cdot w_2$, or simply $w_1 w_2$ when it is clear enough where one word starts and the other begins. The empty word is ε.

For the remainder of this work, some knowledge of non-deterministic finite state automata is necessary. A communicating automaton A is a finite state machine that performs actions of two kinds: either sends or receptions, which we may also refer to as outputs or inputs, respectively. $\mathcal{L}(A)$ denotes the language accepted by automaton A.

A *network* N of communicating automata, or simply a network, is the parallel composition of a finite set $\mathbb{P}$ of participants (commonly referred to as peers) that exchange messages from a finite message set $\mathbb{M}$. From here on, we consider only automata - and hence networks - where all states are final. We denote $a^{p \to q} \in \mathbb{M}$ the message sent from peer p to q with the finite payload a. Note that $p \neq q$, as we consider only messages with a distinct sender and receiver, respectively. An action is the sending $!m$ or the reception $?m$ of a message $m \in \mathbb{M}$. In cases where the sender and receiver of a message are apparent (or unnecessary), we may omit them and shorten $!a^{p \to q}$ to $!a$ (resp. $?a^{p \to q}$ to $?a$).

Definition 3.1.1 (Network of Communicating Automata)**.** $N = ((A_p)_{p\in\mathbb{P}}, \mathbb{M})$ is a network of communicating automata, where:

1. for each $p \in \mathbb{P}$, $A_p = (S_p, s_0^p, \mathbb{M}, \rightarrow_p, F_p)$ is a communicating automaton with S_p is a finite set of states, $s_0^p \in S_p$ the initial state, $\rightarrow_p \subseteq S_p \times Act_p \times S_p$ is a transition relation, and F_p a set of final states and
2. for each $m \in \mathbb{M}$, there are $p \in \mathbb{P}$ and $s_1, s_2 \in S_p$ such that $(s_1, !m, s_2) \in \rightarrow_p$ or $(s_1, ?m, s_2) \in \rightarrow_p$.

To identify the flow of messages and hierarchy between peers of a network, we define the topology as a graph with edges from senders to receivers. We use standard notions for graphs in the following two definitions concerning network topologies.

Definition 3.1.2 (Topology)**.** Let $N = ((A_p)_{p\in\mathbb{P}}, \mathbb{M})$ be a network of communicating automata. Its topology is an oriented graph $G(N) = (V, E)$, where $V = \{p \mid p \in \mathbb{P}\}$ and $E = \{(p, q) \mid \exists a^{p\rightarrow q} \in \mathbb{M}\}$.

We will focus exclusively on tree topologies, where each of the inner automata (nodes) receives messages from exactly one other peer. This peer is then considered their unique direct parent. In contrast to nodes, the root receives messages from no other peer, and also no peer can send any message to it. This distinction between sending and receiving is necessary, as in asynchronous networks, messages may be sent but never received. The source paper denotes $\mathbb{P}^p_{send}$ (resp. $\mathbb{P}^p_{rec}$) as "the set of participants sending to (resp. receiving from) p". We will specify this notation further in Subsection 4.1, but we will first continue with the definitions as they were originally introduced in [DLP24].

Definition 3.1.3 (Tree Topology)**.** Let $N = ((A_p)_{p\in\mathbb{P}}, \mathbb{M})$ be a network of communicating automata and $G(N) = (V, E)$ its topology. $G(N) = (V, E)$ is a tree if it is connected, acyclic, and $\mid \mathbb{P}^p_{send} \mid \leq 1$ for all $p \in \mathbb{P}$.

For networks, many different communication mechanisms and corresponding semantics exist. A *system* combines a network with a communication mechanism. We consider only synchronous systems $N_{\texttt{sync}}$, as well as the asynchronous mailbox systems $N_{\texttt{mbox}}$. In synchronous communication, each send necessitates an immediate reception, whereas in the asynchronous kind, messages are sent to and received from memory (buffers). Here we only consider FIFO buffers, which can be bounded or unbounded. Thus, asynchronous messages must be received in the same order that they are sent in (or not received at all). As an overview, we refer mostly to the following two systems throughout this thesis:

- Synchronous (`sync`): there are no buffers in the system, messages are immediately received when they are sent;
- Mailbox (`mbox`): each peer has their own unique buffer, each peer receives all its messages in this buffer, regardless of who sent it.

We use *configurations* to describe the state of a system and its buffers.

Definition 3.1.4 (Configuration)**.** Let $N = \big((A_p)_{p\in\mathbb{P}}, \mathbb{M}\big)$ be a network. A `sync` configuration (respectively a `mbox` configuration) is a tuple $C = \big((s^p)_{p\in\mathbb{P}}, \mathbb{B}\big)$ such that:

- s^p is a state of automaton A_p, for all $p \in \mathbb{P}$
- $\mathbb{B}$ is a set of buffers whose content is a word over $\mathbb{M}$ with:
  - an empty tuple for a `sync` configuration,
  - and a tuple $(b_1, \ldots, b_n)$ for a `mbox` configuration.

We write ε to denote an empty buffer, and $\mathbb{B}^\emptyset$ to denote that all buffers are empty. We write $\mathbb{B}\{b_i/b\}$ for the tuple of buffers $\mathbb{B}$, where b_i is substituted with b. The set of all configurations is $\mathbb{C}$, $C_0 = \big((s_0^p)_{p\in\mathbb{P}}, \mathbb{B}^\emptyset\big)$ is the *initial configuration*, and $\mathbb{C}_F \subseteq \mathbb{C}$ is the set of *final configurations*, where $s^p \in F_p$ for all participants $p \in \mathbb{P}$. Due to the source paper considering only automata where all states are final, we will apply the same restriction within this work. Thus, in our case, we consider all configurations to be final.

We describe the behavior of a system with *runs*. A run is a sequence of transitions starting from an initial configuration C_0. Let $\mathtt{com} \in \{\mathtt{sync}, \mathtt{mbox}\}$ be the type of communication. We define $\xrightarrow[\mathtt{com}]{}{}^*$ as the transitive reflexive closure of $\xrightarrow[\mathtt{com}]{}$.

Without loss of generality, we label the transition in the following definitions of executions and traces with the sending message $!a^{p\to q}$ for brevity.

In a synchronous communication, we consider the send and receive actions to occur together without delay; thus, the synchronous relation `sync`-send merges these two actions.

Definition 3.1.5 (Synchronous System)**.** Let $N = \big((A_p)_{p\in\mathbb{P}}, \mathbb{M}\big)$ be a network. The synchronous system $N_{\mathtt{sync}}$ associated with N is the smallest binary relation $\xrightarrow[\mathtt{sync}]{}$ over `sync`-configurations such that:

$$(\mathtt{sync\text{-}send})\ \frac{s^p \xrightarrow{!a^{p\to q}}_p s'^p \qquad s^q \xrightarrow{?a^{p\to q}}_q s'^q}{\big((s^1,\ldots,s^p,\ldots,s^q,\ldots,s^n), \mathbb{B}^\emptyset\big) \xrightarrow[\mathtt{sync}]{!a^{p\to q}} \big((s^1,\ldots,s'^p,\ldots,s'^q,\ldots,s^n), \mathbb{B}^\emptyset\big)}$$

Definition 3.1.6 (Mailbox System)**.** Let $N = \big((A_p)_{p\in\mathbb{P}}, \mathbb{M}\big)$ be a network. The mailbox system $N_{\texttt{mbox}}$ associated with N is the smallest binary relation $\xrightarrow[\texttt{mbox}]{}$ over mbox-configurations such that for each configuration $C = \big((s^p)_{p\in\mathbb{P}}, \mathbb{B}\big)$, we have $\mathbb{B} = (b_p)_{p\in\mathbb{P}}$ and $\xrightarrow[\texttt{mbox}]{}$ is the smallest transition such that:

$$\text{(mbox-send)}\ \frac{s^p \xrightarrow{!a^{p\to q}}_p s'^p}{\big((s^1,\dots,s^p,\dots,s^n),\mathbb{B}\big) \xrightarrow[\texttt{mbox}]{!a^{p\to q}} \big((s^1,\dots,s'^p,\dots,s^n),\mathbb{B}\{b_q/b_q\cdot a\}\big)}$$

$$\text{(mbox-rec)}\ \frac{s^q \xrightarrow{?a^{p\to q}}_q s'^q \qquad b_q = a\cdot b'_q}{\big((s^1,\dots,s^q,\dots,s^n),\mathbb{B}\big) \xrightarrow[\texttt{mbox}]{?a^{p\to q}} \Big((s^1,\dots,s'^q,\dots,s^n),\mathbb{B}\{b_q/b'_q\}\Big)}$$

To reason about the behavior of the introduced systems, we now define executions and traces. An execution e is a sequence of actions, and the corresponding trace t is the projection of this sequence on only the send actions.

Definition 3.1.7 (Execution)**.** Let $N = \big((A_p)_{p\in\mathbb{P}}, \mathbb{M}\big)$ be a network and $\texttt{com} \in \{\texttt{sync}, \texttt{mbox}\}$ be the type of communication. $\mathsf{E}(N_{\texttt{com}})$ is the set of executions, with C_0 as the initial configuration, C_n a final configuration and $a_i \in Act$ for all $1 \le i \le n$, by:

$$\mathsf{E}(N_{\texttt{com}}) := \{a_1 \cdot \ \dots\ \cdot a_n \mid C_0 \xrightarrow[\texttt{com}]{a_1} C_1 \xrightarrow[\texttt{com}]{a_2} \ \dots\ \xrightarrow[\texttt{com}]{a_n} C_n\}.$$

For a given word w over *actions*, let $w{\downarrow_!}$ (resp. $w{\downarrow_?}$) be its projection on only send (resp. receive) actions, let $w{\downarrow_P}$ be its projection on actions that involve only the participants in a set P, let $w{\downarrow_p}$ be its projection on sends from p (to some peer) and receptions of p (of messages from other peers), and let $w{\downarrow_{\not{!}\not{?}}}$ denote the word over *messages* in w, which is the result of removing all !- and ?-signs. We extend the operators $\downarrow_!$, $\downarrow_?$, $\downarrow_P$, $\downarrow_p$, and $\downarrow_{\not{!}\not{?}}$ to languages, by applying them on every word of the language. Note that $w{\downarrow_{\{p\}}}$ is always empty, since each action involves two distinct peers and never just p, whereas $w{\downarrow_p}$ is the projection of w to actions in which p has an active role (sender in send actions and receiver in receive actions).

Definition 3.1.8 (Trace)**.** Let $N = \big((A_p)_{p\in\mathbb{P}}, \mathbb{M}\big)$ be a network and $\texttt{com} \in \{\texttt{sync}, \texttt{mbox}\}$ be the type of communication. $\mathsf{T}(N_{\texttt{com}})$ is the set of traces:

$$\mathsf{T}(N_{\texttt{com}}) := \{e{\downarrow_!} \mid e \in \mathsf{E}(N_{\texttt{com}})\}.$$

In this Definition 3.1.8, like the source paper [DLP24], we do not consider the content of buffers, in contrast to other works like [FL17], where the authors study stable configurations (where all buffers are empty). The traces only consider the different send-sequences a system can produce, which are gathered by projecting all executions to only the send-actions in them.

3.2. Synchronizability

We have provided all the general definitions of the source paper, and now move on to more in-depth definitions and notions needed for reasoning about such systems. A system is synchronizable if its asynchronous behavior can be related to its synchronous one. Thus, an asynchronous system is synchronizable if its set of traces is the same as the one obtained from the synchronous system. Formally, if $\mathsf{T}(N_{\mathtt{mbox}}) = \mathsf{T}(N_{\mathtt{sync}})$ for a given network, i.e. if the send-sequences (the projections of executions to only send-actions) are equivalent.

We define the *Synchronizability Problem* as the decision problem of determining whether a given system is synchronizable or not. The authors in [DLP24] have shown that the *Generalized Synchronizability Problem* (where states are not constrained in any way) is undecidable for mailbox systems. In recent research, [DMS25] have shown that synchronizability is also undecidable for networks where each state is final, or equivalently, where no state is final. As the second major contribution of [DLP24], they claim that synchronizability is decidable for mailbox systems with tree topologies, which we will aim to verify in this work. To summarize, synchronizability is undecidable for mailbox systems in general, but has been determined to be decidable if such a system has a tree topology. [DLP24; DMS25]

To illustrate the notion of synchronizability in mailbox systems, we analyze Figure 3.1 in the following two examples.

Example 3.2.1 (Synchronizability and Missing Receptions)**.** In the network in Figure 3.1, q can send $!b^{q\to p}$ but p cannot receive $?b^{q\to p}$. Since $!b^{q\to p}$ can occur from the initial state of A_q, q does not depend on any other peer to perform it, and hence $!b^{q\to p} \in \mathsf{E}(N_{\mathtt{mbox}})$. However, $!b^{q\to p} \notin \mathsf{E}(N_{\mathtt{sync}})$, as sending $!b$ requires the reception $?b$ to happen as well. This is not possible here, as $?b^{q\to p} \notin \mathcal{L}(A_p)$. Since $!b^{q\to p} \in \mathsf{E}(N_{\mathtt{mbox}})$ consists only of sends, it is also a trace $!b^{q\to p} \in \mathsf{T}(N_{\mathtt{mbox}})$ of the asynchronous system. We can infer $!b^{q\to p} \in \mathsf{T}(N_{\mathtt{mbox}}) \neq \mathsf{T}(N_{\mathtt{sync}})$ and therefore that this network is not synchronizable.

We can observe that the absence of receptions can hinder the synchronizability of a network. In cases where the respective message is never sent, there is no issue. There is another property of this network that is hindering its synchronizability, which we will elaborate on in the following example.

Example 3.2.2 (Synchronizability and Blocking Sends)**.** Consider the network from Figure 3.1 again. We established that peers cannot block one another from sending messages, as the automata in a network work independently from one another. It follows that $!a^{r\to q}.!b^{q\to p} \in \mathsf{E}(N_{\mathtt{mbox}})$ is also a valid execution of this network. The only possible synchronous execution with the same trace $!a^{r\to q}.!b^{q\to p}$ is $!a^{r\to q}.?a^{r\to q}.!b^{q\to p}.?b^{q\to p} \notin$

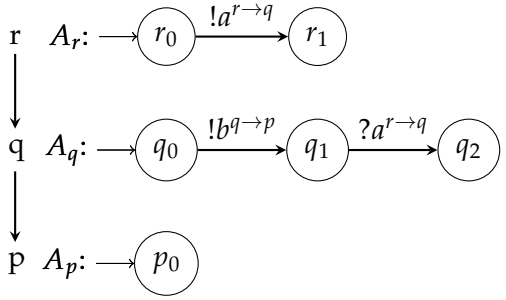


Figure 3.1.: A network that is not synchronizable because of A_q and A_p, respectively.

$\mathsf{E}(N_{\mathtt{sync}})$. This cannot be executed in our network, since q cannot perform $?a^{r\to q}$ before sending $!b^{q\to p}$. We conclude that the $!b^{q\to p}$ blocks a reception that is needed for the synchronous communication of the given trace, thus the network is not synchronizable.

We may observe that having dependencies in a network, specifically those that force a send of some peer to happen before a reception of some other peer can, may also hinder the synchronizability.

The authors of [DLP24] take both of the observations we showcased in Examples 3.2.1 and 3.2.2 into account and apply them in their main theorem, which we will detail in the next chapter. We want to restate their notion of the shuffled language here, as we have briefly motivated it with the previous example. The idea of shuffling is to ensure that receptions can be moved further left in a word, without impacting the order of the receptions (and sends, respectively). A more thorough explanation about the reasoning behind this construction, as well as an additional example[1], can be found in the source paper [DLP24]. The word w' is a *valid input shuffle of* w, denoted as $w' \sqcup\!\!\sqcup_? w$, if w' is obtained from w by a (possibly empty) number of swappings that replace some $!x?y$ within w by $?y!x$. In other words, receptions can be shuffled further left in a word (and thus sends further right), but the order of both (respectively) is preserved.

Definition 3.2.1 (Shuffled Language)**.** Let $p \in \mathbb{P}$. We define the shuffled language of $\mathcal{L}^{\lozenge}(A_p) \subseteq \mathcal{L}(A_p)$ as follows:

$$\mathcal{L}^{\lozenge}_{\sqcup\!\!\sqcup}(p) := \left\{ w' \mid w \in \mathcal{L}^{\lozenge}(A_p) \wedge w' \sqcup\!\!\sqcup_? w \right\}$$

Example 3.2.3 (Shuffled Language Demonstration)**.** Going back to Figure 3.1, we can observe $\mathcal{L}(A_q) = \{\varepsilon, !b^{q\to p}, !b^{q\to p}\cdot?a^{r\to q}\}$. The only word in this language that can be shuffled is $!b^{q\to p}\cdot?a^{r\to q} \in \mathcal{L}(A_q)$, in which we can swap both actions exactly

[1]Note that we found a small typo in [DLP24, Example 4.3]: $!b^{q\to p}?a^{r\to q}$ should not be in the shuffled language, since shuffles can only occur in one direction and not be reversed.

once to receive $?a^{r\to q}.!b^{q\to p}$. We can conclude that the shuffled language of $\mathcal{L}(A_q)$ is $\mathcal{L}(A_q) \cup \{?a^{r\to q}.!b^{q\to p}\} = \{\varepsilon, !b^{q\to p}, !b^{q\to p}.?a^{r\to q}, ?a^{r\to q}.!b^{q\to p}\}$.

As is illustrated in Example 3.2.3, the reception $?a^{r\to q}$ that q needs to perform the trace $!a^{r\to q}.!b^{q\to p}$ synchronously is not present in $\mathcal{L}(A_q)$, but is present in its shuffled language. This indicates that a reception is blocked by a send, i.e. before the reception can occur, a different send must be performed first. In turn, we may conclude that this network is not synchronizable. It should be noted that the shuffled language alone does not always suffice to prove or disprove the synchronizability of a network.

We stated that shuffling does not change the order of receptions or sends, respectively. Formally, for any shuffle $w' \sqcup\!\sqcup_? w$, we can infer from the construction that $w'\downarrow_? = w\downarrow_?$ and $w'\downarrow_! = w\downarrow_!$ hold.[2] In particular, a word w can be shuffled until all receptions precede the sends, i.e. $(w\downarrow_? \cdot w\downarrow_!) \sqcup\!\sqcup_? w$; we call this process a *full shuffle*, or *fully shuffling* w. If $\mathcal{L}(A_p)$ does not include the full shuffle of one of its words w, that indicates that there is some $!a$ that occurs before some $?b$ in w. This dependency, that the send must come before the reception in w, may have implications for the network. We demonstrated this in Example 3.2.3, where q could not perform the full shuffle $?a^{r\to q}.!b^{q\to p}$ of its word $!b^{q\to p}.?a^{r\to q} \in \mathcal{L}(A_q)$, rendering the network not synchronizable.

We will revisit this notion at a later point, when we have introduced the influenced language $\mathcal{L}^{\lozenge}(A_p)$ for a peer p. Note that one can shuffle any language, but the main theorem applies it exclusively to the influenced language, which will be formally introduced in Chapter 4, as we have revised it within this work.

We have now introduced all definitions, which we will need in the coming parts, but which we have not altered in their formalization, throughout the mechanization process. The coming chapter will detail all remaining definitions, as well as the main theorem, which have all been adjusted and/or clarified throughout our work.

3.3. The Proof Assistant Isabelle

We would like to highlight some code excerpts of our formalization [Ket25] in the coming chapters, so we will provide a short overview of our chosen proof assistant and how it can be applied. Where not explicitly stated otherwise, all code boxes shown throughout this work are from our code in [Ket25] and were exported to LaTeX by using Isabelle's export functionality. In the remaining cases that we will specify, the code is gathered from the initial formalizations we were provided by the authors of [DLP24].

As alluded to in the introduction, theorem provers are extremely useful to ensure the correctness of handwritten proofs. They can be applied by first mechanizing (i.e.,

[2] We have proven this in Isabelle in lemmas *shuffle_keeps_send_order* and *shuffle_keeps_recv_order*.

rewriting handwritten definitions, lemmas, and their proofs into machine-readable syntax) and then running the theorem prover. The theorem prover then either approves the contents, gives warnings, or does not compile, or, in some cases, directly finds and provides counterexamples if the content is incorrect. Warnings, errors, or endless loops of any type (if no explicit counterexample was found) can indicate that either a step may be incorrect, missing, or the entire hypothesis is faulty as it is written down. We will now detail how Isabelle [Pau94] can be used to accomplish this and some of its advantages.

One of the benefits to Isabelle is its *sledgehammer* functionality [Pau94]. It makes use of Automatic Theorem Provers (ATPs), such as agsy-HOL [Lin14], as well as Satisfiability-Modulo-Theories (SMTs) such as cvc5 [Bar+22], and automatically applies them to find proofs [Bla+25; BP10]. In this process, Isabelle uses all currently known and provided lemmas and then pipes them into all ATPs and SMTs that are chosen in its settings, together with the to-be-proven instance. If one of the tools finds a proof, it is piped back into Isabelle, which then reconstructs the found proof into Isabelle syntax. If this conversion back to Isabelle succeeds, Isabelle trusts the result and *sledgehammer* has provided a result that may be continued with in Isabelle [NK23; Bla+25]. This means, in particular, Isabelle does not rely on the correctness of the ATPs and SMTs it delegates to, since the reconstruction back to Isabelle (and its trusted kernel [Mil+97]) is mandatory for *sledgehammer* to succeed [NK23]. If all needed lemmas are provided, it is possible that *sledgehammer* can find a proof. The success rate is higher the more context is provided, since even the smallest logical inferences need to be explicitly proven, or Isabelle will not accept it as formally correct. In practice, that makes *sledgehammer* a useful tool, especially when proving smaller results, for example, the commutativity of some simple self-defined datatype. For entire theorems, however, a large remainder of unautomated work is left for the user. In particular, all the necessary lemmas for use with *sledgehammer* need to first be identified and manually formalized and then proven in some way. For this thesis, we have utilized *sledgehammer* with the default settings. [Pau94; NK23; Bla+25; BP10]

Apart from this functionality simplifying the process of proving, data types and functions can be defined in a fashion similar to the syntax of common functional programming languages like Haskell[3] or OCaml[4]. In the remainder of this section, we will showcase parts of our mechanizations for the definitions we have already introduced. We assume the reader has a foundational understanding of functional programming languages, so we will not elaborate on basic syntax like the typing in the declarations or syntactic sugar for lists, and instead refer to learning material such as

[3]https://www.haskell.org
[4]https://ocaml.org

the great work of Miran Lipovača [Lip11] for any uncertainties in that regard. As a first code example, we show the formalization of messages in Example 3.3.1, as provided to us from the authors of [DLP24].

Example 3.3.1 (Formalizing Messages in Isabelle)**.** For example, we can formalize the syntax for messages from Section 3.1 as well as simple getter functions. One way to do this is shown in Figure 3.2, which is an excerpt from the original formalization by [DLP24]. Here, *primrec* is a constrained version of *fun*, which can be used to define any kind of function, not just primitive recursive ones. The *datatype* keyword denotes the definition of some custom datatype. In this example, the types *'information*, *'peer* are used in the definition of *message*, as a message must contain the sender, receiver, and some information that is being communicated. One can also define custom syntax, as is done here with the arrow and the wildcards "_" for the three parameters, respectively.

Figure 3.2.: Formalization of Messages

```
datatype ('information, 'peer) message =
  Message 'information 'peer 'peer  ("_-→-" [120, 120, 120] 100)

primrec get_information :: "('information, 'peer) message ⇒ 'information" where
  "get_information (i^p→q) = i"

primrec get_sender :: "('information, 'peer) message ⇒ 'peer" where
  "get_sender (i^p→q) = p"

primrec get_receiver :: "('information, 'peer) message ⇒ 'peer" where
  "get_receiver (i^p→q) = q"
```

Figure 3.3 contains the formalizations for the valid input shuffle and the shuffled language, as they are introduced in Definition 3.2.1 and directly above it. A word w is always a shuffle of itself, or exactly one swap of some $!x$ and $?y$ to $?y$ and x can occur, or w can be shuffled an arbitrary number of times to yield some w'. These three distinct rules of the shuffle-relation are formalized as *refl*, *swap*, and *trans* in the *inductive shuffled*. The keyword *inductive* works similarly to a function, but it is often more straightforward to prove positive properties using this keyword instead, since the prerequisites can be specified directly within the *inductive*. On the other hand, proving negative properties, like e.g. a certain w' *not* being a shuffle of some w, is generally harder using this keyword. Since we only consider *shuffled languages* and not ones containing no or only certain shuffles, using an *inductive* for the mechanization seemed more fruitful. The inherent advantage of this keyword is that all prerequisites

are clearly stated, so when proving lemmas using this keyword, there is no need to consider cases where those prerequisites do not hold. To prove properties of a function, all cases of parameters need to be considered (unless specified in the lemma), even if the function is only ever used in contexts where certain properties hold.

Figure 3.3.: Formalization of Shuffling and the Shuffled Language

```
inductive shuffled ::"('information, 'peer) action word ⇒ ('information, 'peer)
action word ⇒ bool" where

  refl: "shuffled w w" |

  swap: "⟦ is_output a; is_input b ; w = (xs @ a # b # ys) ⟧
        ⟹ shuffled w (xs @ b # a # ys)" |

  trans: "⟦ shuffled w w'; shuffled w' w'' ⟧ ⟹ shuffled w w''"

abbreviation valid_input_shuffle ::
  "('information, 'peer) action word ⇒ ('information, 'peer) action word ⇒ bool"
(infixl "⧢?" 60) where
  "w' ⧢? w ≡ shuffled w w'"

definition all_shuffles :: "('information, 'peer) action word ⇒ ('information, 'peer)
action word set" where
  "all_shuffles w = {w'. shuffled w w'}"

definition shuffled_lang :: "('information, 'peer) action  language ⇒ ('information,
'peer) action language" where
  "shuffled_lang L = (⋃w∈L. all_shuffles w)"
```

In Figure 3.3, we see how we can use an *inductive* to decompose the construction of the shuffled language into different rules or cases. Intuitively, a shuffle w' can be created from some w in three ways using Definition 3.2.1: either $w = w'$ (reflexive case), w was shuffled exactly once to receive w', or w' was gathered through an arbitrary number of shuffles. We can make these implicit cases explicit in the *inductive*. For example, the *trans* rule, which specifies that the shuffle-relation is transitive, requires some w to shuffle into w' and also w' to shuffle into w'' to conclude the transitive step that w can shuffle into w''. The other rules *refl* and *swap* follow the same concept and formalize the reflexive and single swap properties, respectively. The *abbreviation* below

introduces a shorthand to use the above relation with, using a symbol similar to $\sqcup\!\!\sqcup_?$ as in Definition 3.2.1. The two instances of *definition* below use list comprehension syntax to collect all shuffles of some word w, and all shuffles of all words in a language L, respectively. Having established these formalizations in Isabelle, we can then begin to prove lemmas about the shuffle relation.

Example 3.3.2 (Proving a Small Lemma in Isabelle)**.** Figure 3.4 shows the declaration and proof of a lemma that shows that we can prepend some a to a shuffle, without destroying the shuffle. Formally: for two words w and w' where $w' \sqcup\!\!\sqcup_? w$, we show that $(a \cdot w') \sqcup\!\!\sqcup_? (a \cdot w)$. In Isabelle, we state this lemma using the *lemma* keyword, followed by a name (or none) and then the assumptions (with *assumes*) and statements (using *shows*). Naming lemmas and theorems is optional, but it generally makes further usage and referencing significantly easier.

The *using assms* means that the assumptions are to be applied in the proof below; without this, Isabelle does not automatically consider them in the proof context. It can be helpful to provide certain definitions or lemmas directly with *using*, but it can also overcomplicate the proof in other cases. In general, the assumptions (*assms*) that were declared with *assumes* can and should often be added with *using*.

Figure 3.4.: A Prepend-Lemma for the Shuffle-Relation

```
lemma shuffled_prepend_inductive:
  assumes "shuffled w w''"
  shows "shuffled (a # w) (a # w')"using assms
proof (induct)
  case (refl w)
  then show ?case using shuffled.refl by auto
next
  case (swap a b w xs ys)
  then show ?case by (metis (no_types, lifting) Cons_eq_appendI shuffled.simps)
next
  case (trans w w' w'')
  then show ?case using shuffled.trans by auto
qed
```

We then do an induction over the definition of the inductive *shuffled* from Figure 3.3, by invoking *proof (induct)*. In cases where it is not obvious to Isabelle what the induction is over, parameters and rules need to be specified together with the *(induct)*. Since the shuffle relation was formalized as an *inductive*, we can use this property in proofs and the rules of the *inductive* form the cases of the induction proof. This can

be observed, since the cases contain the rule names (refl, swap and trans), as well as the respective parameters of these rules in the brackets after the *case* keyword. The reflexive and transitive cases are both solved by only using the respective *inductive* rule and applying *auto*. The proof method *auto* automatically tries to simplify the subgoals of the statement at hand, which here suffices to complete the proof. Depending on the proof goal, using *auto* can also make the proof more complicated and result in more subgoals than necessary [NK23]. It can be noted that for many proofs, *auto* often finds a good simplification. The *swap* case in Figure 3.4 is the most complicated. Here we applied *sledgehammer*, which then found the solution of applying the ATP *metis* with a list append lemma and some simplification rules (*shuffled.simps*) of the *shuffled* formalization. How *sledgehammer* works in more detail, how *metis* functions and related topics can be referenced in e.g. [Bla+25]. With this, all three cases are proven, and the keyword *qed* concludes our first showcased Isabelle proof. [NPW25; NK23; Bla+25; Pau94]

Apart from the induction we showcased in Example 3.3.2, most other common proof methods are also available in Isabelle, or can be specified by the user themselves. A majority of the proofs rely on some form of induction (e.g. with *induct*), but proof by contradiction (*rule ccontr*) or letting Isabelle automatically detect what proof method is "best" (with *auto*) are some of the other methods available.

We have introduced the core features of Isabelle that we will need throughout this work. In the following chapter, we will first provide handwritten proofs and intuition, and then supplement those with excerpts of our Isabelle formalization [Ket25] as we see fit. Where not otherwise specified, the formalization excerpts are our work; in specified cases, we also showcase some of the original framework that was provided to us by the authors of [DLP24].

The reader may have noticed that we have not yet specified the main theorem of our source paper [DLP24, Theorem 4.5] anywhere, which we want to mechanize. The reason is that we improved the original version throughout our work in multiple iterations, which will be detailed in the coming chapter. This means we will present the original formalization(s), followed by our comments and corrections or further specifications where needed.

4. Inconsistencies, Errors and Corrections

This work aims to mechanize the main theorem [DLP24, Theorem 4.5] concerning synchronizability in mailbox systems with tree topologies in the proof assistant Isabelle, to ensure their correctness; this includes the lemmas and framework needed to formalize the theorem, as well. This chapter will detail the process of creating and iteratively improving our formalizations, as well as formally introduce the main theorem.

Because of our mechanization process, we were able to discover some edge cases that were missed in the original paper. Throughout our work, there were multiple moments where proving certain proof steps or lemmas was not possible with Isabelle. This can occur for a multitude of reasons, some include that smaller lemmas are still needed but have not yet been proven, some edge case(s) still remain(s), some definition does not function as expected, or when the to-be-proven step is not actually correct [NK23]. Because this project involves many of our own definitions and the project itself has a large scope, Isabelle was not able to find counterexamples automatically in most instances[1]. This prompted a manual search of counterexamples and other formalization errors, which ended up yielding multiple results across different revisions of the main theorem.

The counterexamples presented in the following sections consist of small but representative instances, found after much deliberation. The initial counterexamples were larger in some cases. Thus, the progression from one revision to the next might seem less intuitive here than it was in reality. Still, for brevity, we have chosen to showcase smaller instances that function analogously to their initial drafts, respectively.

Before detailing the verification process, a short foreword about our code and Isabelle experience will follow. The first part of this work was gaining proficiency in Isabelle [Pau94], for which we fully utilized all available learning materials, namely [Wen25; NPW25; NK23; Nip23; Bla+25; BP10]. The mechanization consists mostly of our own formalizations; the rest is supplemented by the ones we were initially provided by the authors of [DLP24]. In addition to these, we used the "Main"[2] library of Isabelle, which provides standard lemmas such as those for lists, etc., as well as the "Sublist" library [Nip+], which provides definitions and lemmas related to prefixes.

[1] For easy contradictions, e.g., a lemma that assumes $x = 1$ and shows $x = 0$, Isabelle's automatic checks succeed, it will warn that this lemma is wrong and (if it finds one) provide a counterexample.

[2] `https://isabelle.in.tum.de/library/HOL/HOL/Main.html`

Our full Isabelle formalization is publicly accessible on GitHub [Ket25], located within *Formalization_Isabelle_Extension*. As referenced in [DLP24, Discussion], the authors of the source paper have started formalizing the paper in Isabelle. We were provided this code as a starting point, which can be found in our repository in *Formalization_Isabelle*, as well throughout our final formalization. This initial version contained many rudimentary definitions (for formal languages, communicating automata, etc.), but was still extremely limited concerning the formalizations needed for proving the main theorem. We will use this original mechanization as a reference point for our contribution. When comparing the formalizations' lengths by exporting them as separate PDFs, the initial formalization is around 26 pages [3] whereas the final version is 114 pages long.[4] For the conversion, we used Isabelle's functionality to export the code as LaTeX and then compiled the PDF, which contains exclusively the code, majorly sans comments[5] [NPW25; Wen25; Pau94].

4.1. Formalization of Sends and Receptions

We will begin detailing our process by clarifying two definitions. As alluded to in the previous chapter, we wish to specify the sets of $\mathbb{P}^p_{send}$ and resp. $\mathbb{P}^p_{rec}$ a bit further than in the original paper [DLP24]. The introduction of these two sets is slightly ambiguous, as one can understand them to either be defined on the network topology or on each peer automaton A_p locally. In the latter case, we cannot always determine the root directly, as explained in Example 4.1.1 and elaborated more below. While this has no impact on the overall integrity of the theorem, we choose here to take the full network information into account, to allow for direct root identification and, consequently, clearer reasoning. Thus, this work will denote $\mathbb{P}^p_{send}$ and $\mathbb{P}^p_{rec}$ as gathered from the topology. Also, we will denote with r the unique root of the network, which is precisely the peer $r \in \mathbb{P}$ with $\mathbb{P}^r_{send} = \{\}$, i.e. r receives from no other peer, and no other peer sends to r.

Example 4.1.1 (Local and Global Topology Inferences)**.** In Figure 4.1, only A_q provides information on the messages being sent and received in this network (since both A_r and A_p only produce ε). Thus, locally A_r and A_p only know that they send and receive no messages, respectively. In a tree, only the root receives from no other peer, so both A_r and A_p could be considered the root from just their local information alone.

[3] Some of the original code from [DLP24] (at most half a page) is not included in this PDF, as it was not compiling, in which case Isabelle's export functionality does not work.

[4] These PDFs can also be found in the repository, in *Original.pdf* and *Synchronizability.pdf*, respectively, in the folder *Latex_Exports*.

[5] Some comments are included in the PDF, but only a handful, because the export seemingly includes inline comments, whereas it disregards multiline ones.

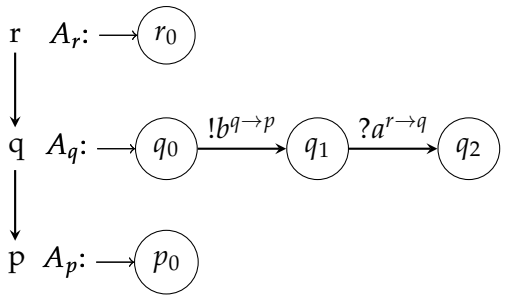


Figure 4.1.: In this network, r can be determined as root globally, but not directly from the local information alone, since both A_r and A_p contain no transitions and hence locally: $\mathbb{P}^r_{send} = \mathbb{P}^r_{rec} = \{\} = \mathbb{P}^p_{send} = \mathbb{P}^p_{rec}$.

With the addition of the transitions in A_q, however, we can determine $\mathbb{P}^q_{send} = \{r\}$ and $\mathbb{P}^q_{rec} = \{p\}$, and with this, we can then label r as the root. This highlights that at some point, we can receive the same knowledge about the network locally or from the topology, but the latter is more straightforward. Put differently, locally, $\mathbb{P}^r_{send} = \{\}$ does not directly confirm that r is the root (as is the case in Figure 4.1), whereas it does when the full topology is taken into account.

In this example, we have only considered the transitions of each automaton. When looking at the message set, which is the same for each automaton in the network due to Definition 3.1.1, the topology can be deduced immediately. The message set essentially delivers global information about the network, within each automaton. In other words, depending on what local information is considered (transitions or message set), the root can be uniquely identified.

To underline the difference between determining the root from topology or from local transitions only, we provide Figure 4.2. Here, we determine the root using the topology by first ensuring that the topology is in fact a *tree_topology* and then checking that there are no ingoing edges to p in the topology graph $\mathcal{G}$ of our network. This suffices to identify the unique root in any network when given the topology. If only the local knowledge of each respective peer is taken into account, we must perform two separate checks. For p, we need to ensure that it does not perform any receptions by examining $\mathcal{P}_?(p)$ and also whether there is any other peer q that sends any message to p, i.e., where $p \in \mathcal{P}_!(q)$.

The $\mathcal{P}_?(p)$ (respectively $\mathcal{P}_!(p)$) in Isabelle are similar to $\mathbb{P}^p_{send}$ ($\mathbb{P}^p_{rec}$) in this work, but are defined in an inverse manner. The former is the set of peers that p receives from, and the latter is the set of peers to which it sends; both sets are gathered from the transitions in A_p. The matching Isabelle formalizations for these sets can be found in Figures 5.4 and 5.5 in Chapter 5. They rely directly on the formalizations of communicating

automata and their networks, which we will detail then. For this section, it suffices to know the notions of $\mathbb{P}^p_{send}$ and $\mathbb{P}^p_{rec}$, and the context we give for the two figures here.

Whether we use the local transitions of each automaton to determine the root, or the entire topology, we can determine the root uniquely in both cases. In fact, we have shown in Isabelle that our formalizations *is_root_from_topology* and *is_root_from_local* are equivalent, as is expected.

Figure 4.2.: Formalization of the Root

```
abbreviation is_root_from_topology :: "'peer ⇒ bool" where
  "is_root_from_topology p ≡ (tree_topology ∧ 𝒢⟨→p⟩ = {})"

abbreviation is_root_from_local :: "'peer ⇒ bool"  where
  "is_root_from_local p ≡ tree_topology ∧ 𝒫?(p) = {} ∧ (∀ q. p ∉ 𝒫!(q))"

abbreviation is_root :: "'peer ⇒ bool"  where
  "is_root p ≡ is_root_from_local p ∨ is_root_from_topology p"
```

The formalization of a node in the tree follows the same pattern as can be seen in Figure 4.3. The difference to the root is that here, we check that there is exactly one edge towards p in the topology graph, which delivers us its unique parent q. In the local contexts, we search for a transition with a message that either p receives from q in A_p, or q sends to p in A_q. Since we are analyzing asynchronous systems, one may occur without the other, as illustrated in Figure 4.1.

Figure 4.3.: Formalization of a Node

```
abbreviation is_node_from_topology :: "'peer ⇒ bool" where
  "is_node_from_topology p ≡ (tree_topology ∧ (∃ q. 𝒢⟨→p⟩ = {q}))"

abbreviation is_node_from_local :: "'peer ⇒ bool"  where
  "is_node_from_local p ≡ tree_topology ∧ (∃ q. 𝒫?(p) = {q} ∨ p ∈ 𝒫!(q))"

abbreviation is_node :: "'peer ⇒ bool"  where
  "is_node p ≡ is_node_from_topology p ∨ is_node_from_local p"
```

We have shown how the transitions of only one peer's automaton do not suffice in all cases to determine whether it is the root of a given tree. In the global context of this work, this does not make a difference, as will become obvious later. However, in the intermediate steps, it does indeed have an impact. In the beginning, we determined the

root by simply checking that p receives no message from any other peer, by enforcing $\mathcal{P}_?(p) = \{\}$. When we wanted to infer that, then also no q sends any message to p, i.e. $p \notin \mathcal{P}_!(q)$ for all q in the network, Isabelle would not allow it. This prompted us to reexamine our formalization, create the network in Figure 4.1 as well as Example 4.1.1, and clarify the definitions.

4.2. Revision of the Influenced Language

Thus far, we have only clarified existing definitions without altering them. In contrast, this section details an edge case with significant implications for the main theorem and the formalization of the source paper [DLP24] in general. Following this finding, we altered one of the proposed definitions. To elaborate on this, we first need to introduce the notion of the *influenced language*, which is at the root of the issue.

In [DLP24, Definition 4.2], the authors introduce the definition of the *influenced language* to "capture the influence the automata have on the language of each other". The idea is to remove all words from the language of a peer p, which it cannot reasonably produce given the context of its network. If a peer p receives some m, then it must have been sent first by its parent q, and q may in turn have to receive something from its parent (p's grandparent), and so on, until the root r is reached. We denote as ancestors of p any peers along the path from p to the root that are not p itself. Since we are only considering trees, the path from any peer to the root is unique, and so are the nodes visited along this path. We write (direct) child for p and (direct) parent for q, if they satisfy $\mathbb{P}^p_{send} = \{q\}$ (i.e., q sends to p) or $p \in \mathbb{P}^q_{rec}$ (i.e., p receives from q).

If somewhere along such a unique path from the root to peer p, some ancestor of p cannot provide a necessary send for this execution to its direct child, then p cannot produce its desired word in the end, either. Such words, for which there is no valid execution, are not present in the influenced language. In the original paper, this was formalized into [DLP24, Definition 4.2], which we restate below and apply to a small scale in Example 4.2.1.

Definition 4.2.1 (Original Influenced Language)**.** Let $p \in \mathbb{P}$. The influenced language and its projections to sends/receptions are defined as follows:

$$\mathcal{L}^{\Diamond}(A_p) := \begin{cases} \mathcal{L}(A_r) & \text{if } p = r \\ \left\{ w \mid w \in \mathcal{L}(A_p) \wedge (w{\downarrow_?}) \not\downarrow_{!?} \in \left(\mathcal{L}^{\Diamond}_!(A_q)\right) \not\downarrow_{!?} \wedge \mathbb{P}^p_{send} = \{q\} \right\} & \text{otherwise} \end{cases}$$

$$\mathcal{L}^{\Diamond}_?(A_p) := \left(\mathcal{L}^{\Diamond}(A_p)\right){\downarrow_?}$$

$$\mathcal{L}^{\Diamond}_!(A_p) := \left(\mathcal{L}^{\Diamond}(A_p)\right){\downarrow_!}$$

Example 4.2.1 (Original Influenced Language Demonstration)**.** Consider Figure 4.4, which is the network from Figure 4.1 with the transitions $?a$ and $!b$ swapped. Here, the root r sends nothing, but q needs to perform $?a^{r\to q}$ before being able to send $!b^{q\to p}$. We can infer $\mathcal{L}(A_r) = \{\varepsilon\} = \mathcal{L}^{\emptyset}(r) = \mathcal{L}^{\emptyset}_{!}(r)$ since r is the root and because $\{\varepsilon\} = \{\varepsilon\}\downarrow_{!}$. All states in this network are considered final, so r can only perform ε and ε cannot be erased by applying any projections. Since the influenced language of r contains no proper sends, there can be no $w \in \mathcal{L}(A_q)$ s.t. $(w\downarrow_{?})\downarrow_{!?} \in (\mathcal{L}^{\emptyset}_{!}(A_r))$, other than ε. Thus, $\mathcal{L}^{\emptyset}(q) = \{\varepsilon\}$ and we can gather that q can never send or receive any message other than ε.

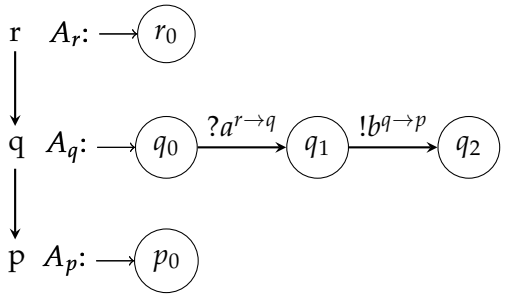


Figure 4.4.: A network where the influenced language of q is empty, i.e. $\mathcal{L}^{\emptyset}(A_q) = \{\}$.

However, as it is written down, the formalization in Definition 4.2.1 is too strong: it removes words for which there are valid executions, which is undesired. For trees where each parent sends to at most one child, respectively, this definition works as expected. However, if any parent sends a message to two children within one word, the influenced language removes this word, even if there is a valid execution for it. We want to illustrate this issue in Example 4.2.2, using the network in Figure 4.5.

Example 4.2.2 (Counterexample 0)**.** Consider the network in Figure 4.5. The only peer that can perform send-actions is the root r. Thus, we can infer the traces from only $\mathcal{L}(A_r) = \{\varepsilon, !a^{r\to q}, !a^{r\to q}!b^{r\to p}\}$. Both q and p are ready to perform the matching receptions ($?a$ and $?b$, respectively) at any point from their initial state. We observe that each send of r can be immediately received. Thus, the network is synchronizable, as all its traces can be performed both synchronously and asynchronously.

We now begin considering the influenced language. There is an execution where p can perform $?b^{r\to p}$, since r can send the matching message $!b$ as its second action. We can observe that r is p's only ancestor, and as it is the root, r requires no sends from any peer. There is also no other peer on the path from r to p. To summarize, r can send

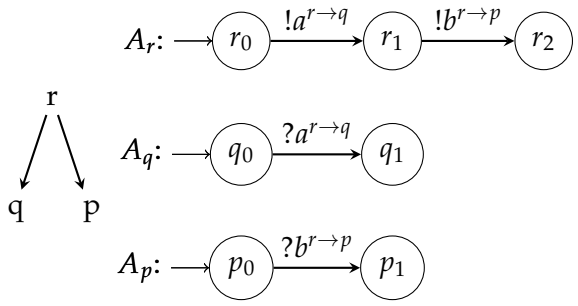


Figure 4.5.: Counterexample 0: A network where the original definition of the influenced language (as in Definition 4.2.1) of p contains only ε.

$!b$ (this action is not blocked by a reception), p can receive $?b$ from its initial state. Since all conditions for the influenced language intuition hold for p in this network, one would expect $?b^{r\to p} \in \mathcal{L}^{\emptyset}(A_p)$. To show the contrary, we first consider the influenced language $\mathcal{L}^{\emptyset}(A_p)$ of p's only ancestor r, the root:

$$\mathcal{L}^{\emptyset}(A_r) = \mathcal{L}(A_r) = \{\varepsilon, !a^{r\to q}, !a^{r\to q}!b^{r\to p}\} = \mathcal{L}^{\emptyset}_{!}(A_r)$$

We consider all states accepting, thus $\mathcal{L}(A_p) = \{\varepsilon, ?b^{r\to p}\}$ can be inferred from the respective automaton in Figure 4.5. With this, and knowing that $(?b^{r\to p})\downarrow_{!?} = b^{r\to p}$, we can perform the check in the second part of the conjunction of the influenced language definition. Using the facts gathered thus far, we check whether $((?b^{r\to p})\downarrow_{?})\downarrow_{!?} \in (\mathcal{L}^{\emptyset}_{!}(A_r))\downarrow_{!?}$ holds, and can infer the following:

$$((?b^{r\to p})\downarrow_{?})\downarrow_{!?} = b^{r\to p} \notin \{\varepsilon,\ a^{r\to q},\ a^{r\to q}b^{r\to p}\} = \{\varepsilon,\ !a^{r\to q},\ !a^{r\to q}!b^{r\to p}\}\downarrow_{!?} = (\mathcal{L}^{\emptyset}_{!}(A_r))\downarrow_{!?}$$

Since this check fails, $?b^{r\to p} \notin \mathcal{L}^{\emptyset}(A_p) = \{\varepsilon\}$, i.e. the definition is not working as intended. It does not work, since there are multiple asynchronous executions where $?b$ can be performed, such as $!a^{r\to q}\cdot!b^{r\to p}\cdot?b^{r\to p}$. This execution is valid, since p's buffer only contains $b^{r\to p}$ before p receives it, as the message $a^{r\to q}$ gets sent to q's own buffer. This means p can in fact perform this action $?b$, which is not reflected by Definition 4.2.1.

To summarize, the influenced language is meant to remove words from peers for which there are no valid executions, i.e., to remove uninteresting cases for the synchronizability decision procedure. A trace requires at least one valid execution, i.e., one that is possible for the network in terms of reachable configurations for all its peers. If a trace is not possible (for both synchronous and asynchronous communication), then it can also not hinder the synchronizability in any way.

The illustrated network from Figure 4.5 is synchronizable as explained in Example 4.2.2, since the only traces possible are exactly the words in $\mathcal{L}^{\emptyset}(A_r)$, and both children are ready to receive without any blocking sends. We satisfy the left side of the *if and only if* of the original main theorem [DLP24, Theorem 4.5] below:

Theorem 4.2.2 (Original Mailbox Synchronizability Theorem)**.** *Let N be a network such that $\mathbb{C}_F = \mathbb{C}$ and $G(N)$ is a tree. Then $\mathsf{T}(N_{\texttt{mbox}}) = \mathsf{T}(N_{\texttt{sync}})$ iff, for all $p, q \in \mathbb{P}$ with $\mathbb{P}^p_{send} = \{q\}$, we have $\left(\left(\mathcal{L}^{\emptyset}_{!}(A_q)\right)\downarrow_{\{p,q\}}\right)\downarrow_{\not{!}\not{?}} \subseteq \left(\mathcal{L}^{\emptyset}(A_p)\right)\downarrow_{\not{!}\not{?}}$ and $\mathcal{L}^{\emptyset}(A_p) = \mathcal{L}^{\emptyset}_{\sqcup\!\sqcup}(p)$.*

We will refer to the rightmost part of the theorem ($\mathcal{L}^{\emptyset}(A_p) = \mathcal{L}^{\emptyset}_{\sqcup\!\sqcup}(p)$) as the *shuffled language condition*, and to the first condition on the right side as the *subset condition*[6].

Since the network in Figure 4.5 satisfies the left side of the theorem, it should consequently satisfy the right side as well. We will show the contrary in the following example:

Example 4.2.3 (Continuation of Counterexample 0)**.** Consider again the network in Figure 4.5. We gathered in Example 4.2.2, that $(\mathcal{L}^{\emptyset}_{!}(A_r))\downarrow_{\not{!}\not{?}} = \{\varepsilon,\ a^{r\to q},\ a^{r\to q}b^{r\to p}\}$. Projecting $(\mathcal{L}^{\emptyset}_{!}(A_r))\downarrow_{\not{!}\not{?}}$ to the set $\{p, r\}$ removes both occurrences of $a^{r\to q}$, since $q \notin \{p, r\}$, but q is the receiver in this message. It follows that $(\mathcal{L}^{\emptyset}_{!}(A_r)\downarrow_{\{p,r\}})\downarrow_{\not{!}\not{?}} = \{\varepsilon,\ b^{r\to p}\}$. We also observe directly from the network that $\mathbb{P}^p_{send} = \{r\}$. We can finally infer

$$(\mathcal{L}^{\emptyset}_{!}(A_r)\downarrow_{\{p,r\}})\downarrow_{\not{!}\not{?}} = \{\varepsilon,\ b^{r\to p}\} \not\subseteq \{\varepsilon\} = \mathcal{L}^{\emptyset}(A_p)\downarrow_{\not{!}\not{?}}.$$

Thus, the network in Figure 4.5 satisfies only the left side of the *if and only if* in Theorem 4.2.2, but not the right side, a contradiction.

To conclude, the definition [DLP24, Definition 4.2] in the source paper not only does not fully work as intended (see Example 4.2.2), but it in turn renders the entire theorem[7] to be false (as seen Example 4.2.3). We can remedy this oversight by projecting $\mathcal{L}^{\emptyset}_{!}(A_q)$ in Definition 4.2.1 to the two peers p and q. Since $\mathcal{L}^{\emptyset}_{!}(A_q)$ is the language (projected to only sends) of A_q, it only contains send-actions of q. We need to ensure that only sends from q to p are considered, thus a projection on *actions where p is the second participant*, i.e., here the recipient, is necessary. Intuitively, the original definition did not work in most instances where some peer sends to more than one child, so the solution is to only consider one child with the parent at a time. By using such a pair-projection, we may discard any actions that do not concern the current child for which we are computing the influenced language.

[6] Currently, the subset condition is $((\mathcal{L}^{\emptyset}_{!}(A_q))\downarrow_{\{p,q\}})\downarrow_{\not{!}\not{?}} \subseteq (\mathcal{L}^{\emptyset}(A_p))\downarrow_{\not{!}\not{?}}$.

[7] Both Lemma 4.4 and Theorem 4.5 in [DLP24] are affected by this, but the fix for both is to add the projection to the set $\{p, q\}$, so Lemma 4.4 is not explicitly stated for brevity.

In the remainder of this work, whenever we refer to the *influenced language* of some peer and its projections (as defined in the original definition), we will refer instead to this revised version[8]:

Definition 4.2.3 (Revised Influenced Language)**.** Let $p \in \mathbb{P}$. We define the influenced language as follows:

$$\mathcal{L}^{\lozenge}(A_p) := \begin{cases} \mathcal{L}(A_r) & \text{if } p = r \\ \left\{ w \mid w \in \mathcal{L}(A_p) \wedge (w{\downarrow_?}){\downarrow_{\not{!}\not{?}}} \in \left((\mathcal{L}^{\lozenge}_{!}(A_q)){\downarrow_{\{p,q\}}} \right){\downarrow_{\not{!}\not{?}}} \wedge \mathbb{P}^{p}_{send} = \{q\} \right\} & \text{otherwise} \end{cases}$$

The projections of the influenced language to sends and receptions, respectively, remain the same, thus they are not repeated here. Analogously, we will use this revised version when applying the shuffled language as introduced in Definition 3.2.1. The only difference from the original definition of the influenced language is the added pair projection in the second case of the conjunction. Lastly, we can also apply the definition in the following manner: given a $w \in \mathcal{L}^{\lozenge}(A_p)$ we can infer that there exists some $w' \in \mathcal{L}^{\lozenge}(A_q)$ such that $((w'{\downarrow_!}){\downarrow_{\{p,q\}}}){\downarrow_{\not{!}\not{?}}} = (w{\downarrow_?}){\downarrow_{\not{!}\not{?}}}$.

Figure 4.6.: Formalization of the Revised Influenced Language

```
inductive is_in_infl_lang :: "'peer ⇒ ('information, 'peer) action word ⇒ bool"
where
  IL_root: "⟦is_root r; w ∈ ℒ(r)⟧ ⟹ is_in_infl_lang r w" |
  IL_node: "⟦tree_topology; is_parent_of p q; w ∈ ℒ(p); is_in_infl_lang q w';
((w↓?)↓!?) = (((w'↓{p,q})↓!)↓!?)⟧ ⟹ is_in_infl_lang p w"

abbreviation InfluencedLanguage :: "'peer ⇒ ('information, 'peer) action lan-
guage"  ("ℒ* _" [90] 100) where
  "ℒ* p ≡ {w. is_in_infl_lang p w}"
```

Figure 4.6 shows our Isabelle formalization of this revised definition. We chose an *inductive* here, since there are many preconditions that need to be satisfied to build the set of the influenced language of some peer. There are two rules in *is_in_infl_lang*, one for the root and one for the node case, which match the case distinction in Definition 4.2.3. In *IL_root*, if r is the root and $w \in \mathcal{L}(A_r)$, then w is also in the influenced language. The node case *IL_node* assumes we have a node p, its direct parent q, and a

[8]We use the same notation as for the original version, which may not be ideal for formal correctness, but we will see this as "overriding" the original definition, which did in the Isabelle code as well.

word $w \in \mathcal{L}(A_r)$. The rule also enforces that there is some word $w' \in \mathcal{L}^{\emptyset}(A_q)$, s.t. the sent messages to p in w' match the received ones in w exactly. Intuitively, we recreate the handwritten definition in two separate steps. First, we implement *is_in_infl_lang*, which is *True* for a given peer p and word w, if $w \in \mathcal{L}(A_p)$. In the second step, we introduce the *abbreviation InfluencedLanguage*, which performs a set comprehension given some peer p and collects all the w in its influenced language, i.e. for which our *inductive* holds.

Building lists in two steps is not necessary in Isabelle; one can, in many cases, use the *inductive_set* keyword instead. The syntax is similar to the more general *inductive*, but a bit more restrictive. In fact, all parameters that occur in the calls need to be specified with names and types, using the keyword *for* before the *where*. Only the introduced variables may be assumed to be in the set, or added to it. Here, this meant we couldn't enforce that q has some word w' in its influenced language, since then we would need the sets for p and q, which *inductive_set* does not easily allow. There are ways to circumvent this issue, but the simplest solution was to use the less constraining *inductive*. [Wen25]

4.3. Revisions of the Main Theorem

Within this and the following sections, we will describe the iterative process of contradicting and then revising [DLP24, Theorem 4.5]. The original theorem can be referenced in Theorem 4.2.2, and we will from here on use this theorem with our revised influenced language from Definition 4.2.3. The faulty original influenced language also contradicts the main theorem, but in the previous section we only adjusted the definition. Commencing with this section, we will begin altering the main theorem directly, where necessary.

As was explained right below Theorem 4.2.2, we refer to the two conditions on the right side as the subset condition and the shuffled language condition, respectively. We will exclusively revisit the subset condition in the following sections, and restate each resulting variation of the theorem as a conjecture until we prove our final version in the end. The general construction of the theorem will remain the following, where *the subset condition* is a placeholder for the original one, or any of our revisions:

> Let N be a network such that $\mathbb{C}_F = \mathbb{C}$ and $G(N)$ is a tree. Then $\mathsf{T}(N_{\texttt{mbox}}) = \mathsf{T}(N_{\texttt{sync}})$ iff, for all $p, q \in \mathbb{P}$ with $\mathbb{P}^p_{send} = \{q\}$, we have *the subset condition* and $\mathcal{L}^{\emptyset}(A_p) = \mathcal{L}^{\emptyset}_{\sqcup}(p)$ (*the shuffled language condition*).

As a reminder, our current subset condition is the original one proposed in [DLP24, Theorem 4.5]: $((\mathcal{L}^{\emptyset}_{!}(A_q))\downarrow_{\{p,q\}})\downarrow_{\not!?} \subseteq (\mathcal{L}^{\emptyset}(A_p))\downarrow_{\not!?}$ for all $p, q \in \mathbb{P}$ with $\mathbb{P}^p_{send} = \{q\}$.

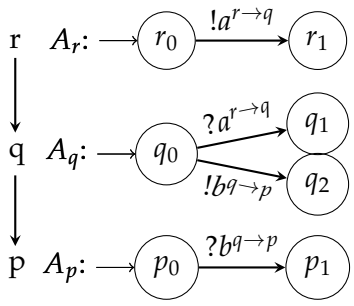


Figure 4.7.: Counterexample 1: A network that is not synchronizable, but satisfies the right side of the original theorem, as restated in our Theorem 4.2.2.

4.3.1. First Counterexample

For this first theorem-altering counterexample, we are applying the original Theorem 4.2.2 as introduced in [DLP24, Theorem 4.5]. Since the influenced language occurs on the right side of the original theorem, and the influenced language has already posed a problem in our initial counterexample, we explored the possibility of there being more issues related to its application.

The original subset condition only requires the sends of a parent to be receivable by the respective children. It does not restrict other branches, however, where a child may not be able to receive all such sends. We will use the notion of branches of an automaton for this counterexample, as well as the remaining ones. To clarify it, we refer to Example 4.3.1.

Example 4.3.1 (Branches)**.** By *branches*, we denote different paths that can be taken in an automaton. For example, there are two branches from the initial state in A_q in Figure 4.7. Here, q can either perform $?a^{r\to q}$ and move to state q_1, send $!b^{q\to p}$ and move to q_2, or do nothing and stay in q_0. The other two peers only have one branch, respectively.

Branches may also be longer than one action. We differentiate between branches by the respective words they produce. As each state is final, any path corresponds to an accepting run of the respective automaton. In the networks we showcase, each branch produces distinct words, i.e. they are deterministic. One recurring feature of our counterexamples is that we exploit differing branches.

Restating our hypothesis, the original subset condition only ensures that at least one branch of each node's automaton in the network can receive all of its direct parents' sends. Intuitively, having one branch in the child-automaton that receives exactly the sends to it from the parent, and another branch that performs a send itself, but receives none of the parent's, may pose a problem. We apply this idea in Example 4.3.2 using

the original theorem with the subset condition: $((\mathcal{L}_!^{\emptyset}(A_q))\downarrow_{\{p,q\}})\downarrow_{\not!?} \subseteq (\mathcal{L}^{\emptyset}(A_p))\downarrow_{\not!?}$ for all $p, q \in \mathbb{P}$ with $\mathbb{P}^p_{send} = \{q\}$.

Example 4.3.2 (Counterexample 1)**.** With a network as in Figure 4.7, there is no synchronous execution of the trace $!a^{r\to q}.!b^{q\to p} \in \mathsf{T}(N_{\texttt{mbox}})$: q cannot receive $a^{r\to q}$ before, or after sending $b^{q\to p}$, as the two actions are on different branches of A_q and thus mutually exclusive. In the mailbox system, however, the message can be sent, since messages do not necessarily need to be received in an asynchronous execution. It follows, we have found a trace that is in $\mathsf{T}(N_{\texttt{mbox}})$ but not in $\mathsf{T}(N_{\texttt{sync}})$, which contradicts the left side of the *if and only if* in the original Theorem 4.2.2. Put differently, the network is not synchronizable due to the differing trace sets.

We will now show that despite this, the right side of the *if and only if* is satisfied. The shuffled language condition is trivially satisfied, since each possible word in any peer's language consists of exactly one action only. To perform an actual shuffle, at least two actions are needed. Each word is a shuffle of itself, and thus is also in the shuffled language, but no additional words can be created through shuffling. This means the shuffled language contains exactly the reflexive shuffle of each word in the peer's language, which is the peer's language again.

The subset condition is also satisfied, as is inferred in the following steps. We first consider the (influenced) languages of the three peers in the network:

$$\begin{aligned}
\mathcal{L}(A_r) &= \{\varepsilon, !a^{r\to q}\} = \mathcal{L}^{\emptyset}(A_r) = (\mathcal{L}_!^{\emptyset}(A_r))\downarrow_{\{p,q\}} \\
\mathcal{L}(A_q) &= \{\varepsilon, ?a^{r\to q}, !b^{q\to p}\} = \mathcal{L}^{\emptyset}(A_q) = \mathcal{L}^{\emptyset}_{\sqcup\!\sqcup}(q) \\
\mathcal{L}(A_p) &= \{\varepsilon, ?b^{q\to p}\} = \mathcal{L}^{\emptyset}(A_p) = \mathcal{L}^{\emptyset}_{\sqcup\!\sqcup}(p).
\end{aligned}$$

Since $\{\varepsilon, !a^{r\to q}\}\downarrow_{\not!?} \subseteq \{\varepsilon, ?a^{r\to q}, !b^{q\to p}\}\downarrow_{\not!?}$, because both sets include the messages ε and $a^{r\to q}$, we then receive

$$((\mathcal{L}_!^{\emptyset}(A_r))\downarrow_{\{r,q\}})\downarrow_{\not!?} \subseteq (\mathcal{L}^{\emptyset}(A_q))\downarrow_{\not!?}.$$

The subset condition as stated in Theorem 4.2.2 is satisfied for the peer pair r and q. The only other parent-child pair to analyze in this network is q and p and here we can infer

$$((\mathcal{L}^{\emptyset}(A_q))\downarrow_{\{q,p\}})\downarrow_{\not!?} = (\{\varepsilon, ?a^{r\to q}, !b^{q\to p}\}\downarrow_{\{q,p\}})\downarrow_{\not!?} = \{\varepsilon, b^{q\to p}\} = (\mathcal{L}^{\emptyset}(A_p))\downarrow_{\not!?}.$$

There is no other pair we can consider for the subset condition, as the root has no parent and the two other peers have one unique parent each, as the network is a tree. To summarize, the right side of the *if and only if* in Theorem 4.2.2 is satisfied, as both conditions hold for the network in Figure 4.7, but the left side does not hold; a contradiction is found.

We can observe, the original subset condition is not strong enough. In particular, the problem here is that by sending a message, the child enters a branch where it can no longer receive all potential sends of its parent. Put differently, prefixes consisting exclusively of send-actions in the words of some peer's language might pose problems concerning the synchronizability of a network. We write $outputprefixes$ for all prefixes $w \in \mathcal{L}^{\emptyset}(A_p)$ of some peer p, s.t. $w{\downarrow_!} = w$, i.e. $outputprefixes(\mathcal{L}^{\emptyset}(A_p))$ is the set of words in $\mathcal{L}^{\emptyset}(A_p)$ consisting only of outputs. All prefixes are also words in the language, since all states are final. We revise the original theorem to the following, and continue on with it as our working hypothesis:

Conjecture 4.3.1 (First Revision of the Mailbox Synchronizability Theorem)**.** Let N be a network such that $\mathbb{C}_F = \mathbb{C}$ and $G(N)$ is a tree. Then $\mathsf{T}(N_{\texttt{mbox}}) = \mathsf{T}(N_{\texttt{sync}})$ iff, for all $p, q \in \mathbb{P}$ with $\mathbb{P}^p_{send} = \{q\}$, we have (1) and (2):

(1) $\forall w \in outputprefixes\left(\mathcal{L}^{\emptyset}(A_p)\right).$
$\left(\left(\mathcal{L}^{\emptyset}_!(A_q){\downarrow_{\{p,q\}}}\right){\downarrow_{!?}} \subseteq \{x \mid wx \in \mathcal{L}^{\emptyset}(A_p) \wedge x = x{\downarrow_?}\}{\downarrow_{!?}}\right)$

(2) $\mathcal{L}^{\emptyset}(A_p) = \mathcal{L}^{\emptyset}_{\sqcup\!\sqcup}(p)$

The new subset condition - the (1) in this conjecture - checks for each output prefix w of some child p with parent q, whether there is some suffix x, such that x consists only of inputs and $w \cdot x \in \mathcal{L}^{\emptyset}(A_p)$. This is to enforce that after performing any amount of initial send actions, there is still some suffix x that p can perform, which receives all the sends its parent might send. The remaining parts of the theorem, such as the shuffled language condition, are unaltered.

4.3.2. Second Counterexample

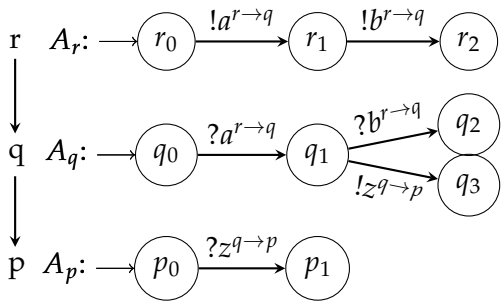


Figure 4.8.: Counterexample 2: A network that is not synchronizable, but satisfies the right side of Conjecture 4.3.1.

We continue on with the current revision of the theorem using the first revision of the subset condition, as stated in Conjecture 4.3.1. This condition specifically only considers output prefixes; thus, the idea arose to explore whether prefixes starting in inputs can break this new condition. This lead turned out to be correct, resulting in the next problematic network in Figure 4.8 and the issue elaborated in Example 4.3.3.

Example 4.3.3 (Counterexample 2)**.** The network in Figure 4.8 and resulting counterexample follow the same intuition as the previous Example 4.3.2 with its network from Figure 4.7. The difference is that here, A_r sends a second message to q, so q can first perform an input; otherwise, the automata have not changed. Both r and q now first exchange the message $a^{r\to q}$ and then have the same structure as the previous example (after states r_1 and q_1, respectively). The intuition and inference steps are similar to the previous example.

The problematic trace for this network is $!a^{r\to q}\cdot!b^{r\to q}\cdot!z^{q\to p} \in \mathsf{T}(N_{\texttt{mbox}})$, which again cannot be executed synchronously, i.e. $!a^{r\to q}\cdot!b^{r\to q}\cdot!z^{q\to p} \notin \mathsf{T}(N_{\texttt{sync}})$. The network is not synchronizable, since $!a^{r\to q}\cdot?a^{r\to q}\cdot!b^{r\to q}\cdot?b^{r\to q}\cdot!z^{q\to p}\cdot?z^{q\to p} \notin \mathsf{E}(N_{\texttt{sync}})$. There are two branches in A_q after q_1: one receives $?b^{r\to q}$ and the other sends $!z^{q\to p}$. As there are no other transitions with either of these messages in A_q, we can infer that either q sends $!z$ or receives $?b$. The synchronous execution of trace $!a\cdot!b\cdot!z$ would require q to perform both mutually exclusive actions $!z$ and $?b$. We can conclude that this network is not synchronizable.

We will now examine the two conditions on the right side of Conjecture 4.3.1. As the automata vary minimally from the last example, we can observe that $\mathcal{L}(A_r) = \mathcal{L}^{\lozenge}(A_r)$, $\mathcal{L}(A_q) = \mathcal{L}^{\lozenge}(A_q)$ and $\mathcal{L}(A_p) = \mathcal{L}^{\lozenge}(A_p)$. We will not compute the languages explicitly here again; it suffices to examine the network in Figure 4.8: q has transitions for $?a$ and $?b$, for which r can send $!a$ and $!b$ in the correct order to match q's receptions. For p's reception $?z$, q is able to send $!z$, albeit only in one branch. The influenced language does not take into account any branching; it only matters that there is some possible execution for each reception, which is the case here. There is no need to exclude any action here, so each influenced language of this network is equal to its respective peer's entire language.

We can now begin examining the shuffled language condition. It holds trivially for both p and r, as p performs only receptions and r only sends. For q, there is exactly one word containing both a send and a reception, namely $?a^{r\to q}\cdot!z^{q\to p} \in \mathcal{L}^{\lozenge}(A_q)$. Due to the reception already being to the left of the send, however, this word cannot be shuffled either. We conclude that each shuffled language is trivially equal to the respective influenced language, and thus, the shuffled language condition holds.

Left to consider is the subset condition, which we currently consider to be for all

$p, q \in \mathbb{P}$ with $\mathbb{P}^p_{send} = \{q\}$:

$$\forall w \in outputprefixes(\mathcal{L}^{\emptyset}(A_p)).((\mathcal{L}^{\emptyset}_{!}(A_q)\downarrow_{\{p,q\}}))\downarrow_{\not!\not?} \subseteq \{x \mid wx \in \mathcal{L}^{\emptyset}(A_p) \wedge x = x\downarrow_{?}\}\downarrow_{\not!\not?}.$$

The only pairs to consider are r and q, as well as q and p. For the former pair, ε is the only element in $outputprefix(\mathcal{L}^{\emptyset}(A_q))$, since q can only perform the reception $?a$ to leave its initial state. Consequently, the only set we must consider for q is $\{x \mid x \in \mathcal{L}^{\emptyset}(A_q) \wedge x = x\downarrow_{?}\}\downarrow_{\not!\not?}$, since $\varepsilon \cdot x = x$. This set contains exactly the entire upper branch and its prefixes, since the lower branch ends in a send. We infer

$$\{x \mid x \in \mathcal{L}^{\emptyset}(A_q) \wedge x = x\downarrow_{?}\}\downarrow_{\not!\not?} = \{\varepsilon, a^{r\to q}, a^{r\to q} \cdot b^{r\to q}\} = (\mathcal{L}^{\emptyset}_{!}(A_r)\downarrow_{\{q,r\}}))\downarrow_{\not!\not?},$$

and observe that the condition is satisfied for the first pair. Considering the second pair, we gather that $(\mathcal{L}^{\emptyset}_{!}(A_q)\downarrow_{\{p,q\}}))\downarrow_{\not!\not?} = \{\varepsilon, z^{q\to p}\}$, since all of q's other actions involve r instead of p. Like q, p only has the ε as its unique $outputprefix$. With analogous reasoning, we reach $\{x \mid x \in \mathcal{L}^{\emptyset}(A_p) \wedge x = x\downarrow_{?}\}\downarrow_{\not!\not?} = \{\varepsilon, z^{q\to p}\} = (\mathcal{L}^{\emptyset}_{!}(A_q)\downarrow_{\{p,q\}}))\downarrow_{\not!\not?}$. We conclude that the subset condition is satisfied in this network. Together with the shuffled language condition, we satisfy the entire right side of the *if and only if* in Conjecture 4.3.1. This contradicts our current conjecture, since the network is not synchronizable, but satisfies both conditions.

To summarize the core of the contradiction again: For the only existing output prefix ε of both q and p, respectively, the revised subset condition is satisfied. This is because from the initial state, descendants of the root can receive all the sends they might receive. For example, q is able to receive all possible sends of r, namely the messages $\varepsilon, a^{r\to q}$ and $a^{r\to q}b^{r\to q}$. However, once q has entered the branch where it sends $z^{q\to p}$, it can no longer receive $b^{r\to q}$, even though r could have already sent it or can still send it in the future. To conclude, the issue is that words starting with receptions may also clash with the possible synchronizability of the network, e.g., if some branching, like in our examples, occurs.

Taking this into account, we extended the previous subset condition to *all* possible prefixes, which are exactly all words in the language of a peer, since all states are accepting. We want to keep the remainder of the subset condition akin to the last revision, as the idea remains the same. Because we now also consider prefixes w that may contain receptions, we need to make sure that we do not discard these. In the previous version, there was no possibility for w to contain receptions, so any suffix x could be examined. Now, w might contain receptions, so the suffix x does not necessarily need to receive all messages, but only the remaining ones that w has not already received. In other words, we need to isolate the receptions that could occur in w, and then allow for any receptions x afterward.

To formally introduce the resulting second revision of the main theorem [DLP24, Theorem 4.5], we need a notation for prefixes. If we do not allow for prefixes of w, then words such as ε pose a threat as soon as w contains at least one reception. In this case, x can be ε or consist of further receptions, but cannot shorten w again. This means the word ε, which is in every peer's language and thus also in the parent's language, can never be an element of the set for w. Without this element, the set constructed around w can never be a superset of the parent's language (including the projections).

Thus, we need to allow for any prefix of the receptions in w, and hence introduce a notation for prefixes. We write *is prefix of* to refer to the non-strict standard prefix relation, e.g. $!a^{q\to p}$ is prefix of $!a^{q\to p}$.

Conjecture 4.3.2 (Second Revision of the Mailbox Synchronizability Theorem)**.** Let N be a network such that $\mathbb{C}_F = \mathbb{C}$ and $G(N)$ is a tree. Then $\mathsf{T}(N_{\texttt{mbox}}) = \mathsf{T}(N_{\texttt{sync}})$ iff, for all $p, q \in \mathbb{P}$ with $\mathbb{P}^p_{send} = \{q\}$, we have (1) and (2):

(1) $\forall w \in \mathcal{L}^{\emptyset}(A_p)$.
$((\mathcal{L}^{\emptyset}_{!}(A_q))\downarrow_{\{p,q\}})\downarrow_{\not!?} \subseteq \{yx \mid wx \in \mathcal{L}^{\emptyset}(A_p) \wedge x = x\downarrow_{?} \wedge \text{y is prefix of } (w\downarrow_{?})\}\downarrow_{\not!?}$

(2) $\mathcal{L}^{\emptyset}(A_p) = \mathcal{L}^{\emptyset}_{\sqcup}(p)$

4.3.3. Third Counterexample

With the latest revision of the subset condition (the condition (1)) as introduced in Conjecture 4.3.2, we now enforce that parents' sends can *always* be received fully, regardless of one of their children's current prefix. In other words, no matter which prefix we have read in the child automaton thus far, we can still perform the matching reception for any message that might come from the parent afterward. Another addition to this revision is that the right set considers the receptions that have already occurred in the child, as well. This was another oversight of the previous revision, since otherwise we would force the child to receive something again, even if it already received it. Because of this, we apply the *prefix* relation, which also captures potential misses like the ε, which is in each peer's language and would not be captured in this condition if $w\downarrow_{?} \neq \varepsilon$.

We can observe from Conjecture 4.3.2 that the right set in the subset relation is significantly more nuanced than the left. This sparked the idea that perhaps the left set could be too general, i.e., that now the condition has become too strong. We enforce that regardless of the prefix $w \in \mathcal{L}^{\emptyset}(A_p)$, *any* send of the parent can still be received (if it hasn't been received already). This does not consider the case in which the parent q has already entered some branch in which it can no longer send *all* the words in $\mathcal{L}^{\emptyset}(A_q)$ anymore, and consequently all of $\mathcal{L}^{\emptyset}_{!}(A_q)\downarrow_{\{p,q\}}$. Since we now allow for inputs

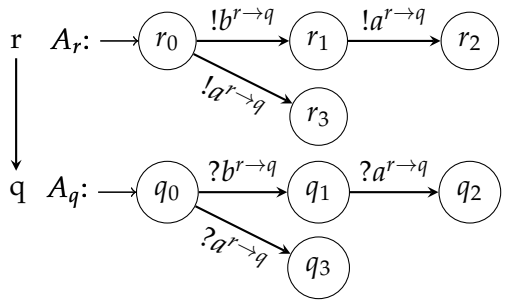


Figure 4.9.: Counterexample 3: A network that is synchronizable, but does not satisfy the right side of Conjecture 4.3.2.

in the prefixes of p as well, we indirectly force q to take certain actions. We currently consider even cases where q does not provide matching sends to the receptions p has committed to by performing w. This unfortunately poses a problem to the integrity of our current conjecture, as we illustrate in Example 4.3.4.

Example 4.3.4 (Counterexample 3)**.** Consider the network in Figure 4.9, which illustrates the issue that arises by not considering the parent's branching. Here, r can either send $!a^{r\to q}$, or $!b^{r\to q}$, as its first nonempty action. Consider $w = ?a^{r\to q} \in \mathcal{L}^{\emptyset}(A_q)$, then in all valid executions where q can perform this action, r must have sent it already, as q can only receive when that exact message is in in the first position of its buffer. Then, the possible values for y in the right set of our subset condition are $y \in \{\varepsilon, ?a^{r\to q}\}$, as those are the prefixes of our w. Since q must have entered its lower branch by performing $?a^{r\to q}$, the only possibility for x is $x \in \{\varepsilon\}$. Then we can infer

$$\{yx \mid wx \in \mathcal{L}^{\emptyset}(A_q) \wedge x = (x{\downarrow_?}) \wedge \text{y is prefix of } (w{\downarrow_?})\}{\downarrow_{\not !?}} = \{\varepsilon, ?a^{r\to q}\}{\downarrow_{\not !?}} = \{\varepsilon, a^{r\to q}\},$$

and for the root r, which is the direct parent of q, we receive

$$((\mathcal{L}_!^{\emptyset}(A_r)){\downarrow_{\{q,r\}}}){\downarrow_{\not !?}} = \{\varepsilon, !a^{r\to q}, !b^{r\to q}, !b^{r\to q}!a^{r\to q}\}{\downarrow_{\not !?}} = \{\varepsilon, a^{r\to q}, b^{r\to q}, b^{r\to q}a^{r\to q}\}.$$

We observe $\{\varepsilon, a^{r\to q}, b^{r\to q}, b^{r\to q}a^{r\to q}\} \not\subseteq \{\varepsilon, a^{r\to q}\}$. Finally, we can conclude

$$\left((\mathcal{L}_!^{\emptyset}(A_r)){\downarrow_{\{q,r\}}}\right){\downarrow_{\not !?}} \not\subseteq \{yx \mid wx \in \mathcal{L}^{\emptyset}(A_q) \wedge x = x{\downarrow_?} \wedge \text{y is prefix of } (w{\downarrow_?})\}{\downarrow_{\not !?}}.$$

It is clear that the subset condition does not hold for this network. However, this network is synchronizable, contradicting the current formalization of the main theorem in Conjecture 4.3.2 once again. The network is synchronizable, since there are only two peers, and only the root r is performing sends, which q can all receive immediately after they are sent. In other words, the only possible traces are exactly the language of r:

$\mathcal{L}(A_r) = \{\varepsilon, !a^{r\to q}, !b^{r\to q}, !b^{r\to q}!a^{r\to q}\}$. By observation, it is clear that q can receive each of the sends belonging to the respective traces in this set. Thus, $\mathsf{T}(N_{\texttt{mbox}}) = \mathsf{T}(N_{\texttt{sync}})$ and the network is synchronizable.

To conclude, this counterexample contradicts the current theorem revision, because the subset condition is now too strong. It does not track which actions are even still possible for the parent depending on the child's actions, but instead forces the child to be able to receive all sends its sends, starting from the parent's initial state. This is undesired, as in our example, it forces q to be able to receive sends that r cannot perform anymore because it must have sent $!a^{r\to q}$ already. By sending $!a^{r\to q}$ so that q can receive $?a^{r\to q}$, both q and r must enter their respective lower branches (states r_3 and q_3). From r_3, r cannot send $!b^{r\to q}$ or $!b^{r\to q}!a^{r\to q}$ anymore, but the condition still forces q to be able to receive those two impossible sends.

This, in turn, means that we must track both the parent and the child, depending on which receptions have already occurred. If the child receives a message, the parent must have sent it. Thus, we can disregard all possible words of the parent, where these matching sends have either not (fully) occurred, or where the ordering of sends does not match that of the receptions. This led us to the third and final revision of the subset condition, which resulted in Theorem 4.4.5, where the subset condition is again the condition (1) of the theorem. We also restate the final condition in Definition 4.4.1, since we will elaborate on it a bit more before stating and proving the full theorem.

4.4. Final Theorem Formalization and Proof

Definition 4.4.1 (Subset Condition Final Revision)**.** For $p, q \in \mathbb{P}$ with $\mathbb{P}^p_{send} = \{q\}$, we have

$$\forall w \in \mathcal{L}^{\emptyset}(A_p).\ \forall w' \in \mathcal{L}^{\emptyset}(A_q) : \Big(((w'{\downarrow_!}){\downarrow_{\{p,q\}}}){\downarrow_{!?}} = (w{\downarrow_?}){\downarrow_{!?}}\Big) \longrightarrow$$
$$\Big(\{(x'{\downarrow_!}){\downarrow_{\{p,q\}}} \mid w'x' \in \mathcal{L}^{\emptyset}(A_q)\}{\downarrow_{!?}} \subseteq \{x{\downarrow_?} \mid wx \in \mathcal{L}^{\emptyset}(A_p)\}{\downarrow_{!?}}\Big)$$

As mentioned in the previous section, the final subset condition (in Definition 4.4.1) now tracks both the parent q and the child p for each parent-child pair. For each word w of the child, the condition tracks all possible w' in the parent's language, which provide *exactly* the matching sends to the receptions $w{\downarrow_?}$ of w. We do not constrain these w' otherwise. In particular, w' may contain any number of sends to children other than p, or receptions if q is not the root. We only choose the words that exactly provide what p needs from q, but disregard all actions to other peers. This ensures that w' really captures all possible words for q, and in turn all possible executions that might hinder the synchronizability of a network. We know that at least one such w' that matches the receptions in w must exist, otherwise $w \notin \mathcal{L}^{\emptyset}(A_p)$ by Definition 4.2.3.

Then for every such pair w and w', we need to ensure that for all possible suffixes x' s.t. $w'x' \in \mathcal{L}^{\emptyset}(A_q)$, there is at least one suffix x, s.t. $((x'\downarrow_!)\downarrow_{\{p,q\}})\downarrow_{!?} = (x\downarrow_?)\downarrow_{!?}$. Put into words, we check for the current branch of q and of p, whether p can still receive everything q is still able to send after that point. With this, we have slightly weakened the subset condition again, so that it now only checks the reachable words, given the current prefix of p and the resulting possible words of q.

In Figure 4.10, we showcase our Isabelle formalization of the subset condition matching the definition we just detailed. Due to the complexity, we divided the set comprehension of the handwritten definition into three smaller formalizations for better clarity. The first *abbreviation* constructs a set containing all possible suffixes, projected to only receptions, of a given word w and peer. For the given peer p, $w \in \mathcal{L}^{\emptyset}(p)$ should hold when applying this construct, otherwise the result is not useful. We use this first formalization to gather all the receptions possible for the child after performing w. Next, we need to formalize all sends the parent is still capable of after performing w'. In the second *abbreviation*, we provide q, w', and p, where q should be the parent of p for the result to be useful, to gather the possible send suffixes for q. The definition on the bottom combines the two formalizations into the entire subset condition, also ensuring the correct preconditions hold. The formalizations from Figure 4.10 and Definition 4.4.1 look almost equivalent, except for the differing symbols. The $\mathcal{L}^*(p)$ in Isabelle refers to the influenced language $\mathcal{L}^{\emptyset}(A_p)$. All other symbols are close enough to the ones in this thesis, apart from the two shorthands for the sets from Definition 4.4.1, which are introduced in the abbreviations directly above the *definition*.

Figure 4.10.: Formalization of the Final Subset Condition

```
abbreviation possible_recv_suffixes :: "('information, 'peer) action word ⇒ 'peer
⇒ ('information, 'peer) action language"  ("‡_‡_" [90, 90] 110) where
  "‡w‡p ≡ {x↓? | x. (w ▪ x) ∈ ℒ*(p)}"

abbreviation possible_send_suffixes_to_peer :: "'peer ⇒ ('information, 'peer)
action word ⇒ 'peer ⇒  ('information, 'peer) action language"  ("_‡_‡_" [90, 90,
90] 110) where
  "q‡w‡p ≡ {(x↓!)↓{p,q} | x. (w ▪ x) ∈ ℒ*(q)}"

definition subset_condition :: "'peer ⇒ 'peer ⇒ bool"
  where "subset_condition p q ⟷ (∀ w ∈ ℒ*(p). ∀ w' ∈ ℒ*(q).
  (((w'↓!)↓{p,q})↓!? = ((w↓?)↓!?)) ⟶ ((q‡w'‡p)⌊!? ⊆ (‡w‡p)⌊!? ))"
```

Now that our final version of the subset condition is formalized, we will start proving

and verifying the new formalization of the main theorem. The original proof of the theorem [DLP24, Theorem 4.5] is now unfortunately mostly obsolete, as the change of both the influenced language and subset condition affects the previous reasoning a great amount. We still need one more result from the source paper; the following lemma [DLP24, Lemma 4.4] is proposed and proven:

Lemma 4.4.2 (Original Lemma 4.4)**.** *Let N be a network such that $\mathbb{C}_F = \mathbb{C}$, $G(N)$ is a tree, $q \in \mathbb{P}$, and $w \in \mathcal{L}^{\emptyset}(A_q)$. Then there is an execution $w' \in \mathsf{E}(N_{\texttt{mbox}})$ such that $w'{\downarrow_q} = w$ and $w'{\downarrow_p} = \varepsilon$ for all $p \in \mathbb{P}$ with $\mathbb{P}^p_{send} = \{q\}$.*

This lemma is built upon the original version of the influenced language in Definition 4.2.1, which we have revised to Definition 4.2.3 for this work. This has implications for the proof in the paper as well, since it suffers from the same oversight as the original influenced language. In fact, the proof can be corrected by adding the missing projection to $\downarrow_{\{p,q\}}$ to the parts of the proof where the influenced language is actively used. Apart from this, the original proof follows directly from the construction of the influenced language, and thus, we will not rewrite the entire proof here, as adding the projections to the original proof is the only change needed. To summarize, the proof for this lemma in [DLP24, Lemma 4.4] is only missing the projections to p and q in the proof when applying the influenced language. Since we are using this lemma in the proof of the adjusted main theorem and plan to highlight our formalization of this lemma, we will now provide some intuition for the reasoning behind it.

The proof works as follows. First, we obtain the unique[9] path from the root r to the peer q. Since $w \in \mathcal{L}^{\emptyset}(A_q)$, either $q = r$ and we are done, or q has some parent h, and by the definition of the influenced language, there is some $w' \in \mathcal{L}^{\emptyset}(A_h)$ such that $((w'{\downarrow_!}){\downarrow_{\{q,h\}}}){\downarrow_{\not!?}} = (w{\downarrow_?}){\downarrow_{\not!?}}$. This reasoning can be repeated until the root is reached. Simultaneously, we accumulate all the visited words s.t. in the end we receive a word $w_r \dots w'w$, where the leftmost component is the root word, and the rightmost component is the word of q we started with. Through this construction, all sends happen in the correct order and all existing receptions also happen in the correct order. We must make the distinction of existing receptions, since we only visit the peers on the path, and so only these peers may perform actions in the accumulated word. However, peers can send messages to multiple children, so there may be some sends in the accumulation that are never received, but this is not a problem, as we are constructing an asynchronous execution within this lemma. To summarize, we can accumulate this word by the construction of the influenced language, and this word is a valid mailbox execution, because by construction: all sends happen before the receptions, and all receptions are performed in the correct order.

[9] Since the network is a tree, there can only be *exactly one* path between each respective peer pair.

We mechanized most steps of (the proof of) this lemma in Isabelle using the construction *concat_infl* from Figure 4.11, but some minor results are left for future work. Most missing parts are intuitively true; for example, the property that a unique path from the root to q exists. This follows directly from the tree topology, as a tree is connected, and each peer has at most one parent. We will detail these missing results and why we have not fully mechanized them in the discussion in Chapter 5.

Figure 4.11.: Formalization of the Construction for Lemma 4.4

inductive concat_infl :: "'peer ⇒ ('information, 'peer) action word ⇒ 'peer list ⇒ ('information, 'peer) action word ⇒ bool" **for** p::"'peer" **and** w:: "('information, 'peer) action word" **where**
at_p: "⟦tree_topology; w ∈ $\mathcal{L}^*$(p); path_to_root p ps⟧ ⟹ concat_infl p w ps w" |
reach_root: "⟦is_root q; qw ∈ $\mathcal{L}^*$(q); path_to_root x (x # [q]); (∀ g. w_acc$\downarrow_g$ ∈ $\mathcal{L}^*$(g)); concat_infl p w (x # [q]) w_acc; (((w_acc$\downarrow_x$)$\downarrow_?$)$\downarrow_{!?}$) = (((qw$\downarrow_{\{x,q\}}$)$\downarrow_!$)$\downarrow_{!?}$)⟧ ⟹ concat_infl p w [q] (qw ▪ w_acc)" |

node_step: "⟦tree_topology; $\mathcal{P}_?$(x) = {q}; (∀ g. w_acc$\downarrow_g$ ∈ $\mathcal{L}^*$(g)); path_to_root x (x # q # ps); qw ∈ $\mathcal{L}^*$(q); concat_infl p w (x # q # ps) w_acc; (((w_acc$\downarrow_x$)$\downarrow_?$)$\downarrow_{!?}$) = (((qw$\downarrow_{\{x,q\}}$)$\downarrow_!$)$\downarrow_{!?}$)⟧ ⟹ concat_infl p w (q#ps) (qw ▪ w_acc)"

Our mechanization in Figure 4.11 works as follows: there are three distinct rules to apply, depending on whether we are currently at the starting node p, whether we have reached the root, or whether we are some step in between p and the root. The construction always keeps track of the peer p and word w for which we want to apply the lemma as the first two parameters. The third parameter ps is the path from p to the root, and the fourth is the accumulated word. In the initial step, we start accumulating w. Then, through an arbitrary number of *node_step* applications, we check that we have a node x at the current place of our path, with its parent q directly after, and then add the word providing necessary receptions for x's part in the accumulated word to it. When the root is reached, i.e., when the path to the root contains only two nodes (the root and our current node), we do the same as in *node_step*. If the initially given peer p is the root, only *at_p* can be applied and we end with $(concat\_infl\ p\ w\ [p]\ w)$. We use this construction in our formalization of the proof for [DLP24, Lemma 4.4], by obtaining a *concat_infl* for p and w, where the third parameter contains only the root. Because of our construction, this then means that either p is the root or the fourth parameter is the accumulated word with the properties we enforced in every inductive step. Our full proof is in Appendix A.3.1, but relies on several helper lemmas about

concat_infl which are not shown in this work, but are of course available at [Ket25].

We realized that the properties we have set in place for this lemma are not entirely sufficient for our new formalization of the theorem. In particular, the construction relies heavily on the tree's property that each path between any two nodes is unique. This property was hard to implement given our current formalization in Isabelle, as we will discuss in Chapter 5, but it should be noted in general. Thus, we would like to revise the original lemma's statement as proposed in [DLP24, Lemma 4.4], to make it even stronger. The execution w' is constructed by including only peers on the path from root to q, hence $w'{\downarrow_p} = \varepsilon$ for all $p \in \mathbb{P}$ with $\mathbb{P}^p_{send} = \{q\}$ holds as the lemma claims. Note that this only means that all direct children of q do not perform actions in w'. However, any further descendants of q (children of such p, and so on) are also not actors for any action in w', as they cannot be on the traversed path to q. In fact, no peer outside of those on the path can perform actions in w' themselves (they can still be sent to, however). Thus, we strengthen the last condition and use this slightly changed version of the original lemma for the remainder of this work[10]:

Lemma 4.4.3 (Strengthened Lemma 4.4)**.** *Let N be a network such that $\mathbb{C}_F = \mathbb{C}$, $G(N)$ is a tree, $q \in \mathbb{P}$, and $w \in \mathcal{L}^{\emptyset}(A_q)$. Let $q, \ldots, r$ be the unique path from q to the root r, and let $B \subseteq \mathbb{P}$ denote the set of peers involved in that path, i.e. $q, \ldots, r \in B$. Then there is an execution $w' \in \mathsf{E}(N_{\texttt{mbox}})$ such that $w'{\downarrow_q} = w$ and $w'{\downarrow_p} = \varepsilon$ for all $p \in B$.*

We have introduced one of the needed lemmas, but as the proof of the main theorem is complex, we need to introduce more formalizations before we can state and prove it. As a reminder, we are still using the same scheme (see Section 4.3) as the original theorem [DLP24, Theorem 4.5], but using our final revised subset condition from Definition 4.4.1.

To prove the direction that assumes synchronizability and shows the two conditions on the right hold, we need to define two constructions first. In the following, we will quickly explain the main idea and show pseudocode for those constructions. The Isabelle code for them, which is quite similar in syntax to functional programming languages, can be referenced in Figures 4.12 and 4.13, respectively. Let $w[i]$ denote word w at index i. It is important to note that the following two constructions implicitly assume that each message is unique, i.e., there is no such case where the message a is sent between more than one unique pair of peers. Additionally, within the language of one peer, all messages are also distinct; this follows from the previous point.[11]

[10] In Isabelle, we currently only use this strengthened version implicitly through some of the helper lemmas used in Proof A.3.1, but it needs to be changed to the explicit stronger version there as well, to allow for full verification of the source paper.

[11] If messages are not unique, they can be made unique, e.g., by adding subscripts that tag the peers involved to the message.

For ($\Rightarrow$ 1.), we use a construction as detailed in Figure 4.12 which mixes two words $w \in \mathcal{L}^{\emptyset}(A_p)$ and $w' \in \mathcal{L}^{\emptyset}(A_q)$, where p is q's direct child and $((w'{\downarrow_!}){\downarrow_{\{p,q\}}}){\downarrow_{\not!?}} = (w{\downarrow_?}){\downarrow_{\not!?}}$ holds. We denote with (mix_pair (w', w)) a call to the function as introduced in Figure 4.12 above over w' and w. When we refer to the construction without applying it explicitly, we write *mix_pair* instead.

Figure 4.12 shows the recursive Isabelle formalization, for which we also provide iterative pseudocode in Listing A.1 of the appendix. The function takes two words w' and w and accumulates the result in the third parameter. The first three cases should be self-explanatory[12], the only complicated case is the last case, where both words are non-empty.

Figure 4.12.: Formalization of *mix_pair*

```
fun mix_pair :: "('information, 'peer) action word ⇒ ('information, 'peer) action
word ⇒ ('information, 'peer) action word ⇒ ('information, 'peer) action word"
where
 "mix_pair [] [] acc = acc" |
 "mix_pair (a # w') [] acc = mix_pair w' [] (a # acc)" |
 "mix_pair [] (a # w) acc = mix_pair [] w (a # acc)" |
 "mix_pair (a # w') (b # w) acc  = (if a = !⟨get_message b⟩
     then (if b = ?⟨get_message a⟩ then mix_pair w' w (a # b # acc) else mix_pair
(a # w') w (b # acc))
     else mix_pair w' (b # w) (a # acc))"
```

Intuitively, whenever a send from q to p occurs in w', we append the matching receive of p for that message directly afterwards. All actions not concerning p and q, e.g., sends and receptions from q involving other peers, or sends from p to its children, are appended s.t. that their original order is kept. In particular, (mix_pair $(w', w)){\downarrow_q} = w'$ and (mix_pair $(w', w)){\downarrow_p} = w$ hold. We alternate sends and receptions of p and q. The case distinction in the algorithm is sufficient, because of the assumption that $((w'{\downarrow_!}){\downarrow_{\{p,q\}}}){\downarrow_{\not!?}} = (w{\downarrow_?}){\downarrow_{\not!?}}$, meaning for each j s.t. $w'[j] = !a^{q \to p}$, there is some i s.t. $w[i] = ?a^{q \to p}$. The (implicit) outer loop is over j, so when $w'[j] = !a^{q \to p}$, we know that the matching reception (i.e. the i, s.t. $w[i] = ?a^{q \to p}$) is either at the current i, or at some future i within w, again due to the assumption. To summarize thus far, we accumulate a return value w_{mix} which puts each matching send and reception directly after one another, while keeping the order of the remaining actions for both words. If the returned word is part of an execution e, s.t. $e \in \mathsf{E}(N_{\texttt{mbox}})$ and e contains no other

[12] If there is nothing left to accumulate, we are done; if only one of the two words still has actions, accumulate those until empty.

actions of p or q, then a synchronous execution with the same trace as e would also satisfy $e{\downarrow_q} = w'$ and $e{\downarrow_p} = w$.

For ($\Rightarrow$ 2.), we use *mix_pair* as explained above to construct a slightly different version of a mix between two words in Definition 4.4.4. As with the previous construction, we denote a call to the function over some w'', w' and w with (mix_shuf (w'', w', w)), while we refer to the construction overall with *mix_shuf*.

Definition 4.4.4 (mix_shuf)**.** Let $p, q, x \in \mathbb{P}$ with $\mathbb{P}^p_{send} = \{q\}, \mathbb{P}^x_{send} = \{p\}$, let $w \in \mathcal{L}^{\lozenge}(A_p)$, $(xs \cdot !a^{p \to x} \cdot ?b^{q \to p} \cdot ys) = w$ and $(xs \cdot ?b^{q \to p} \cdot !a^{p \to x} \cdot ys) = w' \in \mathcal{L}^{\lozenge}_{\sqcup\!\sqcup}(p)$ s.t. $w' \sqcup\!\sqcup_? w$. Let $(as \cdot !b^{q \to p} \cdot bs) = w'' \in \mathcal{L}^{\lozenge}(A_q)$.
Then we construct
mix_shuf $(w'', w', w) :=$ $((\text{mix\_pair } (as, xs)) \cdot !b^{q \to p} \cdot !a^{p \to x} \cdot ?b^{q \to p} \cdot (\text{mix\_pair } (bs, ys)))$

The implementation for this construction can be found in Figure 4.13. We chose an *inductive* here, but this could be implemented as another function as well. Since we want to use *mix_shuf* only when we have certain preconditions, and want to continue with the results in proofs, an *inductive* seemed like the better choice. The *inductive* only has one rule, where the result of *mix_shuf* is accumulated by concatenating calls of *mix_pair*, if the preconditions are met.

Figure 4.13.: Formalization of *mix_shuf*

inductive mix_shuf :: "('information, 'peer) action word ⇒ ('information, 'peer) action word ⇒ ('information, 'peer) action word ⇒ ('information, 'peer) action word ⇒ bool" **where**
mix_shuf_constr: "⟦vq↓!↓{p,q}↓!? = v↓?↓!?; v' ∈ $\mathcal{L}^*{}_{\sqcup\!\sqcup}$(p); v' ⊔⊔? v; v ∈ $\mathcal{L}^*$(p); vq ∈ $\mathcal{L}^*$(q); vq = (as ▪ a_send # bs); v = xs ▪ b # a_recv # ys; get_message a_recv = get_message a_send; is_input a_recv; is_output a_send; is_output b⟧
⟹ mix_shuf vq v v' ((mix_pair as xs []) ▪ a_send # b # a_recv # (mix_pair bs ys []))"

Intuitively, w and w' in this construction are equal, except for one input-output pair in the middle, i.e., exactly one shuffle occurred from w to w'. We can infer (mix_shuf $(w'', w', w)){\downarrow_q} = w''$, by the construction of (mix_pair (w'', w)). We can observe that (mix_shuf (w'', w', w)) performs (mix_pair(w'', w)), except for the part where w' and w differ. However, (mix_shuf $(w'', w', w)){\downarrow_p} = w$, since the ordering is kept. To showcase the effect of this construction, consider an execution e, s.t. (mix_shuf (w'', w', w)) occurs as a whole in $e \in \mathsf{E}(N_{\texttt{mbox}})$ and $e{\downarrow_q} = w''$ and $e{\downarrow_p} = w$. Due to the placement of $!b^{q \to p}$ before $!a^{p \to x} \cdot ?b^{q \to p}$, a synchronous execution e' with

the same trace as e would now satisfy $e'\downarrow_p = w'$. Otherwise, not every send would immediately be followed by the matching receive, and e' wouldn't be synchronous. We will explain how this and the first construction are relevant when they occur in the proof, as without their context, they may seem quite arbitrary.

As hinted at before, most of our proof execution is entirely different from the source, but some of the proof structure and ideas were extremely helpful in our formalization. The original proof uses a similar approach in the first direction of the theorem, but their construction is not sufficiently specific, and thus, the claim is not proven in all cases of the arbitrary words that are chosen in the original proof [DLP24, Theorem 4.5]. For the first case of the second direction, we operate with the same idea, but due to the subset condition becoming stricter, this part of the proof also had to become more elaborate.

Apart from the condition changes needed, the original proof was not fully formalized either way. In some parts of the original proof, cases were missed or simply not explicitly mentioned, although they are formally required[13]. And of course in other parts of the proof, we have adjusted the formalization. Because of this, most of the original proof is obsolete regardless of the exact reasons; thus, we will not detail all minute formalization errors or oversights of the original proof. We instead provide and explain our new version in detail. In general, we have made some of the implicit steps from the source explicit and provided definitions for some constructions used. To conclude, for the first direction, the proof idea is similar to the original one in [DLP24, Theorem 4.5], although the execution is different and a bit more detailed. For the second direction, the case distinction and the proof for the second case remain true to the original, but the first case is completely changed due to the change in the subset condition.

Figure 4.14.: Formalization of the Final Main Theorem

definition theorem_rightside :: "bool"
  where "theorem_rightside $\longleftrightarrow$ ($\forall$ p $\in$ $\mathcal{P}$. $\forall$ q $\in$ $\mathcal{P}$. ((is_parent_of p q) $\longrightarrow$ ((subset_condition p q) $\wedge$ (($\mathcal{L}^*$(p)) = ($\mathcal{L}^*{}_{\sqcup\sqcup}$(p))))))"

theorem synchronisability_for_trees:
  assumes "tree_topology"
  shows "is_synchronisable $\longleftrightarrow$ (($\forall$ p $\in$ $\mathcal{P}$. $\forall$ q $\in$ $\mathcal{P}$. ((is_parent_of p q) $\longrightarrow$ ((subset_condition p q) $\wedge$ (($\mathcal{L}^*$(p)) = ($\mathcal{L}^*{}_{\sqcup\sqcup}$(p)))))))" (**is** "?L $\longleftrightarrow$?R")

[13]For example, the first direction of the proof does a case distinction on the possibilities of the shuffle. They only prove the claim for the case where the word can be decomposed in a certain way, which does not necessarily generalize to all words (which are needed to prove the claim, however).

We now finally present the revised main theorem and our proof. The lengthy Isabelle proof for it can be found in Appendix A.3.2, but we will detail the proof here in a more regular format. Figure 4.14 contains our mechanization of the theorem in Isabelle, and Theorem 4.4.5 states it in regular format. As the Isabelle formalization contains no new symbols, and the subset condition in the theorem has been detailed, we simply provide the two definitions as a reference.

Theorem 4.4.5 (Final Mailbox Synchronizability Theorem)**.** *Let N be a network such that $\mathbb{C}_F = \mathbb{C}$ and $G(N)$ is a tree. Then $\mathsf{T}(N_{\texttt{mbox}}) = \mathsf{T}(N_{\texttt{sync}})$ iff, for all $p, q \in \mathbb{P}$ with $\mathbb{P}^p_{send} = \{q\}$, we have the subset condition (1) and the shuffled language condition (2):*

(1) $\forall w \in \mathcal{L}^{\emptyset}(A_p).\, \forall w' \in \mathcal{L}^{\emptyset}(A_q) : \Big(((w'{\downarrow_!}){\downarrow_{\{p,q\}}}){\downarrow_{\not!?}} = (w{\downarrow_?}){\downarrow_{\not!?}}\Big) \longrightarrow$
$\Big(\{(x'{\downarrow_!}){\downarrow_{\{p,q\}}} \mid w'x' \in \mathcal{L}^{\emptyset}(A_q)\}{\downarrow_{\not!?}} \subseteq \{x{\downarrow_?} \mid wx \in \mathcal{L}^{\emptyset}(A_p)\}{\downarrow_{\not!?}}\Big)$

(2) $\mathcal{L}^{\emptyset}(A_p) = \mathcal{L}^{\emptyset}_{\sqcup\!\sqcup}(p)$

The proof of this revised theorem is lengthy, as we have made the subset condition significantly more complex. Because of this, we have divided the theorem into two lemmas (4.4.6 and 4.4.7) so we can elaborate on the proof ideas and methods more easily. The proof of the entire theorem follows from the conjunction of both referenced lemmas. We will begin with the Lemma 4.4.6, which details the first direction of the proof where synchronizability is assumed and the two conditions are shown.

In this direction, we utilize the two constructions from Figures 4.12 and 4.13 in Parts 1. and 2. of the proof, respectively. The proof idea and partition of this first part remains the same as in the original [DLP24, Theorem 4.5 "⇒"]. Since w and w' are chosen arbitrarily, we cannot assume much about their structure. In particular, we make the original proof's steps more explicit here using our constructions.

Lemma 4.4.6 (First Direction of Final Theorem)**.** *Let N be a network such that $\mathbb{C}_F = \mathbb{C}$ and $G(N)$ is a tree. Let $\mathsf{T}(N_{\texttt{mbox}}) = \mathsf{T}(N_{\texttt{sync}})$. Then, for all $p, q \in \mathbb{P}$ with $\mathbb{P}^p_{send} = \{q\}$, we have the subset condition (1) and the shuffled language condition (2):*

(1) $\forall w \in \mathcal{L}^{\emptyset}(A_p).\, \forall w' \in \mathcal{L}^{\emptyset}(A_q) : \Big(((w'{\downarrow_!}){\downarrow_{\{p,q\}}}){\downarrow_{\not!?}} = (w{\downarrow_?}){\downarrow_{\not!?}}\Big) \longrightarrow$
$\Big(\{(x'{\downarrow_!}){\downarrow_{\{p,q\}}} \mid w'x' \in \mathcal{L}^{\emptyset}(A_q)\}{\downarrow_{\not!?}} \subseteq \{x{\downarrow_?} \mid wx \in \mathcal{L}^{\emptyset}(A_p)\}{\downarrow_{\not!?}}\Big)$

(2) $\mathcal{L}^{\emptyset}(A_p) = \mathcal{L}^{\emptyset}_{\sqcup\!\sqcup}(p)$

Proof. Assume $\mathsf{T}(N_{\texttt{mbox}}) = \mathsf{T}(N_{\texttt{sync}})$.

1. **To show:** the subset condition (1) holds.
   Proof: Fix w, w' s.t. $((w'\!\downarrow_!)\!\downarrow_{\{p,q\}})\!\downarrow_{\not!\not?} = (w\!\downarrow_?)\!\downarrow_{\not!\not?}$ as in the precondition of the subset condition; fix x' s.t. $w'x' \in \mathcal{L}^{\emptyset}(A_q)$. We now have to show, that there exists some x with $wx \in \mathcal{L}^{\emptyset}(A_p)$ and $(((w'x')\!\downarrow_!)\!\downarrow_{\{p,q\}})\!\downarrow_{\not!\not?} = ((wx)\!\downarrow_?)\!\downarrow_{\not!\not?}$, which is the subset condition (1) rewritten on words instead of subsets. With these assumptions, we will now do a case distinction over whether q is the root or not.
   Case: q is the root.
   If q is the root, then $w'x'$ consists only of sends and hence $w'x' \in \mathsf{E}(N_{\texttt{mbox}})$, because $w'x'$ cannot contain receptions, and sends cannot be blocked without receptions being present.
   Then obtain $w_{mix} = (\text{mix\_pair}\ (w', w))$ and append x' to this to receive the execution $e = w_{mix} \cdot x' \in \mathsf{E}(N_{\texttt{mbox}})$. This is an execution, since both w' and x' cannot receive anything from any other peer (as q is the root), and w' provides all necessary sends and in the correct order for w, by assumption. Then using synchronizability, obtain a synchronous execution e' for the trace of the constructed execution, s.t. $e\!\downarrow_! = e'\!\downarrow_!$ and $e' \in \mathsf{E}(N_{\texttt{sync}})$. Then we can obtain x where $e'\!\downarrow_p = w \cdot x$. This is possible, since p only receives from q and by the construction of w_{mix}, each send of q to p in w' is directly followed by the reception in w. If x' now contains any further sends to p, p will perform the receptions in e' after performing the actions in w. Thus, $e'\!\downarrow_p$ consists of exactly w and some x as a peer word of p, which receives $w' \cdot x'$ exactly, and the subset condition (1) is proven for the case where q is the root.
   Case: q is not the root.
   Then q is some node in the tree with some parent r. The construction process of the two executions is analogous, but since q is not the root, finding e is more complicated. First, we obtain some w'' s.t. $((w''\!\downarrow_!)\!\downarrow_{\{q,r\}})\!\downarrow_{\not!\not?} = ((w' \cdot x')\!\downarrow_?)\!\downarrow_{\not!\not?}$, which must exist since $(w' \cdot x') \in \mathcal{L}^{\emptyset}(A_q)$ by assumption.
   Then, we apply Lemma 4.4.3 to obtain execution e for w''. By construction, p and q perform no actions here and q gets sent the necessary sends to perform $w' \cdot x'$, while p gets sent nothing, and p and q send and receive nothing in e. We could append $w' \cdot x'$ directly to e, this is possible since w'' provides all sends for $w' \cdot x'$ and $e\!\downarrow_r = w''$. Instead, we first want to "mix" w' and w to construct a certain appendix for e. So, we obtain $w_{mix} = (\text{mix\_pair}\ (w'', w))$ and in turn $e' = e \cdot w_{mix} \cdot x'$: this is possible, since e provides all the necessary sends for $w' \cdot x'$, and w_{mix} is constructed s.t. each reception of w has the corresponding send of q directly before it (and by assumption w' and w have matching sends and receptions). Furthermore, x' can be performed at the end, since $w_{mix}\!\downarrow_q = w'$ and thus q is still in the position to perform x' (peers cannot block each other's actions

directly, so whether w is performed inbetween e and x' or not has no impact on q). Since the network is synchronizable, obtain the synchronous execution e'' with the same trace as e'. By construction of e', we can infer that both e'' and e' projected to only actions between p and q, before x' (i.e. sends from q to p and receptions of these sends) are equal. Since w_{mix} performs each send of q directly before the reception of p (i.e., simulating the synchronous execution between these two peers), e'' must also contain w in its execution, otherwise a different trace is performed. By construction, then $e''{\downarrow_p} = w \cdot x$ for some x, s.t. $wx \in \mathcal{L}^{\emptyset}(A_p)$ and $(((w'x'){\downarrow_!}){\downarrow_{\{p,q\}}}){\downarrow_{!?}} = ((wx){\downarrow_?}){\downarrow_{!?}}$.
In e'', the projection on q may not be $w'x'$ (if w' contains any receptions, these will get pulled further left, where the respective sends to q are located by construction of e). The important part is, however, that $(e'{\downarrow_q}){\downarrow_!} = (e{\downarrow_q}){\downarrow_!}$, i.e. the sends of this projection are the same as the sends in $w'x'$, which is exactly what is needed for p (p's actions are not influenced by what receptions or sends to different children q performs). Finally, we have found a matching x for our arbitrary x' and thus shown the subset condition also for the case where q is not the root.

2. **To show:** the shuffled language condition (2) holds.
   Proof: By Definition 4.2.3, $\mathcal{L}^{\emptyset}(A_p) \subseteq \mathcal{L}^{\emptyset}_{\sqcup\!\sqcup}(p)$. Assume $v' \in \mathcal{L}^{\emptyset}_{\sqcup\!\sqcup}(p)$. We have to show that $v' \in \mathcal{L}^{\emptyset}(A_p)$. Since $v' \in \mathcal{L}^{\emptyset}_{\sqcup\!\sqcup}(p)$, then by definition of the shuffled language, there is some v such that $v \in \mathcal{L}^{\emptyset}(A_p)$ and $v' \sqcup\!\sqcup_? v$. We prove $v' \in \mathcal{L}^{\emptyset}(A_p)$ by induction over the shuffle $v' \sqcup\!\sqcup_? v$. There are three cases: either no shuffle occurred, exactly one occurred, or v' was received through transitivity of the shuffle relation from Definition 3.2.1. The first and last case are both trivial, and Isabelle was able to prove them directly, so the handwritten proof of these will be left as a small exercise to the reader. We will now detail the only complicated case, where exactly one shuffle occurred, so v and v' can be decomposed into some $(xs \cdot ?b^{q \to p} \cdot !a^{p \to x} \cdot ys) = v' \in \mathcal{L}^{\emptyset}_{\sqcup\!\sqcup}(p)$ and $(xs \cdot !a^{p \to x} \cdot ?b^{q \to p} \cdot ys) = v \in \mathcal{L}^{\emptyset}(A_p)$.
   We can infer that p is a node in the tree with some parent q, as the root cannot receive messages, ruling out any non-trivial shuffles. Then, obtain $vq \in \mathcal{L}^{\emptyset}(A_q)$, s.t. $((vq{\downarrow_!}){\downarrow_{\{p,q\}}}){\downarrow_{!?}} = (v{\downarrow_?}){\downarrow_{!?}}$, by applying the definition of the influenced language (Definition 4.2.3). Similar to the previous proof part, we now do a case distinction over whether q is the root or some node with parent r.
   Case: q is the root.
   Then vq contains no receptions and consequently vq is a valid execution in $\mathsf{E}(N_{\texttt{mbox}})$. Next, we construct $w_{mix} = (\text{mix\_shuf}\ (vq, v', v))$, i.e. mix vq with v, which then is also an execution $\in \mathsf{E}(N_{\texttt{mbox}})$ by construction, as explained beneath Definition 4.4.4. Using synchronizability, we then obtain the synchronous execution $e' \in \mathsf{E}(N_{\texttt{sync}})$ with the same trace as w_{mix}. By construction, $e'{\downarrow_p} = v'$

and consequently $v' \in \mathcal{L}^{\lozenge}(A_p)$, otherwise e' is not an execution.
Case: q is not the root. Then q is a node with parent r. We obtain $vr \in \mathcal{L}^{\lozenge}(A_r)$ which provides exactly matching sends for vq using the definition of the influenced language (Definition 4.2.3). Then apply Lemma 4.4.3 to vr and receive execution $e \in \mathsf{E}(N_{\texttt{mbox}})$ which contains no actions of p or q, respectively, by construction.
Next, obtain $w_{mix} = (\text{mix\_shuf}\ (vq, v', v))$. Then, $e \cdot w_{mix} \in \mathsf{E}(N_{\texttt{mbox}})$, since both p and q are in their initial states after e, and e provides all sends for q in the correct order and w_{mix} keeps all sends of q before the receptions of p. Now we again use synchronizability to receive $e' \in \mathsf{E}(N_{\texttt{sync}})$ with $e'{\downarrow_!} = (e \cdot w_{mix}){\downarrow_!}$. By construction of w_{mix} and e', we know that $e'{\downarrow_p} = v'$. Consequently, $v' \in \mathcal{L}^{\lozenge}(A_p)$ must hold, or e' is no execution.
In all cases and for an arbitrary v', we have shown that $v' \in \mathcal{L}^{\lozenge}(A_p)$, so we can conclude that $\mathcal{L}^{\lozenge}(A_p) = \mathcal{L}^{\lozenge}_{\sqcup\!\sqcup}(p)$.

□

This concludes the first direction of the proof. The second direction is more complex, since synchronizability is an extremely strong property. Additionally, we changed the subset condition, so the original proof was no longer applicable to our new situation. We keep the case distinction over $w \in \mathsf{T}(N_{\texttt{mbox}})$ or $w \in \mathsf{T}(N_{\texttt{sync}})$ and then show that w is present in the respective other trace set as well. We also keep the induction of the first case and the structure of the second case, but the main matters of the inductions are a bit different, due to the distinct subset conditions. In particular, our version has to make some more detailed and explicit steps in order to be able to apply the subset condition to receive the correct result. Since the new subset condition is more constrained, some more pre- and post-work is necessary to apply it for this portion of the proof.

In this part, we use the formalization in Figure 4.15 to make the "word obtained from w by adding the matching receive action directly after every send action" used in the original (and by extension our) proof explicit [DLP24, Theorem 4.5 "⇐"].

Figure 4.15.: Formalization of *add_matching_recvs*

```
fun add_matching_recvs :: "('information, 'peer) action word ⇒ ('information,
'peer) action word" where
  "add_matching_recvs [] = []" |
  "add_matching_recvs (a # w) = (if is_output a
     then a # (?⟨get_message a⟩) # add_matching_recvs w
     else a # add_matching_recvs w)"
```

Lemma 4.4.7 (Second Direction of Final Theorem)**.** *Let N be a network such that $\mathbb{C}_F = \mathbb{C}$ and $G(N)$ is a tree. Let the subset condition (1) and the shuffled language condition (2) hold for all $p, q \in \mathbb{P}$ with $\mathbb{P}^p_{send} = \{q\}$. Then, $\mathsf{T}(N_{\mathtt{mbox}}) = \mathsf{T}(N_{\mathtt{sync}})$.*

(1) $\forall w \in \mathcal{L}^{\emptyset}(A_p).\, \forall w' \in \mathcal{L}^{\emptyset}(A_q) : \Big(((w'{\downarrow_!}){\downarrow_{\{p,q\}}}){\downarrow_{\not{!}?}} = (w{\downarrow_?}){\downarrow_{\not{!}?}}\Big) \longrightarrow$
$\Big(\{(x'{\downarrow_!}){\downarrow_{\{p,q\}}} \mid w'x' \in \mathcal{L}^{\emptyset}(A_q)\}{\downarrow_{\not{!}?}} \subseteq \{x{\downarrow_?} \mid wx \in \mathcal{L}^{\emptyset}(A_p)\}{\downarrow_{\not{!}?}}\Big)$

(2) $\mathcal{L}^{\emptyset}(A_p) = \mathcal{L}^{\emptyset}_{\sqcup\!\sqcup}(p)$

Proof. Assume that the subset condition (as in Definition 4.4.1) and $\mathcal{L}^{\emptyset}(A_p) = \mathcal{L}^{\emptyset}_{\sqcup\!\sqcup}(p)$ hold for all $p, q \in \mathbb{P}$ with $\mathbb{P}^p_{send} = \{q\}$. We have to show that $\mathsf{T}(N_{\mathtt{mbox}}) = \mathsf{T}(N_{\mathtt{sync}})$.

Case: $w \in \mathsf{T}(N_{\mathtt{mbox}})$. Let w' be the word obtained from w by adding the matching receive action directly after every send action.[14] We show that $w' \in \mathsf{E}(N_{\mathtt{mbox}})$, by an induction on the length of w.

Base Case: If $w = !a^{q\to p}$, then $w' = !a^{q\to p}?a^{q\to p}$. Since $w \in \mathsf{T}(N_{\mathtt{mbox}})$, A_q is able to send $!a^{q\to p}$ in its initial state within the system $N_{\mathtt{mbox}}$. Then $!a^{q\to p} \in \mathcal{L}^{\emptyset}_{!}(A_q)$. We apply the subset condition (1) to $\varepsilon \in \mathcal{L}^{\emptyset}(A_p)$ and $\varepsilon \in \mathcal{L}^{\emptyset}(A_q)$ and the suffix $!a^{q\to p}$. Since $\varepsilon \cdot !a^{q\to p} = !a^{q\to p} \in \mathcal{L}^{\emptyset}(A_q)$, p must then be able to perform some word $x \in \mathcal{L}^{\emptyset}(A_q)$ from its initial state, which has $?a^{q\to p}$ as the only reception. We only know about x that it contains exactly this input, but it may also contain sends. This is undesired, given that we want the execution where p only receives this, so we fully shuffle x: We know that $?a^{q\to p} = x{\downarrow_?}$; let $ys = x{\downarrow_!}$ be the possibly existing sends of x, then obtain $(?a^{q\to p} \cdot ys) \sqcup\!\sqcup_? x$, by repeatedly shuffling $?a^{q\to p}$ further left until it cannot be shuffled anymore. Then, $?a^{q\to p} \cdot ys \in \mathcal{L}^{\emptyset}(A_q)$ by the shuffled language condition (2). Thus, $?a^{q\to p} \in \mathcal{L}^{\emptyset}(A_q)$, as all states of our network are accepting. Then $w' \in \mathsf{E}(N_{\mathtt{mbox}})$.

Inductive Step: If $w = v \cdot !a^{q\to p}$ with $v \cdot !a^{q\to p} \in \mathsf{T}(N_{\mathtt{mbox}})$, then $w' = v' \cdot !a^{q\to p} \cdot ?a^{q\to p}$. By induction, $v' \in \mathsf{E}(N_{\mathtt{mbox}})$.

To show: $w' \in \mathsf{E}(N_{\mathtt{mbox}})$.

Proof: We decompose the inductive step into the following subgoals:

(I) $(v'{\downarrow_q}) \cdot !a^{q\to p} \in \mathcal{L}^{\emptyset}(A_q)$,

(II) from this $v' \cdot !a^{q\to p} \in \mathsf{E}(N_{\mathtt{mbox}})$.

(III) Then $(v' \cdot !a^{q\to p}){\downarrow_p} \cdot ?a^{q\to p} \in \mathcal{L}^{\emptyset}(A_p)$,

(IV) hence $w' \in E_{mbox}$.

We will show these subgoals in two steps. First we show that (I) holds, and then we prove the remaining three goals together in a second step.

[14] A function in Isabelle that does this named *add_matching_recvs*, can be found in Figure 4.15.

To show: (I) holds.
Proof: We perform a case distinction over whether q is the root or not.
Case: q is the root.
Then $v'{\downarrow_q} = v{\downarrow_q}$, otherwise q would be receiving messages despite being the root. Since w is a valid trace by assumption and consequently q must be able to perform all its actions in w, we can conclude $w{\downarrow_q} = w'{\downarrow_q} = (v{\downarrow_q}) \cdot !a^{q \to p} \in \mathcal{L}^{\emptyset}(A_q)$.
Case: q is not the root.
Then q is a node with some parent r. Since w is a valid trace, there exists some $wq \in \mathcal{L}^{\emptyset}(A_q)$ and $e \in \mathsf{E}(N_{\texttt{mbox}})$, s.t. $e{\downarrow_!} = w$ and $e{\downarrow_q} = wq$. By construction, $(wq){\downarrow_!} = (v'{\downarrow_q} \cdot !a^{q \to p}){\downarrow_!}$ and in general $(v'{\downarrow_q} \cdot !a^{q \to p}){\downarrow_?} = (v'{\downarrow_q}){\downarrow_?}$, since $!a^{q \to p}$ is not an input. We infer that $(wq){\downarrow_?}$ is a prefix of $(v'{\downarrow_q}){\downarrow_?}$. If we assume the contrary, then the ordering of receptions is different in v' and wq, but since v' receives each send directly afterwards, that would mean that in wq, q receives something that is not in the first position of the buffer, contradicting the assumption that e is an execution.
Moreover, $((wq){\downarrow_?}){\downarrow_{\not!?}}$ must be a prefix of $((w{\downarrow_!}){\downarrow_{\{r,q\}}}){\downarrow_{\not!?}}$, since $((w{\downarrow_!}){\downarrow_{\{r,q\}}}){\downarrow_{\not!?}} = ((w'{\downarrow_?}){\downarrow_{\{r,q\}}}){\downarrow_{\not!?}}$ by construction of w' and we just showed that $(wq){\downarrow_?}$ is a prefix of $(v'{\downarrow_q}){\downarrow_?}$. Since $w'{\downarrow_?} = w = e{\downarrow_?}$, r sends the same messages to q in both e and w', but wq may receive only a prefix of them.
As a reminder, q is a node with parent r, and $wq \in \mathcal{L}^{\emptyset}(A_q)$, so there exists $wr' \cdot x' \in \mathcal{L}^{\emptyset}(A_r)$ such that

$$((wr' \cdot x'){\downarrow_!}){\downarrow_{\{r,q\}}} = (w{\downarrow_!}){\downarrow_{\{r,q\}}} \text{ and } ((wq){\downarrow_?}){\downarrow_{\not!?}} = (((wr'){\downarrow_!}){\downarrow_{\{r,q\}}}){\downarrow_{\not!?}}.$$

Intuitively, we obtain all sends from r to q, and decompose these into wr' and x', so that wr' provides exactly the sends that wq receives from q (since wq may not receive all sends).
By the subset condition (1), since r can perform wr' with the x', q must also be able to perform some x after wq s.t.

$$wq \cdot x \in \mathcal{L}^{\emptyset}(A_q) \text{ and } ((wq \cdot x){\downarrow_?}){\downarrow_{\not!?}} = (((wr' \cdot x'){\downarrow_!}){\downarrow_{\{q,r\}}}){\downarrow_{\not!?}}.$$

We only know about x that it contains exactly the needed receptions, but it may also contain sends. This is undesired, so we fully shuffle it: let $xs = x{\downarrow_?}$ and $ys = x{\downarrow_!}$ be the sends/receptions of x, then $(xs \cdot ys) \sqcup\!\sqcup_? x$.[15] Then we prepend wq to this shuffle, to obtain $wq \cdot xs \cdot ys \sqcup\!\sqcup_? wq \cdot xs$. Then, $wq \cdot xs \cdot ys \in \mathcal{L}^{\emptyset}(A_q)$ by the shuffled language condition (2). Thus, $wq \cdot xs \in \mathcal{L}^{\emptyset}(A_q)$ with projections

[15] Any word can be "fully" shuffled, until all receptions come first, followed by all sends of the given word. We have proven this in Isabelle, but it follows from Definition 3.2.1 relatively directly.

matching those of $w'\downarrow_q$, since all states are accepting and thus any prefix of a valid word is also valid.
We can now infer that $((v'\downarrow_q)\cdot !a^{q\to p}) \sqcup\!\sqcup_? (wq \cdot xs)$ must hold; otherwise the ordering of some send/receive action pair would contradict the trace:

> Let $!x$ and $?y$ be arbitrary send and receive actions, respectively. We write $!x <?y$ in a word w to denote that $!x$ occurs earlier (further left) in w than $?y$. Then there is some output–input pair $!x, ?y$ such that $!x <?y$ in $((v'\downarrow_q)\cdot !a)$ but $?y <!x$ in $(wq \cdot xs)$, meaning that a conflicting shuffle from wq to $(v'\downarrow_q \cdot a)$ occurred. By construction of w', we have $!x <?y$ in w, but $wq \cdot xs$ has $?y <!x$, requiring y to be sent (and to be received) before $!x$ can be sent; hence $wq \cdot xs$ cannot be part of an execution that produces trace w. However, $(e \cdot xs)$ is a valid execution (since xs only receives elements that are already in q's buffer after e and q cannot be blocked by any other peer at the end of e), and $(e \cdot xs)\downarrow_! = e\downarrow_! = w = w'\downarrow_!$ by our assumptions. But $(e \cdot xs)\downarrow_!$ has $?y <!x$ while w has $!x <?y$, yielding a different trace than assumed $\lightning$.

So $((v'\downarrow_q)\cdot !a^{q\to p}) \sqcup\!\sqcup_? (wq \cdot xs)$ holds and in turn by the shuffled language condition (2), $(v'\downarrow_q \cdot !a^{q\to p}) \in \mathcal{L}^{\lozenge}_{\sqcup\!\sqcup}(q) = \mathcal{L}^{\lozenge}(A_q)$ holds as well. This completes the proof for (I). **To show:** (II), (III) and (IV) hold.
Proof: From (I), (II) follows immediately, since sends cannot be blocked by other peers and we have shown that q can perform $(v'\downarrow_q \cdot !a^{q\to p}) \in \mathcal{L}^{\lozenge}(A_q)$, hence $v' \cdot !a^{q\to p} \in \mathsf{E}(N_{\texttt{mbox}})$. Using the subset condition (1) and shuffling the possible sends to the left again, analogously to before, (III) also follows[16]. Finally, (IV) holds because p's buffer cannot contain $a^{q\to p}$, and receptions also cannot be blocked by other peers. This concludes the inductive step; we have shown that $w' \in E_{mbox}$. Since $w'\downarrow_! = w$, then $w \in \mathsf{E}(N_{\texttt{sync}}) = \mathsf{T}(N_{\texttt{sync}})$.

Case: $w \in \mathsf{T}(N_{\texttt{sync}})$. For every output in w, $N_{\texttt{sync}}$ was able to send the respective message and directly receive it. Let w' be the word obtained from w by adding the matching receive action directly after every send action. Then $N_{\texttt{mbox}}$ can simulate the run of w in $N_{\texttt{sync}}$ by sending every message first to the mailbox of the receiver and then receiving this message. Then $w' \in \mathsf{E}(N_{\texttt{mbox}})$ and, thus, $w = w'\downarrow_! \in \mathsf{T}(N_{\texttt{mbox}})$. □

As a note, the last case ("$w \in \mathsf{T}(N_{\texttt{sync}})$") is directly cited from [DLP24], but we have omitted quotation marks for visual clarity. All other cases (for both proof directions) have changed considerably through our work.

[16] We use the same approach and reasoning as in the steps with wr' and x' earlier, where we received xs.

This concludes the main matter of this thesis. We have mechanized the necessary framework and the theorem in Isabelle, ensured a great part of its formal correctness, and improved the parts that weren't fully formal by providing and proving our adjusted version.

5. Discussion

In this section, we want to discuss some limitations of our Isabelle formalization. As alluded to before, some of the lemmas needed for the new revision of the main theorem have not yet been proven in Isabelle. In particular, we have mechanized some lemmas and assume them to be true, and use them in the main theorem proof, among others.

This works with the keyword *sorry* in Isabelle, which can be written in place of an actual proof. The *sorry* will suppress any warnings and errors Isabelle would otherwise throw if an incomplete lemma is used in other proofs. Isabelle will allow the user to use a lemma with *sorry* anywhere, and assume the lemma to be correct. This is dangerous if the lemma is not actually correct, but it works well to try out small lemmas (and see if they are necessary in the first place), without going through the time-intensive process of proving them fully.

In our case, we only apply *sorry* to lemmas that are intuitively true, or should be simple enough to prove in theory, but aren't because of our formalization of the tree topology and, in part, because of time constraints. We shall first address the lemmas we have not yet proven fully, and afterward detail why our current formalization makes these remaining proofs unnecessarily difficult. [Pau94]

5.1. Unproven Lemmas

All unproven lemmas can be found in [Ket25], by either looking through the code or the *Latex_Exports*, and searching for the word *sorry*.[1] In Isabelle, all lemmas used in a proof step are stated by name or are clear from the previous step(s). The names also function as hyperlinks[2], allowing for clear identification of the lemmas that are used in the proof and their status (proven or unproven).

Several of our unproven lemmas concern the construction *path_to_root*, which delivers a list containing all nodes on the unique path from the given node to the root. For example, *path_to_root_exists*, a lemma meant to ensure the existence of a path from any node in the tree toward the root, is still unproven. Intuitively, this is clear, as a

[1] Note that not all lemmas stated with *sorry* are actually used in the proof. We have removed or commented out most of these, but we left some in case they may be interesting for future work.

[2] By holding the *Ctrl* key and clicking on the name of the lemma (or definition, etc.).

tree is by definition connected, but as will be elaborated on in this section, it is not straightforward with our formalization.

The issues with *path_to_root* propagate themselves forward to our construction *concat_infl*, which largely relies on this path. Thus, we have similar unproven helper lemmas for this construction as well. Here, we also need to prove that the accumulated word is a valid mailbox execution, which intuitively follows directly from the construction with the influenced language.

Our other constructions *mix_pair* and *mix_shuf* are also not formally verified. For these, it also remains to be proven that their resulting accumulations (and pre- or appending certain actions) are in fact proper mailbox executions. We have explained why this is intuitively clear around Figures 4.12 and 4.13, as well as in the respective proof portions in Lemma 4.4.6.

Next are the lemmas concerning (influenced) languages, executions, etc., such as *root_lang_is_mbox*, which shows that any word in the language of the root is also a valid mailbox execution. This holds true since the root cannot depend on any receptions, and no peer can stop any other peer from sending a message. Other lemmas are prefix closure lemmas, i.e. when we assume that some $u \cdot v$ is in some language, we can gather that the prefix u alone is also in that language, as we consider all states to be final.[3]

Another group contains lemmas that move from one type of information to another. For example, *mbox_exec_to_peer_act* assumes w is a mailbox execution and $!a$ is a send-action in that word, and shows that the sender has a transition with this action in its automaton. This must hold true, since if the sender does not have a transition, there is no configuration where it can have performed $!a$, which is necessary for w to be an execution. The other lemmas in this group are similarly intuitive, but unlike humans, Isabelle needs to be provided with all the context explicitly to be able to conclude one of these steps; it is tedious to prove. In other words, these lemmas are extremely clear, knowing the global context of a network, but due to time constraints and the missing connections in our formalization, giving this context to Isabelle is quite hard.

In summary, the remaining lemmas are unproven because of the missing connections between many of the components of our formalization, which makes it hard to move from one to the other and keep the assumed properties. To detail the reason for this further, we will address some specifications of the formalization.

[3]For example, the lemmas *is_in_infl_lang_app* or *prefix_mbox_trace_valid*.

5.2. Formalization Framework

All Isabelle formalizations in this chapter adhere directly to the ones given in the original paper, and were provided to us at the beginning of this thesis [DLP24]. To begin reasoning about our perceived issues with the formalization of the (tree) topology, we first introduce those of commmunicating automata and networks, as the topology depends on them.

In Figure 5.1, we see the formalization of a communicating automaton. A *locale* in Isabelle essentially creates a context in which certain assumptions hold and certain objects may be present. It could be loosely compared to a class context in Java. We see that both the automaton and the network from Figure 5.2 contain the same info as in the related Definition 3.1.1 (and in the text above it). In other words, we also have a set of states, initial states, and so on. The conditions defined for the network, such that each message in the message set $\mathcal{M}$ must have a matching transition in some peer's automaton, are also present in the formalization. In fact, the formalizations for communicating automata and their networks adhere directly to the corresponding definitions in the original paper [DLP24].

This was initially a good choice, as the objective was to mechanize the paper exactly as it is, in Isabelle. As we will discuss further, however, Isabelle may need access to more assumptions than the ones provided in the original paper.

Figure 5.1.: Formalization of Communicating Automata

```
locale CommunicatingAutomaton =
  fixes peer        :: "'peer"
    and States      :: "'state set"
    and initial     :: "'state"
    and Messages    :: "('information, 'peer) message set"
    and Transitions :: "('state × ('information, 'peer) action × 'state) set"
  assumes finite_states:          "finite States"
    and initial_state:          "initial ∈ States"
    and message_alphabet:       "Alphabet Messages"
    and well_formed_transition: "⋀s1 a s2. (s1, a, s2) ∈ Transitions ⟹
                                 s1 ∈ States ∧ get_message a ∈ Messages ∧ get_actor a =
peer ∧
                                 get_object a ≠ peer ∧ s2 ∈ States"
```

We want to highlight the *well_formed_transition* assumption from Figure 5.1. It tells us that whenever we have a transition, there must also be a message in the message set

where our current peer is the actor, among some other properties. We can observe that there is no inverse property, i.e. we cannot draw any conclusions from the messages themselves. This trend continues in the network as well, as can be inferred from the assumption *messages_used*; it only enforces that each message must match some transition of some automaton in the network, but we cannot directly derive the concrete transition (or the set of them, if there are multiple that match). In short, both the single automaton and the network exclusively enforce implications from messages to transitions.

Figure 5.2.: Formalization of Networks of Communicating Automata

```
locale NetworkOfCA =
  fixes automata :: "'peer ⇒ ('state set × 'state ×
                ('state × ('information, 'peer) action × 'state) set)"  ("𝒜" 1000)
    and messages :: "('information, 'peer) message set"                ("ℳ" 1000)
  assumes finite_peers:      "finite (UNIV :: 'peer set)"
    and automaton_of_peer: "⋀p. CommunicatingAutomaton p (fst (𝒜 p)) (fst
(snd (𝒜 p))) ℳ
                              (snd (snd (𝒜 p)))"
    and message_alphabet:  "Alphabet ℳ"
    and peers_of_message:  "⋀m. m ∈ ℳ ⟹ get_sender m ≠ get_receiver m"
    and messages_used:     "∀m ∈ ℳ. ∃s1 a s2 p. (s1, a, s2) ∈ snd (snd (𝒜 p)) ∧
                        m = get_message a"
```

The assumption *automaton_of_peer* of the network exacerbates this even further. This assumption enforces that each peer in the network has the same set of messages, namely the one declared for the network. This mirrors the definition and usage of networks in the original paper, as can be gathered from Definition 3.1.1 and their application in [DLP24, Definition 3.2]. It is ensured that each message in the message set is used by some automaton (by the assumption *messages_used*), but to find out by whom, it is necessary to consider the transitions of all peers.

We can now showcase the topology formalizations in Figure 5.3. A topology is a graph, where the peers form the vertices and the edges are derived from the messages $\mathcal{M}$ of the network. This means the topology is formed based on the global information, i.e., based on the messages of each peer combined. This allows for easy identification of the root on the one hand (the only node that has no ingoing edge), but makes drawing conclusions back to the local information harder. As mentioned in Section 4.1, there is an edge (p, q) in the global topology, whether p sends a message to q, q receives a message from p, or both.

Figure 5.3.: Formalization of the Topology

```
type_synonym 'a topology = "('a × 'a) set"

inductive_set Edges :: "'peer topology"  ("𝒢" 110) where
  TEdge: "i^p→q ∈ ℳ ⟹ (p, q) ∈ 𝒢"

inductive is_tree :: "'peer set ⇒ 'peer topology ⇒ bool" where
  ITRoot: "is_tree {p} {}" |
  ITNode: "⟦is_tree P E; p ∈ P; q ∉ P⟧ ⟹ is_tree (insert q P) (insert (p, q) E)"

abbreviation tree_topology :: "bool" where
  "tree_topology ≡ is_tree (UNIV :: 'peer set) (𝒢)"
```

The root of the issue lies in the missing connections between an automaton's transitions, the messages in the network, and the topology. Given the message set alone, we cannot immediately infer whether a certain peer has a transition with that message, and vice versa. This is no problem in and of itself, but it makes subsequent proofs a lot more complicated.

Usually, we cannot assume to know all of these sets as a whole and have to derive them from one another. Since those factors are not connected through the formalization, Isabelle does not easily allow this, even if it might be intuitive to do so by hand. When working with a concrete example, where the automaton of each peer, all transitions, etc., are known, it is easy to draw conclusions. Since Isabelle forces us to make these conclusions generalizable, however, it is harder to do so when not explicitly connecting the different properties from the start.

Figure 5.4.: Formalization of Sends and Receptions in Peer Automata

```
inductive_set SendingToPeers :: "'peer set" where
   SPSend: "⟦(s1, a, s2) ∈ Transitions; is_output a⟧ ⟹ get_object a ∈
SendingToPeers"

inductive_set ReceivingFromPeers :: "'peer set" where
  RPRecv: "⟦(s1, a, s2) ∈ Transitions; is_input a⟧ ⟹ get_object a ∈ Receiv-
ingFromPeers"
```

We have analyzed the formalizations of communicating automata, networks, and the topology we were provided by the authors of [DLP24]. Lastly, we showcase the

construction of the sets of senders and receivers. The two relevant excerpts for this, from Figures 5.4 and 5.5, directly depend on the formalizations of communicating automata and their networks, respectively. The two notions in the following are similar to $\mathbb{P}^p_{send}$ and $\mathbb{P}^p_{rec}$ from the source paper, but are slightly different. The first one in Figure 5.4 is defined on the local level of one peer's automaton, whereas the second in Figure 5.5 is defined over the whole network. These two formalizations were also provided to us by the authors of [DLP24]. These formalizations use the transitions of the respective peer automaton to draw conclusions about the senders and receivers of a given peer. Both function the same, as the code block in Figure 5.5 simply introduces symbols to call on the formalizations from Figure 5.4.

These two notions are defined on the transitions of the given peer/automaton, which is again in contrast to the topology formalization in Figure 5.3. We detailed that communicating automata and networks composed of them only offer assumptions from which we can gather that a message has some matching transition, but not the reverse. This makes it difficult to move from an edge in the topology to a concrete transition that must exist between the two connected peers, and vice versa.

Within one peer's automaton, we may find the set of peers it sends to (or receives from) as shown in Figure 5.4 using the automaton's transitions. In the network, we then enforce that each peer automaton has the message set of the entire network, by assuming *automaton_of_peer* as seen in Figure 5.2.

The formalizations are not consistent in where the information is gathered from, and we always have the message set of the entire network at our disposal, whether in the network or a peer automaton. The issue with that is that we can only gather information about a peer's actual messages (that is, the ones where the peer is the actor) from the peer's automaton's transitions. This is because the message set of one automaton also contains messages of all other peers in the network, not necessarily involving the automaton's peer.

Figure 5.5.: Formalization of Sends and Receptions in Networks

abbreviation sendingToPeers_of_peer :: "'peer ⇒ 'peer set" ("$\mathcal{P}_!$ _" [90] 110)
where
"$\mathcal{P}_!$(p) ≡ CommunicatingAutomaton.SendingToPeers (snd (snd ($\mathcal{A}$ p)))"

abbreviation receivingFromPeers_of_peer :: "'peer ⇒ 'peer set" ("$\mathcal{P}_?$ _" [90] 110)
where
"$\mathcal{P}_?$(p) ≡ CommunicatingAutomaton.ReceivingFromPeers (snd (snd ($\mathcal{A}$ p)))"

To summarize, when examining the formalization gap with the tree topology a bit

further, it turned out to be deeper than expected. The discrepancy between the message set and the transitions of the respective automata is at the root of the issue. All peers in the network have the same set of messages, and each message in the set has at least one matching transition in some peer's automaton, but there is no possibility to directly find out where without considering all peers and all their transitions.

It is important to note that this is not an error in the mechanization; it is a direct consequence of the handwritten definition of communicating automata, as introduced in Section 3.1 or [DLP24, Section 2]. In particular, the network formalizations in Definition 3.1.1 and Figure 5.2 respectively, have the exact same specifications. This is fine on a more traditional medium, but in Isabelle, these specifications are extremely one-sided. Essentially, transitions have implications on messages, but there are no implications the other way around, because of how the network is formalized.

To conclude, to improve the formalization and make proofs in Isabelle easier, it is worth considering moving away slightly from the exact original definitions in [DLP24], and instead connecting transitions and the message sets more than is currently the case. Implicitly, all this information is there and true in the paper, but for simplifying the mechanization with Isabelle, it would need to be specified explicitly.

6. Conclusions

This work has provided another case study of why theorem provers are so instrumental. We have, in large part, formally checked in Isabelle that synchronizability for trees is in fact decidable, as was proposed in [DLP24, Theorem 4.5]. Through our mechanization of their proposed algorithm and our resulting formalizations in Isabelle, we discovered several oversights in the original theorem and related constructions, which we could then correct in multiple iterations. We have rewritten the original theorem and its proof and have thus provided a corrected algorithm to determine the synchronizability of a tree. We also provide a largely complete Isabelle formalization for the entire framework, as well as our final revision of the theorem and its proof.

It is because of the process of mechanizing the proofs of [DLP24, Section 4] in Isabelle that we were able to catch these edge cases and provide improvements. Most notably, we adjusted the definition of the influenced language (Definition 4.2.3) and the formalization of the subset condition (as seen in Theorem 4.4.5). Consequently, the entire theorem and its proof now reflect the intention of the authors formally as well. These changes to the formalization necessitated rewriting large parts of the original proof [DLP24, Theorem 4.5] entirely. In addition to delivering a new rendition of the theorem and its proof by hand, we verified the majority of our lemmas and theorems in Isabelle. There are some lemmas that remain to be fully proven in future work, but each of them is intuitively clear, as we detailed in the discussion. Because a majority of the proofs are complete and the remaining lemmas are explicitly stated in Isabelle, we are confident our new main theorem is sound. Each unproven lemma (or missing step) in the code is surrounded by comments, explaining the intuition behind it and often the proof idea as well.

We thus provide the reader with our mechanization framework and hope to help or at least inspire other researchers in the realm of distributed systems to verify their (or others') works as well.

6.1. Perspectives

The perspectives are twofold; we will first summarize those concerning our and future Isabelle formalizations, and then those concerning the results of this work in general.

As detailed in the discussion, there still remain some small lemmas to be proven to fully verify our claims. However, all of these are intuitive, and several are trivial to do on paper, but with the current formalization in Isabelle and the time constraints, we were unable to complete the proofs in Isabelle in time. We would like to fully mechanize and check [DLP24, Theorem 4.5], although proving all remaining lemmas would need additional time.

We also discussed that the current Isabelle formalization is not ideal for proving all needed lemmas. The issue lies in the encoding of the tree topology (and the formalizations the topology depends on), which does not directly yield any information about the peers and their messages (and consequently, whether they are the root or a node). This adds complexity to any proof in need of these relations, which is why some intuitive lemmas are still unproven. In other words, from the tree topology as it is currently defined in our Isabelle formalization, we cannot directly infer any information about the peers, especially concerning their messages or place in the topology. This formalization was present before this work began; thus, we continued with it and built the rest of the Isabelle framework around it. Consequently, it is now difficult to adapt this tree topology definition without affecting the rest of the code. We have the following dilemma: not all lemmas can be easily proven because the tree topology's formalization in Isabelle is not ideal; however, changing this formalization means having to, in the worst case, rewrite or reprove large parts of definitions, lemmas, etc., where the tree topology currently occurs. In short, it would be best to formalize a stronger tree topology and maybe strengthen other definitions (such as the one for communicating automata and networks) as well in Isabelle to establish a bidirectional relationship between messages and transitions, which can then be used for the missing proof steps in some of the unproven lemmas. The formalization could be adjusted in several places in future work.

We have described how the initial formalization may be improved to allow for easier proving, but our creations are also open to critique. In part due to the tree topology formalization, we had some issues formalizing the construction used in the proof of the original [DLP24, Lemma 4.4] in the source paper (see Definition 4.4.2). We used an overcomplicated and slightly inefficient construct[1]. While we have provided a possible improvement[2], we were not able to check its correctness and move to this version fully, in time. In general, there are a plethora of formalizations that we would now do differently with the new knowledge we gathered through this thesis, and which can surely be improved by other Isabelle enthusiasts as well.

With or without an improved Isabelle formalization, another worthwhile endeavor

[1]The inductive *concat_infl* and all related lemmas.
[2]In the code directly above *concat_infl*, named *acc_infl_lang-word*.

would be to formalize the counterexamples we have found in Isabelle. We started this process, but have encountered similar difficulties as in the process of proving the remaining helper lemmas, which is why their formalization was not completed. However, formalizing the counterexamples would contribute to the goal of fully verifying this entire project and underline the validity of our examples.[3]

Lastly, the result from Theorem 4.4.5 could also possibly be extended to both Peer-to-Peer networks as well as networks with different variants of tree topologies, as was previously mentioned in the source paper. To reiterate their points, networks consisting of multiple or reversed trees may be able to be investigated in terms of their synchronizability, with a similar approach to our end result here. Since we have found a procedure to decide the synchronizability of one tree, one could apply this to each tree of a multitree network. For both of these variants, our requirement of having pairwise unique paths between nodes is still satisfied. To extend this procedure to reversed trees, some of the procedure would likely need to be reversed to account for the change in properties. Additionally, the authors claim that the set of executions of a network with tree topology should be equal for mailbox and Peer-to-Peer systems. This remains to be formally proven and hopefully verified, however. In short, we have found the synchronizability problem to be decidable for tree topologies, and propose a similar approach for investigating the decidability of this problem in related contexts. [DLP24]

To add to the previous point: we have provided a mechanization of trees and the remaining framework needed to reason about synchronizability on trees. Our mechanization can likely be adapted easily to allow for the formalization of multitrees, etc., since they are also tree-related. In short, one could explore the synchronizability of tree-related topologies and machine-check them by adapting our mechanizations slightly.

We are confident that we have achieved a good foundation of the last perspective proposed in [DLP24], namely the mechanization of the paper's proofs in Isabelle, despite its challenging nature. With this foundation and the future prospects detailed above, we hope to inspire the reader to continue improving this project or to delve into their own verification endeavors. This work has been an excellent case study on why theorem provers such as Isabelle are a great tool to assess the formal correctness of scientific work. These tools are instrumental to ensure our intuition is correct and nothing is missed, but without humans, there would be no proofs to check.

[3] We have a rough skeleton of one of the counterexamples in *CounterExamples.thy*, as well as some of the older revisions of the subset condition.

A. Appendix

A.1. Affiliations

The people and organisms involved in this Master's thesis, from and beyond the TUM, are detailed in the following. This work was conducted as an end-of-studies internship as part of my Learning Agreement during my Erasmus+ mobility in the full academic year 2024/25. In France, such an internship is akin to a Master's thesis in other European countries and typically occurs at the end of the second year of a Master's study program. This internship was for a full duration of six months at the Laboratory of Computer Science, Signals and Systems (I3S) laboratory.

It was supervised by Prof. Di Giusto, Prof. Peters, and Prof. Esparza; the first two of whom are co-authors of the paper which forms the basis for this work [DLP24]. Prof. Di Giusto[1] is a professor at the Université Côte d'Azur and the I3S, in the Team MDSC/C&A[2], who focus their research on applying formal methods to topics of different kinds. Professor Peters[3] is a professor in the field of concurrent systems at the University of Augsburg and proficient in Isabelle.

The internship was originally divided into two parts: the familiarization with Isabelle and the mechanization of the proofs in [DLP24, Section 4].

A.2. Pseudocode for *mix_pair*

This section contains supplemental pseudocode matching the Isabelle formalization shown for *mix_pair* in Figure 4.12.

```
1 Let w ∈ 𝓛^∅(A_p) and w′ ∈ 𝓛^∅(A_q) s.t. ((w′↓!)↓{p,q})↓⊬? = (w↓?)↓⊬?.
2 Let a denote an arbitrary message (from q to p).
3
4 w_mix ← [ ]
5 i ← 0, j ← 0
6
```

[1] https://webusers.i3s.unice.fr/~cdigiusto/web
[2] https://ca.i3s.unice.fr
[3] https://www.uni-augsburg.de/en/fakultaet/fai/informatik/prof/swtti/team/kirstin-peters

```
while i < |w| and j < |w'|:
    if w'[j] = !a^{q→p}:
        if w[i] = ?a^{q→p}:
            append !a^{q→p}, ?a^{q→p} to w_mix
            i ← i + 1,  j ← j + 1
        else:
            while w[i] ≠ ?a^{q→p}:
                append w[i] to w_mix
                i ← i + 1
    else:
        append w'[j] to w_mix
        j ← j + 1
return w_mix
```

Listing A.1: Pseudocode for `mix_pair`, formalized in Isabelle and shown in Appendix A.1.

A.3. Isabelle Mechanization Excerpts

This section contains some excerpts from our full Isabelle mechanization, available at [Ket25]. The LaTeX code for the showcased instances here was gathered using the export functionality of Isabelle. The full export (as a PDF), as well as its sources, can be referenced in the repository, inside the folder *Latex_Exports*. Where possible and helpful, we have put the excerpts throughout the text of the main body of this thesis, but some of the code highlighted here is too long to include in that way.

We strongly encourage the reader to reference the actual code (ideally with Isabelle), as the LaTeX exports below do not include multiline comments, of which there are many. Most steps (especially the ones using helper lemmas or ones with *sorry*) are thoroughly explained in countless comments. Additionally, all lemmas, etc., that we have not showcased in the main body or here can easily be found as the references (e.g., with *using*) in Isabelle always link to the respective source. Nevertheless, we provide the most important excerpts in this thesis for completeness.

A.3.1. Lemma 4.4 in Isabelle

The code below contains the mechanization of [DLP24, Lemma 4.4], which we restated in Lemma 4.4.2. The proof below uses the construction *concat_infl* and several lemmas about its properties. Some of these are not proven fully, as mentioned in Chapter 5.

```
lemma lemma4_4 :
  fixes w :: "('information, 'peer) action word"
    and q :: "'peer"
  assumes "tree_topology" and "w ∈ ℒ*(q)" and "q ∈ 𝒫"
  shows "∃ w'. (w' ∈ 𝒯None ∧ w'↓q = w ∧ ((is_parent_of p q) ⟶ w'↓p = ε))"

 using assms
 proof (cases "is_root q")
  case True — q = r
  then have "w ∈ ℒ(q)" using assms(2) is_in_infl_lang.cases by blast
  then have "w = w↓!" by (meson NetworkOfCA.no_inputs_implies_only_sends_alt
NetworkOfCA_axioms True assms(1) global_to_local_root
      p_root)
  then have "w↓? = ε" by (simp add: output_proj_input_yields_eps)
  then have t2: "w = w↓q" by (simp add: ‹w ∈ ℒ q› w_in_peer_lang_impl_p_actor)
  then have "∀ p. p ≠ q ⟶ w↓p = ε" by (metis only_one_actor_proj)
  then have t3: "(∀p. (p ∈ 𝒫 ∧ 𝒫?(p) = {q}) ⟶ w↓p = ε)" by (metis True assms(1)
global_to_local_root insert_not_empty )
  then have "w ∈ ℒ(q)" by (simp add: ‹w ∈ ℒ q›)
  then have "is_root q" using True by auto
  then have "w ∈ 𝒯None" using ‹w ∈ ℒ q› root_lang_is_mbox by auto
  have "w↓q = w" using t2 by auto
  then have "(is_parent_of p q ⟶ w↓p = ε)" by (metis True is_parent_of_rev(2)
iso_tuple_UNIV_I only_one_actor_proj root_defs_eq t3)
  then show ?thesis by (metis ‹w ∈ 𝒯None› t2)
 next
  case False
  then obtain p where q_parent: "is_parent_of q p" by (metis UNIV_I assms(1)
path_to_root.cases path_to_root_exists)
  then obtain ps where p2root: "path_to_root p (p # ps)" by (metis UNIV_I assms(1)
path_to_root_exists path_to_root_rev)
  then have "is_node q" by (metis is_parent_of.cases q_parent)
  have "w ∈ ℒ*(q)" using assms(2) by auto
  then have "is_parent_of q p" by (simp add: q_parent)

  then have "∃w'. w'∈ ℒ* p ∧ ((w↓?)↓!?) = (((w'↓{q,p})↓!)↓!?)" using assms(2)
infl_parent_child_matching_ws by blast
  then obtain w' where w'_w: "((w↓?)↓!?) = (((w'↓{q,p})↓!)↓!?)" and w'_Lp: "w' ∈ ℒ*
p" by blast
```

```
  then have "w' ∈ ℒ p" by (meson mem_Collect_eq w_in_infl_lang)
  have "tree_topology"  using assms(1) by auto
  have c1: "((w↓?)↓!?) = (((w'↓{q,p})↓!)↓!?) ∧ w ∈ ℒ(q) ∧ w' ∈ ℒ(p) ∧ is_node q" using ‹is_parent_of q p›
    ‹is_tree (𝒫) (𝒢) ∧ (∃ qa. 𝒢⟨→q⟩ = {qa}) ∨ is_tree (𝒫) (𝒢) ∧ (∃ qa. 𝒫? q = {qa} ∨ q ∈ 𝒫! qa)› ‹w' ∈ ℒ p›
    assms(2) is_in_infl_lang_rev2(1) w'_w by blast

  obtain r where "is_root r" using assms(1) root_exists by blast
  have "path_to_root q (q # p # ps)" using p2root p2root_down_step q_parent by auto
  then have "concat_infl q w (q # p # ps) w"  using assms(1,2) at_p by auto
  have "w ∈ ℒ(q)"  by (simp add: c1)
  then have "w↓q = w"  using w_in_peer_lang_impl_p_actor by auto
  obtain w_acc where "concat_infl q w [r] w_acc" by (meson ‹concat_infl q w (q # p # ps) w›
      ‹is_tree (𝒫) (𝒢) ∧ 𝒫? r = {} ∧ (∀ q. r ∉ 𝒫! q) ∨ is_tree (𝒫) (𝒢) ∧ 𝒢⟨→r⟩ = {}›
      concat_infl_word_exists)
  then have "w_acc ∈ 𝒯None" by (simp add: concat_infl_mbox)
  have "w_acc↓q = w"  using ‹concat_infl q w (r # ε) w_acc› concat_infl_actor_consistent by blast
  then have "(∀ p. (is_parent_of p q) ⟶  w_acc↓p = ε)"  using ‹concat_infl q w (r # ε) w_acc› concat_infl_children_not_included by blast
  then show ?thesis using ‹w_acc ∈ 𝒯None› ‹w_acc↓q = w› by blast
 qed
```

A.3.2. Main Theorem in Isabelle

This section contains the entirety of the mechanization for Theorem 4.4.5. Note that all names we have not defined in the thesis but that are mentioned throughout this proof, belong to some smaller lemmas that are declared elsewhere in the code, which is not included in this Appendix. All lemmas we applied are of course present in the full code at [Ket25], and are located mainly in *CommunicatingAutomaton.thy*.

Note that the theorem formalization and proof relies on a large lemma which performs much of the proof steps for the second direction "⇐", which is stated in Section A.3.3.

```
 theorem synchronisability_for_trees:
  assumes "tree_topology"
```

```
  shows "is_synchronisable ⟷ ((∀ p ∈ 𝒫. ∀ q ∈ 𝒫. ((is_parent_of p q) ⟶ ((subset_condition
p q) ∧ ((ℒ*(p)) = (ℒ*␣␣(p)))) ))" (is "?L ⟷ ?R")
 proof
  assume "?L"
  show "?R"
  proof clarify
   fix p q
   assume q_parent: "is_parent_of p q"
   have sync_def: "𝒯_None↓! = ℒ₀" using ‹?L› by simp
   show "subset_condition p q ∧ ℒ* p = ℒ*␣␣ p"
   proof (rule conjI)

    show "subset_condition p q" unfolding subset_condition_def
    proof auto

     fix w w' x'
     assume w_Lp: "is_in_infl_lang p w"
      and w'_Lq: "is_in_infl_lang q w'"
      and w'_w_match: "filter (λx. is_output x ∧ (get_object x = q ∧ get_actor x = p
            ∨ get_object x = p ∧ get_actor x = q)) w'↓!? = w↓?↓!?"
      and w'x'_Lq: "is_in_infl_lang q (w' ▪ x')"
     then show "∃wa. filter (λx. is_output x ∧ (get_object x = q ∧ get_actor x = p ∨
get_object x = p ∧
              get_actor x = q)) x'↓!? =  wa↓!? ∧  (∃x. wa = x↓? ∧ is_in_infl_lang p (w ▪
x))"
      using w_Lp  w'_Lq w'_w_match w'x'_Lq
     proof (cases "is_root q")
      case True
      then have "(w' ▪ x') ∈ ℒ q" using w'x'_Lq w_in_infl_lang by auto
      then have "(w' ▪ x') ∈ 𝒯_None" using root_lang_is_mbox True by blast
      have "w'↓!↓{p,q}↓!? = w↓?↓!?" using w'_w_match by force
      let ?mix = "(mix_pair w' w [])"
      have "?mix ▪ x' ∈ 𝒯_None" sorry
      then obtain t where "t ∈ ℒ₀ ∧ t ∈ 𝒯_None↓! ∧ t = (?mix ▪ x')↓!" using sync_def
by fastforce
     then obtain xc where t_sync_run : "sync_run 𝒞_ℐ0 t xc" using SyncTraces.simps
by auto
```

then have "∃xcm. mbox_run $\mathcal{C}_{\mathcal{I}\mathtt{m}}$ None (add_matching_recvs t) xcm" **using** empty_sync_run_to_mbox_run sync_run_to_mbox_run **by** blast

then have sync_exec: "(add_matching_recvs t) ∈ $\mathcal{T}_{\mathtt{None}}$" **using** MboxTraces.intros **by** auto

then have "∃x. (add_matching_recvs t)$\downarrow_p$ = w ▪ x" **sorry**
then obtain x **where** x_def: "(add_matching_recvs t)$\downarrow_p$ = w ▪ x" **by** blast
then have w′x′_wx_match: "(w′ ▪ x′)$\downarrow_!\downarrow_{\{p,q\}}\downarrow_{!?}$ = (w ▪ x)$\downarrow_?\downarrow_{!?}$" **sorry**
have "(w ▪ x) ∈ $\mathcal{L}^*$ p" **using** sync_exec x_def **by** (metis mbox_exec_to_infl_peer_word)
have "∃wa. x′$\downarrow_!\downarrow_{\{p,q\}}\downarrow_{!?}$ = wa$\downarrow_{!?}$ ∧ (∃x. wa = x$\downarrow_?$ ∧ is_in_infl_lang p (w ▪ x))" **using** ‹w ▪ x ∈ $\mathcal{L}^*$ p› ‹w′$\downarrow_!\downarrow_{\{p,q\}}\downarrow_{!?}$ = w$\downarrow_?\downarrow_{!?}$› w′x′_wx_match **by** auto
then show ?thesis **by** simp
next
case False
then have "is_node q" **by** (metis NetworkOfCA.root_or_node NetworkOfCA_axioms assms)
then obtain r **where** qr: "is_parent_of q r" **by** (metis False UNIV_I path_from_root.simps path_to_root_exists paths_eq)

have "(w′ ▪ x′) ∈ $\mathcal{L}^*$ q" **by** (simp add: w′x′_Lq)
then have "∃w′′. w′′ ∈ $\mathcal{L}^*$(r) ∧ (((w′ ▪ x′)$\downarrow_?$)$\downarrow_{!?}$) = (((w′′$\downarrow_{\{q,r\}}$)$\downarrow_!$)$\downarrow_{!?}$)"
using infl_parent_child_matching_ws[of "(w′ ▪ x′)" q r] **using** qr **by** blast
then obtain w′′ **where** w′′_w′_match: "w′′$\downarrow_!\downarrow_{\{q,r\}}\downarrow_{!?}$ = (w′ ▪ x′)$\downarrow_?\downarrow_{!?}$" **and** w′′_def: "w′′ ∈ $\mathcal{L}^*$ r" **by** (metis (no_types, lifting) filter_pair_commutative)

have "∃ e. (e ∈ $\mathcal{T}_{\mathtt{None}}$ ∧ e$\downarrow_r$ = w′′ ∧ ((is_parent_of q r) ⟶ e$\downarrow_q$ = ε))" **using** lemma4_4[of
w′′ r q] **using** ‹w′′ ∈ $\mathcal{L}^*$ r› assms **by** blast
then obtain e **where** e_def: "e ∈ $\mathcal{T}_{\mathtt{None}}$" **and** e_proj_r: "e$\downarrow_r$ = w′′"
and e_proj_q: "e$\downarrow_q$ = ε" **using** qr **by** blast

let ?mix = "(mix_pair w′ w [])"

have "e ▪ ?mix ▪ x′ ∈ $\mathcal{T}_{\mathtt{None}}$" **sorry**

then obtain t **where** "t ∈ $\mathcal{L}_{\mathbf{0}}$ ∧ t ∈ $\mathcal{T}_{\mathtt{None}}\downarrow_!$ ∧ t = (e ▪ ?mix ▪ x′)$\downarrow_!$" **using** sync_def **by** fastforce

then obtain xc **where** t_sync_run : "sync_run $\mathcal{C}_{\mathcal{I}\mathbf{0}}$ t xc" **using** SyncTraces.simps **by** auto

then have "∃xcm. mbox_run $\mathcal{C}_{\mathcal{I}\mathtt{m}}$ None (add_matching_recvs t) xcm" **using** empty_sync_run_to_mbox_run sync_run_to_mbox_run **by** blast

then have sync_exec: "(add_matching_recvs t) ∈ $\mathcal{T}_{\mathtt{None}}$" **using** MboxTraces.intros **by** auto

then have "∃x. (add_matching_recvs t)$\downarrow_p$ = w ▪ x" **sorry**
then obtain x **where** x_def: "(add_matching_recvs t)$\downarrow_p$ = w ▪ x" **by** blast
then have w'x'_wx_match: "(w' ▪ x')$\downarrow_!\downarrow_{\{p,q\}}\downarrow_{!?}$ = (w ▪ x)$\downarrow_?\downarrow_{!?}$" **sorry**

have "(w ▪ x) ∈ $\mathcal{L}^*$ p" **using** sync_exec x_def **by** (metis mbox_exec_to_infl_peer_word)
have "w'$\downarrow_!\downarrow_{\{p,q\}}\downarrow_{!?}$ = w$\downarrow_?\downarrow_{!?}$" **using** w'_w_match **by** force
have "∃wa. x'$\downarrow_!\downarrow_{\{p,q\}}\downarrow_{!?}$ = wa$\downarrow_{!?}$ ∧ (∃x. wa = x$\downarrow_?$ ∧ is_in_infl_lang p (w ▪ x))" **using** ‹w ▪ x ∈ $\mathcal{L}^*$ p› ‹w'$\downarrow_!\downarrow_{\{p,q\}}\downarrow_{!?}$ = w$\downarrow_?\downarrow_{!?}$› w'x'_wx_match **by** auto
then show ?thesis **by** simp
qed
qed

show "$\mathcal{L}^*$(p) = $\mathcal{L}^*{}_{\sqcup\sqcup}$(p)"
proof

show "$\mathcal{L}^*$(p) ⊆ $\mathcal{L}^*{}_{\sqcup\sqcup}$(p)" **using** language_shuffle_subset **by** auto

show "$\mathcal{L}^*{}_{\sqcup\sqcup}$(p) ⊆ $\mathcal{L}^*$(p)"
proof
fix v'

assume "v' ∈ $\mathcal{L}^*{}_{\sqcup\sqcup}$(p)"
then obtain v **where** v_orig: "v' ⊔⊔$_?$ v" **and** orig_in_L: "v ∈ $\mathcal{L}^*$(p)" **using** shuffled_infl_lang_impl_valid_shuffle **by** auto
then show "v' ∈ $\mathcal{L}^*$(p)"
proof (induct v v')
case (refl w)
then show ?case **by** simp
next
case (swap b a w xs ys)

then have "∃ vq. vq ∈ $\mathcal{L}^*$(q) ∧ ((w↓?)↓!?) = (((vq↓{p,q})↓!)↓!?)"
using infl_parent_child_matching_ws[of w p q] orig_in_L q_parent **by** blast
then obtain vq **where** vq_v_match: "((w↓?)↓!?) = (((vq↓{p,q})↓!)↓!?)" **and** vq_def: "vq ∈ $\mathcal{L}^*$ q" **by** auto
have lem4_4_prems: "tree_topology ∧ w ∈ $\mathcal{L}^*$(p) ∧ p ∈ $\mathcal{P}$" **using** assms swap.prems **by** auto
then show ?case **using** assms swap vq_v_match vq_def lem4_4_prems
proof (cases "is_root q")
case True
have "vq ∈ $\mathcal{L}$ q" **using** vq_def w_in_infl_lang **by** auto
then have "vq ∈ $\mathcal{T}_{None}$" **using** root_lang_is_mbox True **by** simp

let ?w′ = "xs ▪ a # b # ys"
have "∃ acc. mix_shuf vq v v′ acc" **sorry**
then obtain mix **where** "mix_shuf vq v v′ mix" **by** blast
let ?mix = "mix"
have "?mix ∈ $\mathcal{T}_{None}$" **sorry**
then obtain t **where** "t ∈ $\mathcal{L}_0$ ∧ t ∈ $\mathcal{T}_{None}$|! ∧ t = (?mix)↓!" **using** sync_def **by** fastforce
then obtain xc **where** t_sync_run : "sync_run $\mathcal{C}_{\mathcal{I}0}$ t xc" **using** SyncTraces.simps **by** auto

then have "∃ xcm. mbox_run $\mathcal{C}_{\mathcal{I}m}$ None (add_matching_recvs t) xcm" **using** empty_sync_run_to_mbox_run sync_run_to_mbox_run **by** blast

then have sync_exec: "(add_matching_recvs t) ∈ $\mathcal{T}_{None}$" **using** MboxTraces.intros **by** auto

then have "(add_matching_recvs t)↓$_p$ = ?w′" **sorry**
then have "?w′ ∈ $\mathcal{L}^*$ p" **using** sync_exec mbox_exec_to_infl_peer_word **by** metis
then show ?thesis **by** simp
next
case False
then have "is_node q" **by** (metis NetworkOfCA.root_or_node NetworkOfCA_axioms assms)
then obtain r **where** qr: "is_parent_of q r" **by** (metis False UNIV_I path_from_root.simps path_to_root_exists paths_eq)

then have "∃ vr. vr ∈ $\mathcal{L}^*$(r) ∧ ((vq↓$_?$)↓$_{!?}$) = (((vr↓$_{\{q,r\}}$)↓$_!$)↓$_{!?}$)"
using infl_parent_child_matching_ws[of vq q r] orig_in_L vq_def **by** blast

then obtain vr **where** vr_def: "vr ∈ $\mathcal{L}^*$(r)" **and** vr_vq_match: "((vq↓$_?$)↓$_{!?}$) = (((vr↓$_{\{q,r\}}$)↓$_!$)↓$_{!?}$)" **by** blast

have "∃ e. (e ∈ $\mathcal{T}_{None}$ ∧ e↓$_r$ = vr ∧ ((is_parent_of q r) ⟶ e↓$_q$ = ε))" **using** lemma4_4[of
vr r q] **using** ‹vr ∈ $\mathcal{L}^*$ r› assms **by** blast
then obtain e **where** e_def: "e ∈ $\mathcal{T}_{None}$" **and** e_proj_r: "e↓$_r$ = vr"
and e_proj_q: "e↓$_q$ = ε" **using** qr **by** blast

let ?w′ = "xs ▪ a # b # ys"
have "∃ acc. mix_shuf vq v v′ acc" **sorry**
then obtain mix **where** "mix_shuf vq v v′ mix" **by** blast
let ?mix = "mix"

have "e ▪ ?mix ∈ $\mathcal{T}_{None}$" **sorry**

then obtain t **where** "t ∈ $\mathcal{L}_0$ ∧ t ∈ $\mathcal{T}_{None}$⌊$_!$ ∧ t = (e ▪ ?mix)↓$_!$" **using** sync_def **by** fastforce
then obtain xc **where** t_sync_run : "sync_run $\mathcal{C}_{\mathcal{I}\mathbf{0}}$ t xc" **using** SyncTraces.simps **by** auto

then have "∃ xcm. mbox_run $\mathcal{C}_{\mathcal{I}\mathtt{m}}$ None (add_matching_recvs t) xcm" **using** empty_sync_run_to_mbox_run sync_run_to_mbox_run **by** blast

then have sync_exec: "(add_matching_recvs t) ∈ $\mathcal{T}_{None}$" **using** Mbox-Traces.intros **by** auto

then have "(add_matching_recvs t)↓$_p$ = ?w′" **sorry**
then have "?w′ ∈ $\mathcal{L}^*$ p" **using** sync_exec mbox_exec_to_infl_peer_word **by** metis
then show ?thesis **by** simp
qed
next
case (trans w w′ w′′)
then show ?case **by** simp
qed

```
      qed
    qed
  qed
qed

next

  assume "?R"
  show "?L"
  proof auto — cases: w in TMbox, w in TSync
    fix w
    show "w ∈ 𝒯_None ⟹ w↓! ∈ ℒ₀"
    proof -
      assume "w ∈ 𝒯_None"
      then have "(w↓!) ∈ 𝒯_None↓!" by blast

      then have match_exec: "add_matching_recvs (w↓!) ∈ 𝒯_None"
      using mbox_trace_with_matching_recvs_is_mbox_exec ‹∀p∈𝒫. ∀q∈𝒫. is_parent_of
p q ⟶ subset_condition p q ∧ ℒ* p = ℒ*_⊔⊔ p› assms theorem_rightside_def
        by blast
      then obtain xcm where "mbox_run 𝒞_𝐼𝔪 None (add_matching_recvs (w↓!)) xcm"
by (metis MboxTraces.cases)
     then show "(w↓!) ∈ ℒ₀" using SyncTraces.simps ‹w↓! ∈ 𝒯_None↓!› matched_mbox_run_to_sync_run
by blast
    qed
  next — w in TSync –> show that w' (= w with recvs added) is in EMbox
    fix w
    show "w ∈ ℒ₀ ⟹ ∃w'. w = w'↓! ∧ w' ∈ 𝒯_None"
    proof -
      assume "w ∈ ℒ₀"
       — For every output in w, Nsync was able to send the respective message and
directly receive it
      then have "w = w↓!" by (metis SyncTraces.cases run_produces_no_inputs(1))
      then obtain xc where w_sync_run : "sync_run 𝒞_𝐼₀ w xc" using SyncTraces.simps
‹w ∈ ℒ₀› by auto
      then have "w ∈ ℒ∞"  using ‹w ∈ ℒ₀› mbox_sync_lang_unbounded_inclusion by
blast
      obtain w' where "w' = add_matching_recvs w" by simp
```

— then Nmbox can simulate the run of w in Nsync by sending every message first to the mailbox of the receiver and then receiving this message
then show ?thesis
proof (cases "xc = []") — this case distinction isn't in the paper but i need it here to properly get the simulated run
case True
then have "mbox_run $\mathcal{C}_{\mathcal{I}\mathtt{m}}$ None (w') []" **using** ‹w' = add_matching_recvs w› empty_sync_run_to_mbox_run w_sync_run **by** auto
then show ?thesis **using** ‹w ∈ $\mathcal{T}_{None}\downarrow_!$› **by** blast
next
case False
then obtain xcm **where** sim_run: "mbox_run $\mathcal{C}_{\mathcal{I}\mathtt{m}}$ None w' xcm ∧ (∀ p. (last xcm p) = ((last xc) p, ε))"
using ‹w' = add_matching_recvs w› sync_run_to_mbox_run w_sync_run **by** blast
then have "w' ∈ $\mathcal{T}_{None}$" **using** MboxTraces.intros **by** auto
then have "w = w'$\downarrow_!$" **using** ‹w = w$\downarrow_!$› ‹w' = add_matching_recvs w› adding_recvs_keeps_send_order **by** auto
then have "(w'$\downarrow_!$) ∈ $\mathcal{L}_\infty$" **using** ‹w' ∈ $\mathcal{T}_{None}$› **by** blast
then show ?thesis **using** ‹w = w'$\downarrow_!$› ‹w' ∈ $\mathcal{T}_{None}$› **by** blast
qed
qed
qed
qed

A.3.3. Formalization of Main Theorem Helper Lemma

The second direction of the main theorem proof above (in Section A.3.2 is relatively short. This is because the main matter of (⇐ 1.) is achieved with the following lemma:

lemma mbox_trace_with_matching_recvs_is_mbox_exec:
assumes "w ∈ $\mathcal{T}_{None}\downarrow_!$" **and** "tree_topology" **and** "theorem_rightside"
shows "(add_matching_recvs w) ∈ $\mathcal{T}_{None}$"
using assms
proof (induct "length w" arbitrary: w)
case 0
then show ?case **by** (simp add: eps_in_mbox_execs)
next
case (Suc n)

then obtain v a **where** w_def: "w = v ▪ [a]" **and** v_len: "length v = n" **by** (metis length_Suc_conv_rev)
then have "v ∈ $\mathcal{T}_{None}\downarrow_!$" **using** Suc.prems(1) prefix_mbox_trace_valid **by** blast
then have v_IH_prems: "n = ∣v∣ ∧ v ∈ $\mathcal{T}_{None}\downarrow_!$ ∧ is_tree ($\mathcal{P}$) ($\mathcal{G}$) ∧ theorem_rightside" **using** Suc.prems(3) assms(2) v_len **by** blast
let ?v′ = "add_matching_recvs v"
have v_IH: "?v′ ∈ $\mathcal{T}_{None}$" **using** v_IH_prems Suc **by** blast
have "(v ▪ [a]) = (v ▪ [a])$\downarrow_!$" **using** Suc.prems(1) w_def **by** fastforce
then obtain i p q **where** a_def: "a = (!⟨(i$^{q\to p}$)⟩)" **by** (metis Nil_is_append_conv append1_eq_conv decompose_send neq_Nil_conv)
then have "∃ s1 s2 . (s1, !⟨(i$^{q\to p}$)⟩, s2) ∈ $\mathcal{R}$ q" **using** Suc.prems(1) assms(2) mbox_exec_to_peer_act w_def **by** auto
then have "p ∈ $\mathcal{P}_!$(q)" **by** (metis CommunicatingAutomaton.SendingToPeers.intros automaton_of_peer get_message.simps(1)
is_output.simps(1) message.inject output_message_to_act_both_known trans_to_edge)
then have "$\mathcal{G}\langle\to$p⟩ = {q}" **by** (simp add: assms(2) local_parent_to_global)
then have pq: "is_parent_of p q" **using** assms **by** (simp add: node_parent)
have "(?v′)$\downarrow_q$ ∈ $\mathcal{L}^*$ q" **using** mbox_exec_to_infl_peer_word v_IH **by** auto
have w_sends_0: "w = ((?v′) ▪ [a])$\downarrow_!$" **by** (metis ‹v ▪ a # ε = (v ▪ a # ε)$\downarrow_!$› adding_recvs_keeps_send_order filter_append w_def)
then have w_sends_1: "w = (?v′)$\downarrow_!$ ▪ [a]" **using** ‹v ∈ $\mathcal{T}_{None}\downarrow_!$› adding_recvs_keeps_send_order w_def **by** fastforce
have a_facts: "is_output a ∧ get_actor a = q ∧ get_object a = p ∧ p ≠ q" **using** a_def is_output.simps(1) **by** (simp add: ‹is_parent_of p q› parent_child_diff)
then have "[a]$\downarrow_q$ = [a]" **by** simp
have "[a]$\downarrow_?$ = ε" **using** a_def a_facts **by** simp
have v′_q_recvs_inv_to_a: "(?v′$\downarrow_q$)$\downarrow_?$ = ((?v′ ▪ [a])$\downarrow_q$)$\downarrow_?$" **using** ‹(a # ε)$\downarrow_?$ = ε› **by** auto

have "p ∈ $\mathcal{P}$ ∧ q ∈ $\mathcal{P}$" **by** simp
then have "(is_parent_of p q) ⟶ ((subset_condition p q) ∧ (($\mathcal{L}^*$(p)) = ($\mathcal{L}^*{}_{\sqcup\sqcup}$(p))))" **using** assms(3) theorem_rightside_def **by** blast
then have theorem_right_pq: "((subset_condition p q) ∧ (($\mathcal{L}^*$(p)) = ($\mathcal{L}^*{}_{\sqcup\sqcup}$(p))))" **using** pq **by** auto

then have a_send_ok: "(?v′ ▪ [a]) ∈ $\mathcal{T}_{None}$" **using** a_def Suc assms
proof (cases "is_root q")
case True

then have "(v$\downarrow_q$ ▪ [!⟨(i$^{q \to p}$)⟩]) ∈ ($\mathcal{L}^*$(q))" **using** mbox_trace_to_root_word[of v i q p] **using** Suc.prems(1) a_def w_def **by** fastforce

have "?v'$\downarrow_q$ = (?v'$\downarrow_q$)$\downarrow_!$" **using** root_infl_word_no_recvs[of q "?v'$\downarrow_q$"] **using** True ‹add_matching_recvs v$\downarrow_q$ ∈ $\mathcal{L}^*$ q› **by** presburger
then have "?v'$\downarrow_q$ ▪ [a] ∈ $\mathcal{L}^*$ q" **by** (metis (no_types, lifting) ‹v$\downarrow_q$ ▪ !⟨(i$^{q \to p}$)⟩ # ε ∈ $\mathcal{L}^*$ q› ‹w = add_matching_recvs v$\downarrow_!$ ▪ a # ε› a_def
append1_eq_conv filter_pair_commutative w_def)
show ?thesis **using** mbox_exec_app_send[of q "?v'" a] **using** ‹add_matching_recvs v$\downarrow_q$ ▪ a # ε ∈ $\mathcal{L}^*$ q› a_facts v_IH **by** linarith
next
case False

obtain e **where** e_def: "e ∈ $\mathcal{T}_{None}$" **and** e_trace: "e$\downarrow_!$ = w" **using** Suc.prems(1) **by** blast
then obtain wq **where** wq_def: "wq = e$\downarrow_q$" **and** wq_in_q: "wq ∈ $\mathcal{L}^*$ q" **using** mbox_exec_to_infl_peer_word **by** presburger

have v'a0: "((?v')$\downarrow_q$ ▪ [a])$\downarrow_!$ = ((?v')$\downarrow_q$)$\downarrow_!$ ▪ [a]$\downarrow_!$" **by** simp
have v'a1: "((?v')$\downarrow_q$)$\downarrow_!$ ▪ [a]$\downarrow_!$ = ((?v')$\downarrow_q$)$\downarrow_!$ ▪ [a]" **using** a_facts **by** simp
then have v'a2: "((?v')$\downarrow_q$)$\downarrow_!$ ▪ [a] = v$\downarrow_q$ ▪ [a]" **by** (smt (verit, ccfv_threshold) ‹v ▪ a # ε = (v ▪ a # ε)$\downarrow_!$› adding_recvs_keeps_send_order append1_eq_conv filter_append filter_pair_commutative same_append_eq)
have "wq$\downarrow_!$ = w$\downarrow_q$" **using** e_trace filter_pair_commutative wq_def **by** blast
have wq_v'_sends: "wq$\downarrow_!$ = ((?v')$\downarrow_!$ ▪ [a])$\downarrow_q$" **using** ‹w = add_matching_recvs v$\downarrow_!$ ▪ a # ε› ‹wq$\downarrow_!$ = w$\downarrow_q$› **by** fastforce
have v'a3: "((?v')$\downarrow_!$ ▪ [a])$\downarrow_q$ = ((?v')$\downarrow_!$)$\downarrow_q$ ▪ [a]$\downarrow_q$" **by** simp
have v'a4: "((?v')$\downarrow_!$)$\downarrow_q$ ▪ [a]$\downarrow_q$ = ((?v')$\downarrow_q$)$\downarrow_!$ ▪ [a]$\downarrow_q$" **using** filter_pair_commutative **by** blast
have "[a]$\downarrow_q$ = [a]" **using** a_def **by** simp
have wq_to_v'a_trace: "wq$\downarrow_!$ = ((?v')$\downarrow_q$)$\downarrow_!$ ▪ [a]" **using** ‹(a # ε)$\downarrow_q$ = a # ε› v'a3 v'a4 wq_v'_sends **by** argo

have "is_node q" **by** (metis False NetworkOfCA.root_or_node NetworkOfCA_axioms assms(2))
then obtain r **where** "is_parent_of q r" **by** (metis False UNIV_I path_to_root.cases path_to_root_exists)

```
    have v'_recvs_match: "(((?v'↓!)↓r)↓{q,r})↓!? = (((add_matching_recvs ((?v'↓!))↓?)↓q)↓!?)"
using matching_recvs_word_matches_sends_explicit[of "?v'" q r] using ‹is_parent_of q
r› v_IH by simp
     then have "(((?v'↓!)↓r)↓{q,r})↓!? = (((?v'↓?)↓q)↓!?)" using ‹w = add_matching_recvs
v↓! ▪ a # ε› w_def by fastforce
    then have wr_0: "(((?v'↓!)↓r)↓{q,r})↓!? = (((?v'↓q)↓?)↓!?)" by (metis filter_pair_commutative)
    then have e_pref:"prefix (((e↓q)↓?)↓!?) ((((e↓r)↓!)↓{q,r})↓!?)" using peer_recvs_in_exec_is_prefix_of_pare
e q r] using ‹is_parent_of q r› e_def by linarith
     then have wq_e_pref: "prefix (((wq)↓?)↓!?) ((((e↓r)↓!)↓{q,r})↓!?)" using wq_def by
fastforce
     have e_trace2: "(e↓!) = ((?v' ▪ [a])↓!)" using ‹w = (add_matching_recvs v ▪ a # ε)↓!›
e_trace by blast
     then have "prefix (((wq)↓?)↓!?) (((((?v' ▪ [a])↓r)↓!)↓{q,r})↓!?)"  by (metis (no_types,
lifting) e_pref filter_pair_commutative
         wq_def)
     have "((((?v' ▪ [a])↓!)↓{q,r})↓!?) = ((((?v')↓!)↓{q,r})↓!?) ▪ (((([a])↓!)↓{q,r})↓!?)" by simp
     have "(((([a])↓!)↓{q,r})↓!?) = ((([a])↓{q,r})↓!?)" using a_facts by simp
     have "r ≠ q" using ‹is_parent_of q r› parent_child_diff by blast
     have "p ≠ q" by (simp add: a_facts)
     have "r ≠ p" proof (rule ccontr)
      assume "¬ r ≠ p"
      then have "r = p" by simp
      then have "is_parent_of q p" using ‹is_parent_of q r› by auto
      then have g1: "𝒢⟨→q⟩ = {p}" using is_parent_of_rev by simp
      then have e1: "(p, q) ∈ 𝒢" by auto
      have g2: "𝒢⟨→p⟩ = {q}" using pq is_parent_of_rev by simp
      then have e2: "(q, p) ∈ 𝒢" by auto
      show "False" using tree_acyclic[of "𝒫" "𝒢" p q] using assms(2) e1 e2 by auto
     qed
     have "[a]↓{q,r} = ε" using a_facts using ‹r ≠ p› by auto
     then have "((([a])↓{q,r})↓!?) = (ε)↓!? " using a_facts by simp
     then have "((((?v' ▪ [a])↓!)↓{q,r})↓!?) = ((((?v')↓!)↓{q,r})↓!?)" by simp
    have "((((?v' ▪ [a])↓!)↓{q,r})↓!?) = (((e↓!)↓{q,r})↓!?)" using ‹e↓! = (add_matching_recvs
v ▪ a # ε)↓!› by presburger
     then have "(((e↓!)↓{q,r})↓!?) = ((((?v')↓!)↓{q,r})↓!?)"  using ‹(add_matching_recvs v
▪ a # ε)↓!↓{q,r}↓!? = add_matching_recvs v↓!↓{q,r}↓!?›
      by argo
    have v'_match: "((((((?v')↓!)↓r)↓{q,r})↓!?) = ((((?v')↓?)↓q)↓!?)" using ‹w = add_matching_recvs
v↓! ▪ a # ε› v'_recvs_match w_def by force
```

then have e_v'_match:"((((e$\downarrow_!$)$\downarrow_r$)$\downarrow_{\{q,r\}}$)$\downarrow_{!?}$) = ((((?v')$\downarrow_?$)$\downarrow_q$)$\downarrow_{!?}$)" **using** ‹(a # ε)$\downarrow_{\{q,r\}}$ = ε› ‹w = add_matching_recvs v$\downarrow_!$ ▪ a # ε› e_trace **by** force

then have wq_recvs_pref: "prefix (((wq)$\downarrow_?$)$\downarrow_{!?}$) ((((?v')$\downarrow_?$)$\downarrow_q$)$\downarrow_{!?}$)" **by** (metis filter_pair_commutative wq_e_pref)

have v'_proj_inv: " ((((?v')$\downarrow_?$)$\downarrow_q$)$\downarrow_{!?}$) = ((((?v')$\downarrow_q$)$\downarrow_?$)$\downarrow_{!?}$)" **by** (metis filter_pair_commutative)

then have wq_recvs_prefix: "prefix (wq$\downarrow_?$) (((?v')$\downarrow_q$)$\downarrow_?$)" **by** (metis wq_recvs_pref filter_recursion no_sign_recv_prefix_to_sign_inv)

have "((((((?v' ▪ [a])$\downarrow_!$)$\downarrow_r$)$\downarrow_{\{q,r\}}$)$\downarrow_{!?}$) = ((((?v' ▪ [a])$\downarrow_?$)$\downarrow_q$)$\downarrow_{!?}$)" **by** (metis (no_types, lifting) e_trace2 e_v'_match filter_pair_commutative v'_q_recvs_inv_to_a)

have "prefix (wq$\downarrow_?$) (((?v' ▪ [a])$\downarrow_q$)$\downarrow_?$)" **using** v'_q_recvs_inv_to_a wq_recvs_prefix **by** presburger

have wq_pref_of_rq_sends: "prefix (((wq)$\downarrow_?$)$\downarrow_{!?}$) ((((((?v')$\downarrow_!$)$\downarrow_r$)$\downarrow_{\{q,r\}}$)$\downarrow_{!?}$)" **using** v'_match wq_recvs_pref **by** argo

then have "prefix (((wq)$\downarrow_?$)$\downarrow_{!?}$) ((((?v'$\downarrow_r$)$\downarrow_!$)$\downarrow_{\{q,r\}}$)$\downarrow_{!?}$)" **by** (metis filter_pair_commutative)

have v'_match_alt: "((((((?v')$\downarrow_r$)$\downarrow_!$)$\downarrow_{\{q,r\}}$)$\downarrow_{!?}$) = ((((?v')$\downarrow_q$)$\downarrow_?$)$\downarrow_{!?}$)" **by** (metis (no_types, lifting) filter_pair_commutative v'_match)

then have "∃wr'. prefix wr' ((?v')$\downarrow_r$) ∧ (((wr'$\downarrow_!$)$\downarrow_{\{q,r\}}$)$\downarrow_{!?}$) = (((wq)$\downarrow_?$)$\downarrow_{!?}$) ∧ wr' ∈ $\mathcal{L}^*$ r"

using match_exec_and_child_prefix_to_parent_match[of r q "?v'" wq] ‹is_parent_of q r› v_IH wq_recvs_prefix **by** blast

then obtain wr' x' **where** v'r_def: "((?v')$\downarrow_r$) = wr' ▪ x'" **and** wr'_match: "(((wr'$\downarrow_!$)$\downarrow_{\{q,r\}}$)$\downarrow_{!?}$) = (((wq)$\downarrow_?$)$\downarrow_{!?}$)" **and** "wr' ∈ $\mathcal{L}^*$ r" **by** (meson prefixE)

have "((?v')$\downarrow_r$) ∈ $\mathcal{L}^*$ r" **using** mbox_exec_to_infl_peer_word[of "?v'" r] **using** v_IH **by** blast

then have "wr' ▪ x' ∈ $\mathcal{L}^*$ r" **by** (simp add: v'r_def)

have "q ∈ $\mathcal{P}$ ∧ r ∈ $\mathcal{P}$" **by** simp

then have "(is_parent_of q r) ⟶ ((subset_condition q r) ∧ (($\mathcal{L}^*$(q)) = ($\mathcal{L}^*{}_{\sqcup\sqcup}$(q))))" **using** assms(3) theorem_rightside_def **by** blast

then have theorem_right_qr: "((subset_condition q r) ∧ (($\mathcal{L}^*$(q)) = ($\mathcal{L}^*{}_{\sqcup\sqcup}$(q))))" **by** (simp add: ‹is_parent_of q r›)

have "∃x. (wq ▪ x) ∈ $\mathcal{L}^*$ q ∧ (((wq ▪ x)$\downarrow_?$)$\downarrow_{!?}$) = ((((wr' ▪ x')$\downarrow_!$)$\downarrow_{\{q,r\}}$)$\downarrow_{!?}$)" **using** subset_cond_from_child_prefix_and_parent[
of q r wq wr' x'] **using** ‹wr' ▪ x' ∈ $\mathcal{L}^*$ r› theorem_right_qr wq_in_q wr'_match **by** fastforce

then obtain x **where** wqx_def: "(wq ▪ x) ∈ $\mathcal{L}^*$ q" **and** wqx_match: "(((wq ▪ x)↓?)↓!?) = ((((wr′ ▪ x′)↓!)↓{q,r})↓!?)" **by** auto
then have wqx_match_v′: "(((wq ▪ x)↓?)↓!?) = (((((?v′ ▪ [a])↓!)↓r)↓{q,r})↓!?)" **using** e_trace2 e_v′_match v′_match_alt v′_proj_inv v′r_def **by** argo

then obtain xs ys **where** x_shuf: "(xs ▪ ys) ⧢? x" **and** "xs↓? = xs" **and** "ys↓! = ys" **using** full_shuffle_of **by** blast
then have xsys_recvs: "(((wq ▪ (xs ▪ ys))↓?)↓!?) = (((((?v′ ▪ [a])↓!)↓r)↓{q,r})↓!?)" **by** (metis (mono_tags, lifting) filter_append shuffled_keeps_recv_order wqx_match_v′)
have "(wq ▪ xs ▪ ys) ⧢? (wq ▪ x)" **using** x_shuf shuffle_prepend **by** auto
then have "wq ▪ xs ▪ ys ∈ $\mathcal{L}^*$ q" **by** (metis UNIV_def ‹is_parent_of q r› ‹wq ▪ x ∈ $\mathcal{L}^*$ q› assms(3) input_shuffle_implies_shuffled_lang
mem_Collect_eq theorem_rightside_def)
then have wqxs_L: "wq ▪ xs ∈ $\mathcal{L}^*$ q" **using** local.infl_word_impl_prefix_valid **by** simp
have "(wq ▪ xs)↓! = wq↓!" **by** (simp add: ‹xs↓? = xs› input_proj_output_yields_eps)
have wqx_match_v′a: "((((?v′ ▪ [a])↓q)↓?)↓!?) = (((wq ▪ x)↓?)↓!?)" **using** e_trace2 e_v′_match v′_proj_inv v′_q_recvs_inv_to_a wqx_match_v′ **by** presburger
have "xs↓? = (xs ▪ ys)↓?" **by** (simp add: ‹ys↓! = ys› output_proj_input_yields_eps)
have "(xs ▪ ys)↓? = (x)↓?" **using** x_shuf **by** (metis shuffled_keeps_recv_order)
then have "xs↓? = (x)↓?" **using** ‹xs↓? = (xs ▪ ys)↓?› **by** presburger
have "(((wq ▪ x)↓?)↓!?) = (((wq ▪ xs)↓?)↓!?)" **by** (simp add: ‹xs↓? = x↓?›)
then have xs_recvs: "(((wq ▪ xs)↓?)↓!?) = (((((?v′ ▪ [a])↓!)↓r)↓{q,r})↓!?)" **using** wqx_match_v′ wqx_match_v′a **by** argo
have v′_eq: "(((((?v′ ▪ [a])↓!)↓r)↓{q,r})↓!?) = (((?v′ ▪ [a])↓q)↓?)↓!?" **using** e_trace2 e_v′_match v′_proj_inv v′_q_recvs_inv_to_a **by** presburger
then have "(((wq ▪ xs)↓?)↓!?) = (((?v′ ▪ [a])↓q)↓?)↓!?" **using** xs_recvs **by** presburger
then have "(wq ▪ xs)↓? = (((?v′ ▪ [a])↓q)↓?)" **using** no_sign_recv_prefix_to_sign_inv[of "(wq ▪ xs)↓?" "(((?v′ ▪ [a])↓q)↓?)"] **by** (metis filter_recursion no_sign_recv_prefix_to_sign_inv prefix_order.dual_order.antisym
prefix_order.dual_order.refl)
then have wqxs_order:"(wq ▪ xs)↓? = (((?v′ ▪ [a])↓q)↓?) ∧ (wq ▪ xs)↓! = ((?v′ ▪ [a])↓q)↓!" **using** ‹(a # ε)↓q = a # ε› ‹(wq ▪ xs)↓! = wq↓!› w_sends_0 w_sends_1 wq_to_v′a_trace **by** force
have wqxs_trace_match: "(((wq ▪ xs)↓?)↓!?) = (((((v ▪ [a])↓!)↓r)↓{q,r})↓!?)" **using** ‹v ▪ a # ε = (v ▪ a # ε)↓!› e_trace e_trace2 w_def xs_recvs **by** presburger
let ?wq = "wq ▪ xs"
show ?thesis **using** wqxs_order
proof (cases "(?v′↓q ▪ [a]) ⧢? (?wq)")

case True
then have "(?v'↓$_q$ ▪ [a]) ∈ ($\mathcal{L}^*{}_{⧢⧢}$(q))" **using** input_shuffle_implies_shuffled_lang wqxs_L **by** blast
then have "(?v'↓$_q$ ▪ [a]) ∈ ($\mathcal{L}^*$(q))" **using** theorem_right_qr **by** simp
then show ?thesis **using** mbox_exec_app_send[of q "?v'" a] **using** a_facts v_IH **by** blast
next
case False
then have "(?v'↓$_q$ ▪ [a]) ≠ (?wq)" **by** (metis shuffled.refl)

then have "¬ ((?v'↓$_q$ ▪ [a]) ⧢⧢$_?$ (?wq))" **using** False **by** blast
then have "¬ ((?v'↓$_q$ ▪ [a]) ⧢⧢$_?$ (?wq)) ∧ (wq ▪ xs)↓$_?$ = (((?v' ▪ [a])↓$_q$)↓$_?$) ∧ (wq ▪ xs)↓$_!$ = ((?v' ▪ [a])↓$_q$)↓$_!$"
using wqxs_order **by** blast

then have "∃ xs' a' ys' b' zs' xs'' ys'' zs''. is_input a' ∧ is_output b' ∧ (?wq) = (xs' @ [a'] @ ys' @ [b'] @ zs') ∧
(?v'↓$_q$ ▪ [a]) = (xs'' @ [b'] @ ys'' @ [a'] @ zs'')" **using** no_shuffle_implies_output_input_exists[of

"?wq" "(?v'↓$_q$ ▪ [a])"] **by** (metis ‹(a # ε)↓$_q$ = a # ε› filter_append)

have diff_trace_prems: "?wq↓$_?$ = (?v'↓$_q$ ▪ [a])↓$_?$ ∧ ?wq↓$_!$ = (?v'↓$_q$ ▪ [a])↓$_!$ ∧
¬((?v'↓$_q$ ▪ [a]) ⧢⧢$_?$?wq) ∧ ?wq ≠ (?v'↓$_q$ ▪ [a])
∧ e ∈ $\mathcal{T}_{None}$ ∧ ?v' ∈ $\mathcal{T}_{None}$ ∧ (v ▪ [a]) ∈ $\mathcal{T}_{None}{\downarrow_!}$ ∧ ?v' = (add_matching_recvs v) ∧ ?v'↓$_q$ ∈ $\mathcal{L}^*$ q
∧ ?wq ∈ $\mathcal{L}^*$ q"
by (metis (no_types, lifting) False Suc.prems(1) ‹(a # ε)↓$_q$ = a # ε› ‹(wq ▪ xs)↓$_!$ = wq↓$_!$›
‹((wq ▪ xs)↓$_?$) = ((add_matching_recvs v ▪ a # ε)↓$_q$)↓$_?$› ‹add_matching_recvs v↓$_q$ ▪ a # ε ≠ wq ▪ xs›
‹add_matching_recvs v↓$_q$ ∈ $\mathcal{L}^*$ q› e_def filter_append v'a1 v_IH w_def wq_to_v'a_trace wqxs_L)

have "(e ▪ xs) ∈ $\mathcal{T}_{None}$" **using** exec_append_missing_recvs[of wq xs r q v a e] **using** diff_trace_prems wq_def wqxs_trace_match
e_trace w_def **by** blast
have "(e ▪ xs)↓$_q$ = e↓$_q$ ▪ xs↓$_q$" **by** simp
have "xs↓$_q$ = xs" **using** infl_word_actor_app **by** (meson wqxs_L)
then have "(e ▪ xs)↓$_q$ = ?wq" **using** ‹(e ▪ xs)↓$_q$ = e↓$_q$ ▪ xs↓$_q$› wq_def **by** presburger

have "(e ▪ xs)↓! = e↓!" **by** (simp add: ‹xs↓? = xs› input_proj_output_yields_eps)
have diff_trace_prems2: "?wq↓? = (?v'↓$_q$ ▪ [a])↓? ∧ ?wq↓! = (?v'↓$_q$ ▪ [a])↓! ∧
¬((?v'↓$_q$ ▪ [a]) ⊔⊔? ?wq) ∧ ?wq ≠ (?v'↓$_q$ ▪ [a])
∧ (e ▪ xs) ∈ $\mathcal{T}_{None}$ ∧ (e ▪ xs)↓$_q$ = ?wq ∧ ?v' ∈ $\mathcal{T}_{None}$ ∧ (v ▪ [a]) ∈ $\mathcal{T}_{None}$↓! ∧ ?v' =
(add_matching_recvs v) ∧ ?v'↓$_q$ ∈ $\mathcal{L}^*$ q
∧ ?wq ∈ $\mathcal{L}^*$ q" **using** ‹(e ▪ xs)↓$_q$ = wq ▪ xs› ‹e ▪ xs ∈ $\mathcal{T}_{None}$› diff_trace_prems **by** blast
then have "(e ▪ xs)↓! ≠ (?v' ▪ [a])↓!" **using** diff_peer_word_impl_diff_trace
[of "?wq" q "?v'" a "(e ▪ xs)" v] **by** simp
then show ?thesis **using** ‹(e ▪ xs)↓! = e↓!› e_trace2 **by** argo
qed
qed
then have "((add_matching_recvs v)↓$_q$ @ [a]↓$_q$) ∈ $\mathcal{L}^*$ q" **using** mbox_exec_to_infl_peer_word
by fastforce
then have q_full: "((add_matching_recvs v)↓$_q$ @ [!⟨(i$^{q\to p}$)⟩]) ∈ $\mathcal{L}^*$ q" **using** a_def **by**
simp
have v'p_in_L: "(add_matching_recvs v)↓$_p$ ∈ $\mathcal{L}^*$ p" **using** mbox_exec_to_infl_peer_word
v_IH **by** blast

have v'_recvs_match_pq: "(((?v'↓!)↓$_q$)↓$_{\{p,q\}}$)↓!? = (((add_matching_recvs ((?v'↓!))↓?)↓$_p$)↓!?)"

using matching_recvs_word_matches_sends_explicit[of "?v'" p q] **using** ‹is_parent_of
p q› v_IH **by** simp
then have v'_recvs_match_pq2: "(((?v'↓!)↓$_q$)↓$_{\{p,q\}}$)↓!? = (((?v'↓?)↓$_p$)↓!?)" **using** ‹w =
add_matching_recvs v↓! ▪ a # ε› w_def **by** fastforce
let ?wp = "(?v'↓$_p$)"
let ?wq_full = "((add_matching_recvs v)↓$_q$ @ [!⟨(i$^{q\to p}$)⟩])"

have "(?wp ▪ [?⟨(i$^{q\to p}$)⟩]) ∈ $\mathcal{L}^*$ p ∧ (((?wp ▪ [?⟨(i$^{q\to p}$)⟩])↓?)↓!?) = ((((?wq_full)↓!)↓$_{\{p,q\}}$)↓!?)"
using subset_cond_from_child_prefix_and_parent_act[of p q "?wp" "(?v'↓$_q$)" i] **by**
(smt (verit, ccfv_SIG) filter_pair_commutative pq q_full theorem_right_pq v'_recvs_match_pq2
v'p_in_L)
then have "(((?v')↓$_p$ ▪ [(?⟨(i$^{q\to p}$)⟩)])) ∈ $\mathcal{L}^*$ p" **by** simp

then have a_ok0: "(?v' ▪ ([!⟨(i$^{q\to p}$)⟩] ▪ [?⟨(i$^{q\to p}$)⟩])) ∈ $\mathcal{T}_{None}$"
using mbox_exec_recv_append[of "?v'" i q p] **using** a_def a_send_ok **by** (metis
(no_types, lifting) append1_eq_conv append_assoc filter_pair_commutative pq v'_recvs_match_pq
w_def
w_sends_1)

have a_match: "(add_matching_recvs [a]) = ([!⟨(i$^{q \to p}$)⟩] ▪ [?⟨(i$^{q \to p}$)⟩])" **using** a_def **by** force
then have a_ok: "((add_matching_recvs v) ▪ (add_matching_recvs [a])) ∈ $\mathcal{T}_{None}$" **using** a_ok0 **by** auto

then show ?case **by** (simp add: add_matching_recvs_app w_def)
qed

Abbreviations

TUM Technical University of Munich

ATPs Automatic Theorem Provers

SMTs Satisfiability-Modulo-Theories

I3S Laboratory of Computer Science, Signals and Systems

FIFO First In First Out

List of Figures

Bibliography

[Bar+22] H. Barbosa, C. Barrett, M. Brain, G. Kremer, H. Lachnitt, M. Mann, A. Mohamed, M. Mohamed, A. Niemetz, A. Nötzli, A. Ozdemir, M. Preiner, A. Reynolds, Y. Sheng, C. Tinelli, and Y. Zohar. "cvc5: A Versatile and Industrial-Strength SMT Solver." In: *Tools and Algorithms for the Construction and Analysis of Systems*. Ed. by D. Fisman and G. Rosu. Cham: Springer International Publishing, 2022, pp. 415–442. ISBN: 978-3-030-99524-9.

[BB16] S. Basu and T. Bultan. "On deciding synchronizability for asynchronously communicating systems." In: *Theoretical Computer Science* 656 (Dec. 2016), pp. 60–75. ISSN: 0304-3975. DOI: 10.1016/j.tcs.2016.09.023.

[BJ19] M. Buyse and J. Jaskolka. "Communicating Concurrent Kleene Algebra for Distributed Systems Specification." In: *Archive of Formal Proofs* (Aug. 2019). https://isa-afp.org/entries/C2KA_DistributedSystems.html, Formal proof development. ISSN: 2150-914x.

[Bla+25] J. Blanchette, M. Desharnais, L. C. Paulson, and L. Bartl. *Hammering Away - A User's Guide to Sledgehammer for Isabelle/HOL*. Mar. 2025. URL: https://isabelle.in.tum.de/dist/Isabelle2025/doc/sledgehammer.pdf.

[Bol+21] B. Bollig, C. Di Giusto, A. Finkel, L. Laversa, É. Lozes, and A. Suresh. "A Unifying Framework for Deciding Synchronizability." In: *32nd International Conference on Concurrency Theory, CONCUR 2021, August 24-27, 2021, Virtual Conference*. Ed. by S. Haddad and D. Varacca. Vol. 203. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021, 14:1–14:18. DOI: 10.4230/LIPICS.CONCUR.2021.14.

[Bou+18] A. Bouajjani, C. Enea, K. Ji, and S. Qadeer. "On the Completeness of Verifying Message Passing Programs Under Bounded Asynchrony." In: *Computer Aided Verification - 30th International Conference, CAV 2018, Held as Part of the Federated Logic Conference, FloC 2018, Oxford, UK, July 14-17, 2018, Proceedings, Part II*. Ed. by H. Chockler and G. Weissenbacher. Vol. 10982. Lecture Notes in Computer Science. Springer, 2018, pp. 372–391. DOI: 10.1007/978-3-319-96142-2_23.

[BP10] J. C. Blanchette and L. C. Paulson. "Three Years of Experience with Sledgehammer, a Practical Link between Automatic and Interactive Theorem Provers." In: *PAAR-2010: Workshop on Practical Aspects of Automated Reasoning*. Talk given by Jasmin Christian Blanchette. An updated version appeared in G. Sutcliffe, E. Ternovska, and S. Schulz (eds.), 8th International Workshop on the Implementation of Logics (IWIL-2010), Yogyakarta, Indonesia, October 2010. Edinburgh, Scotland, July 2010.

[BZ83] D. Brand and P. Zafiropulo. "On Communicating Finite-State Machines." In: *Journal of the ACM* 30.2 (Apr. 1983), pp. 323–342. ISSN: 0004-5411. DOI: 10.1145/322374.322380.

[CHQ16] F. Chevrou, A. Hurault, and P. Quéinnec. "On the diversity of asynchronous communication." In: *Formal Aspects Comput.* 28.5 (2016), pp. 847–879. DOI: 10.1007/S00165-016-0379-X.

[CMT96] B. Charron-Bost, F. Mattern, and G. Tel. "Synchronous, Asynchronous, and Causally Ordered Communication." In: *Distributed Comput.* 9.4 (1996), pp. 173–191. DOI: 10.1007/S004460050018.

[Di +23] C. Di Giusto, D. Ferré, L. Laversa, and É. Lozes. "A Partial Order View of Message-Passing Communication Models." In: *Proc. ACM Program. Lang.* 7.POPL (2023), pp. 1601–1627. DOI: 10.1145/3571248.

[DLL20] C. Di Giusto, L. Laversa, and É. Lozes. "On the k-synchronizability of Systems." In: *Foundations of Software Science and Computation Structures - 23rd International Conference, FOSSACS 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020, Proceedings*. Ed. by J. Goubault-Larrecq and B. König. Vol. 12077. Lecture Notes in Computer Science. Springer, 2020, pp. 157–176. DOI: 10.1007/978-3-030-45231-5_9.

[DLL23] C. Di Giusto, L. Laversa, and E. Lozes. "Guessing the Buffer Bound for k-Synchronizability." In: *International Journal of Foundations of Computer Science* 34.08 (2023), pp. 1051–1076. DOI: 10.1142/S0129054122430018.

[DLP24] C. Di Giusto, L. Laversa, and K. Peters. "Synchronisability in Mailbox Communication." In: *Electronic Proceedings in Theoretical Computer Science* 412 (Nov. 2024). arXiv:2411.14580 [cs], pp. 19–34. ISSN: 2075-2180. DOI: 10.4204/EPTCS.412.3.

[DMS25] R. Delpy, A. Muscholl, and G. Sutre. "On the Send-Synchronizability Problem for Mailbox Communication." In: *36th International Conference on Concurrency Theory (CONCUR 2025)*. Ed. by P. Bouyer and J. van de Pol. Vol. 348.

Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 15:1–15:20. ISBN: 978-3-95977-389-8. DOI: 10.4230/LIPIcs.CONCUR.2025.15.

[FL17] A. Finkel and E. Lozes. "Synchronizability of Communicating Finite State Machines is not Decidable." In: *44th International Colloquium on Automata, Languages, and Programming (ICALP 2017)*. Ed. by I. Chatzigiannakis, P. Indyk, F. Kuhn, and A. Muscholl. Vol. 80. Leibniz International Proceedings in Informatics (LIPIcs). ISSN: 1868-8969. Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2017, 122:1–122:14. ISBN: 978-3-95977-041-5. DOI: 10.4230/LIPIcs.ICALP.2017.122.

[FT20] B. Fiedler and D. Traytel. "A Formal Proof of The Chandy–Lamport Distributed Snapshot Algorithm." In: *Archive of Formal Proofs* (July 2020). https://isa-afp.org/entries/Chandy_Lamport.html, Formal proof development. ISSN: 2150-914x.

[GKM06] B. Genest, D. Kuske, and A. Muscholl. "A Kleene theorem and model checking algorithms for existentially bounded communicating automata." In: *Information and Computation* 204.6 (June 2006), pp. 920–956. ISSN: 0890-5401. DOI: 10.1016/j.ic.2006.01.005.

[Ket25] N. Kettler. *isabelle-sync*. 2025. URL: https://github.com/nicolell/isabelle-sync.

[KM21] D. Kuske and A. Muscholl. "Communicating automata." In: *Handbook of Automata Theory*. Ed. by J. Pin. European Mathematical Society Publishing House, Zürich, Switzerland, 2021, pp. 1147–1188. DOI: 10.4171/AUTOMATA-2/9.

[Lam78] L. Lamport. "Time, Clocks, and the Ordering of Events in a Distributed System." In: *Commun. ACM* 21.7 (1978), pp. 558–565. DOI: 10.1145/359545.359563.

[Lin14] F. Lindblad. "A Focused Sequent Calculus for Higher-Order Logic." In: *Automated Reasoning*. Ed. by S. Demri, D. Kapur, and C. Weidenbach. Cham: Springer International Publishing, 2014, pp. 61–75. ISBN: 978-3-319-08587-6.

[Lip11] M. Lipovaca. *Learn You a Haskell for Great Good!: A Beginner's Guide*. No Starch Press, Apr. 2011. ISBN: 978-1-59327-295-1.

[Mil+97] R. Milner, M. Tofte, R. Harper, and D. MacQueen. *The Definition of Standard ML: Revised*. MIT Press, 1997. ISBN: 978-0-262-63181-5.

[Nip+] T. Nipkow, M. Wenzel, C. Sternagel, and M. Eberl. *HOL/Library/Sublist.thy*. URL: https://isabelle.in.tum.de/dist/library/HOL/HOL-Library/Sublist.html.

[Nip23] T. Nipkow. *Concrete Semantics: With Isabelle/HOL*. Jan. 2023. URL: http://www.concrete-semantics.org/slides-isabelle.pdf.

[NK23] T. Nipkow and G. Klein. *Concrete Semantics: With Isabelle/HOL*. http://www.concrete-semantics.org/concrete-semantics.pdf. Springer International Publishing, Feb. 2023.

[NPW25] T. Nipkow, L. C. Paulson, and M. Wenzel. *A Proof Assistant for Higher-Order Logic*. https://isabelle.in.tum.de/doc/tutorial.pdf. Springer, Mar. 2025.

[Pau88] L. C. Paulson. *The foundation of a generic theorem prover*. Tech. rep. UCAM-CL-TR-130. University of Cambridge, Computer Laboratory, Mar. 1988. DOI: 10.48456/tr-130.

[Pau94] L. C. Paulson. *Isabelle: A Generic Theorem Prover*. Vol. 828. Lecture Notes in Computer Science. Springer, 1994. DOI: 10.1007/BFb0030541.

[Wen25] M. Wenzel. *The Isabelle/Isar Reference Manual*. https://isabelle.in.tum.de/doc/isar-ref.pdf. Springer, Mar. 2025.